

Autenticación con Protocolo Zero-Knowledge-Proof basado en Learning with Errors

Autores:

Cesar Alejandro Avellaneda

Juan Esteban Álvarez

Universidad de los Andes

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas y Computación

Bogotá, Colombia

2026

Autenticación con Protocolo Zero-Knowledge-Proof basado en Learning with Errors

Tesis presentada por: Cesar Alejandro Avellaneda
Juan Esteban Álvarez *Como requisito parcial para optar al título de:*
Ingeniero de Sistemas y Computación

Tutores:
Sandra Julieta Rueda Rodriguez
Milton Emigdio Quiroga Becerra

Universidad de los Andes
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas y Computación
Bogotá, Colombia

2026

Índice

1. Introducción **7**
1.1. Generalidades 7
1.2. Contexto de la Situación 7
1.3. Justificación 8
1.4. Objetivos 8
1.4.1. Objetivo General 8
1.4.2. Objetivos Específicos 9

2. Marco Teórico **9**
2.1. Notación 9
2.2. Zero-Knowledge Proofs (ZKP) 11
2.2.1. Fundamentos Matemáticos y Propiedades de las Pruebas de Conocimiento Cero 13
2.2.2. Construcciones de ZKP sobre Retículos: De Stern a Lyubashevsky y el Framework LNP 18
2.2.3. El Obstáculo del Rebobinado Cuántico (Quantum Rewinding Barrier) 22
2.2.4. Propiedades de Privacidad: No Vinculabilidad y Denegabilidad 25
2.2.5. Protección del Canal de Comunicación: Token Hiding con ML-KEM y AES-GCM 26
2.2.6. Estado del Arte Comparativo de Frameworks ZKP Poscuánticos . . . 29
2.3. Learning With Errors (LWE) 32
2.3.1. Introducción a la Criptografía Basada en Retículos 32
2.3.2. El Problema Learning with Errors (LWE) Estándar 33
2.3.3. Variantes Algebraicas y Estructurales del Problema LWE 36
2.3.4. Muestreo y Gestión del Ruido: De la Distribución Gaussiana a la Distribución Binomial Centrada (CBD) 39
2.3.5. Mecanismos de Reconciliación de Errores y Mapeo Criptográfico . . . 42
2.3.6. Estimación Concreta de Seguridad 44
2.3.7. Elección del Módulo: Pruebas, Cifrado y Multiplicación Rápida . . . 47

3. Propuesta del Protocolo de Autenticación Poscuántica: ML-ZKAuth **49**
3.1. Visión General y Modelo de Amenazas 49
3.2. Justificación de las Decisiones de Diseño 51
3.2.1. Arquitectura de Módulo Dual 54
3.3. Elementos del Protocolo: Identificador, Ticket y Marca de Tiempo 55

3.4. Vector de Parámetros y su Obtención 56
3.5. Configuración del Sistema y Enrolamiento 59
3.6. Flujo de Mensajes 60
3.7. Generación de la Prueba 61
3.8. Ocultación del Token 64
3.9. Verificación en el Servidor 65
3.10. Acuerdo de Clave de Sesión 67
3.11. Análisis de Seguridad 68
3.12. Eventos Improbables, Comparación y Amenazas Prácticas 73
3.13. Correspondencia con la Implementación 78

4. Evaluación experimental y benchmarking de protocolos de autenticación 79
4.1. Objetivos de la evaluación 79
4.2. Principio de comparabilidad 80
4.2.1. Registro 80
4.2.2. Autenticación 80
4.2.3. Separación entre registro y autenticación 81
4.3. Protocolos evaluados 81
4.4. Entorno experimental 83
4.5. Configuración criptográfica 85
4.5.1. Tamaños y costos analíticos de ML-ZKAuth 86
4.6. Diseño de los experimentos 87
4.6.1. Número de ejecuciones 87
4.6.2. Estadísticos reportados 88
4.7. Métricas evaluadas 88
4.7.1. Rendimiento computacional 88
4.7.2. Consumo de memoria y almacenamiento 88
4.7.3. Comunicación 88
4.7.4. Seguridad 89
4.7.5. Características operacionales 89
4.8. Resultados de rendimiento 90
4.8.1. Tiempo de generación de claves 90
4.8.2. Tiempo de autenticación 90
4.8.3. Distribución de los tiempos 91
4.9. Resultados de consumo de memoria 93
4.10. Tamaño de claves y estructuras criptográficas 94

4.11. Resultados de comunicación 94
4.12. Escalabilidad de almacenamiento 96
4.13. Evaluación de seguridad 96
4.14. Análisis de ataques 97
4.15. Seguridad de implementación 100
4.16. Requisitos operacionales y experiencia de usuario 101
4.17. Dependencias de confianza y arquitectura 102
4.18. Comparación global 103
4.19. Análisis multidimensional 104
4.19.1. Tiempo frente a comunicación 105
4.19.2. Seguridad frente a rendimiento 105
4.20. Discusión 106
4.20.1. Comparación con ECDSA 106
4.20.2. Comparación entre las variantes basadas en LWE 107
4.20.3. Evaluación del protocolo propuesto 107
4.21. Amenazas a la validez 108
4.21.1. Validez interna 108
4.21.2. Validez externa 109
4.21.3. Validez de constructo 109
4.21.4. Limitaciones de la estimación de seguridad 110
4.22. Reproducibilidad 110

5. Conclusiones **111**

Índice de figuras

1. Tiempo medio de configuración del sistema, de registro y de autenticación por protocolo (200 mediciones de cada operación por candidato). La configuración se ejecuta una vez por despliegue; el registro, una vez por usuario; la autenticación, en cada inicio de sesión. 91
2. Tiempo medio de cada etapa de la autenticación por protocolo (200 autenticaciones por candidato). El eje vertical es logarítmico, para que se vean a la vez los tiempos del reto, de centésimas de milisegundo, y los de la respuesta, de varios milisegundos. El total se mide en una autenticación completa independiente (Tabla 21). 92
3. Relación entre tiempo de autenticación, volumen de comunicación y memoria. Ambos ejes son logarítmicos; el tamaño de cada punto crece con el pico de memoria asignada durante la autenticación, indicado entre paréntesis. 105
4. Relación entre seguridad estimada y costo computacional. La seguridad es la estimación clásica de la Tabla 27 (Core-SVP para los candidatos de retículos); el eje de tiempo es logarítmico. 105

Índice de tablas

1. Símbolos principales del marco teórico. 11
2. Comparativa Teórica de Paradigmas ZKP frente a Atacantes Cuánticos y Extracción 30
3. Matriz Técnico-Cuantitativa de Frameworks ZKP Poscuánticos (valores reportados por cada fuente) 31
4. Cuadrantes de $\mathbb{Z}_{3329}$, bit de clave y pista. 44
5. Símbolos propios de ML-ZKAuth. 51
6. Vector de parámetros de ML-ZKAuth y origen de cada valor. 57
7. Búsqueda de (k', η') para la clave de identidad ($q' = 8\,380\,417$, $\kappa = 23$, $\gamma_1 = 2^{17}$, $\gamma_2 = 95\,232$). 58
8. Mensajes de una autenticación con acuerdo de clave. 60
9. Distribución de $z = y + \varsigma$ con y uniforme en $\{-4, \ldots, 4\}$ para dos desplazamientos. 62
10. Tamaño del registro de tickets consumidos con $\Delta_\nu = 300$ s. 66
11. Seguridad estimada de ML-ZKAuth con los parámetros por defecto. 70
12. Eventos improbables que solo afectan la disponibilidad. 74
13. Eventos improbables que comprometerían la seguridad. 75
14. Probabilidad de que un atacante acierte por intento, o costo de un ataque, en mecanismos en uso y en ML-ZKAuth. 76
15. Amenazas prácticas y cobertura de ML-ZKAuth. 77
16. Pasos del protocolo y funciones del prototipo. 78
17. Protocolos incluidos en la evaluación experimental. 82
18. Entorno utilizado para ejecutar los experimentos. 84
19. Parámetros criptográficos utilizados en cada implementación. 85
20. Tiempo de generación de claves, en milisegundos. 90
21. Tiempo promedio por etapa del proceso de autenticación. 90
22. Consumo de memoria durante las operaciones criptográficas: pico medio de asignaciones de Python por operación y estado persistente por usuario. . . . 93
23. Tamaño serializado de los principales objetos criptográficos. 94
24. Datos transmitidos durante una autenticación. 94
25. Tiempo proyectado del reto y la respuesta por escenario de red, en milisegundos (un viaje de ida y vuelta). 95
26. Estimación de almacenamiento según el número de usuarios. 96
27. Nivel de seguridad estimado para las configuraciones evaluadas. 97

28. Aspectos de seguridad de las implementaciones evaluadas. 100
29. Comparación de requisitos operacionales. 101
30. Resumen cuantitativo de los resultados principales. 104

1. Introducción

1.1. Generalidades

La autenticación de usuarios constituye uno de los problemas más complejos dentro de la seguridad de la información contemporánea. De acuerdo con múltiples análisis e informes del sector, entre el 67 % y el 80 % de los problemas globales de seguridad están directamente vinculados con la autenticación, la suplantación de credenciales y el robo de identidades [1].

A pesar de la evolución en las tecnologías de verificación mediante contraseñas de un solo uso (*One-Time Passwords*, OTP), esquemas de doble factor basados en los estándares de la alianza FIDO y técnicas biométricas, estas alternativas aún presentan limitaciones considerables [2]:

- **Contraseñas tradicionales:** Constituyen un mecanismo poco adecuado para las capacidades cognitivas humanas y obligan al usuario a ceder la totalidad de su secreto al sistema verificador.
- **Autenticación biométrica:** Su principal problema es que los rasgos biométricos no se pueden cambiar. Una contraseña filtrada se reemplaza por otra, pero una persona no puede cambiar su huella dactilar, su rostro ni su iris. Por eso, si una plantilla biométrica queda expuesta, el daño es permanente: el usuario no puede revocarla ni sustituirla, y quien la obtenga podrá intentar suplantarlo durante el resto de su vida. Además, se trata de datos personales sensibles, cuyo tratamiento genera objeciones importantes en materia de privacidad y de cumplimiento normativo.
- **Esquemas de doble factor con intermediarios:** Cuando la autenticación atraviesa dispositivos o nodos intermedios en redes distribuidas, la combinación de la credencial y un segundo factor (por ejemplo, un PIN) suele obligar a entregar información sensible a esos intermediarios, con lo que el usuario pierde el control sobre sus propios datos.

1.2. Contexto de la Situación

El panorama actual de la seguridad de la información se encuentra condicionado por dos dinámicas críticas: la necesidad de proteger la identidad sin entregar credenciales sensibles y la inminente llegada de la computación cuántica.

Por una parte, la autenticación convencional exige transmitir secretos legibles o equivalentes a través de nodos intermedios. Las Pruebas de Conocimiento Cero (*Zero-Knowledge Proofs*, ZKP) ofrecen una alternativa criptográfica que permite a una parte probadora (P)

demostrar a un verificador (V) que posee cierta información privada sin revelar nada sobre ese secreto más allá de que la afirmación es verdadera.

Por otra parte, la ejecución de algoritmos como el de Shor y Grover en computadores cuánticos amenaza la seguridad de los esquemas criptográficos tradicionales. Al transformar las funciones clásicas de una sola vía basadas en el logaritmo discreto y en la factorización de enteros en funciones fácilmente invertibles, la computación cuántica dejará inútiles los protocolos clásicos, por lo que la migración hacia primitivas resistentes a capacidades cuánticas de ataque (*quantum-safe*) se vuelve imperativa.

1.3. Justificación

Frente a la vulnerabilidad de las credenciales tradicionales y la amenaza de adversarios cuánticos, surge la necesidad de contar con esquemas de autenticación que permitan al usuario conservar el control sobre sus propios datos y que sean robustos frente a ataques clásicos y cuánticos.

Los problemas matemáticos basados en retículos (*lattices*), en particular el problema *Learning with Errors* (LWE) y sus variantes estructuradas, permiten construir funciones de una sola vía respaldadas por una reducción de seguridad del peor caso al caso promedio (*worst-case to average-case reduction*). Para los parámetros que cubre esa reducción, resolver una instancia aleatoria es tan difícil como resolver problemas de retículos en el peor caso, para los que no se conocen algoritmos eficientes clásicos ni cuánticos; para los parámetros concretos que se usan en la práctica, la seguridad se estima a partir del costo de los mejores ataques conocidos (Sección 2.3.6).

La integración de las Pruebas de Conocimiento Cero (ZKP) con la solidez matemática de LWE atiende simultáneamente ambas problemáticas:

1. Resuelve la exposición de secretos al verificar la identidad del usuario sin revelar sus credenciales.

2. Ofrece resistencia frente a adversarios con computadores cuánticos, apoyada en la dureza estimada del problema LWE y de sus variantes.

1.4. Objetivos

1.4.1. Objetivo General

Definir el funcionamiento, robustez, alcance y riesgos de un protocolo de autenticación de conocimiento cero que sea resistente ante ataques cuánticos, en contraste con protocolos

tradicionales, con el propósito de mejorar, para el presente y futuro previsible, la seguridad en sistemas que tengan un fuerte componente de autenticación.

1.4.2. Objetivos Específicos

1. Analizar los fundamentos matemáticos y las limitaciones de los protocolos *Zero-Knowledge-Proof*.

2. Analizar los fundamentos matemáticos y las limitaciones del problema *Learning with Errors*.

3. Diseñar e implementar un prototipo de protocolo que integre ambos conceptos con fines de autenticación segura.

4. Explorar las particularidades necesarias para el correcto funcionamiento del protocolo planteado.

5. Comparar los protocolos de autenticación convencionales más relevantes con el protocolo planteado.

2. Marco Teórico

2.1. Notación

En el resto del documento se usa la siguiente notación, y cada símbolo designa un único objeto.

Enteros, anillos y vectores Para un entero $q \geq 2$, $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$, y sus elementos se representan por $\{0, 1, \ldots, q-1\}$. Para un entero r y un entero positivo m, r mód $^{\pm}m$ es el único entero congruente con r módulo m en el intervalo $(-m/2, m/2]$ (representante centrado). Se trabaja en los anillos

$$\mathcal{R} = \mathbb{Z}[x]/\langle x^d + 1\rangle, \qquad \mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^d + 1\rangle, \tag{1}$$

con d potencia de dos: sus elementos son polinomios de grado menor que d y el producto se reduce con la regla $x^d = -1$. Los escalares y los polinomios se escriben en cursiva (c, f), los vectores de polinomios en negrita minúscula ($\mathbf{s} \in \mathcal{R}_q^k$) y las matrices en negrita mayúscula ($\mathbf{A} \in \mathcal{R}_q^{k\times k}$); $\mathbf{A}^T$ es la traspuesta. Un vector de k polinomios se identifica con el vector de sus kd coeficientes.

Normas y conjuntos de elementos pequeños Para $w \in \mathbb{Z}_q$, $|w|$ denota $|w \bmod^{\pm} q|$. Para un polinomio $f = \sum_i f_i x^i$,

$$\|f\|_\infty = \max_i |f_i|, \qquad \|f\|_1 = \sum_i |f_i|, \qquad \|f\|_2 = \Big(\sum_i f_i^2\Big)^{1/2}, \tag{2}$$

y para un vector de polinomios $\|\mathbf{f}\|_\infty$ es el máximo de $|f_i|$ sobre todos sus coeficientes. Para un entero $\eta \geq 0$, $S_\eta = \{f \in \mathcal{R} : \|f\|_\infty \leq \eta\}$ es el conjunto de polinomios con coeficientes en $\{-\eta, \ldots, \eta\}$.

Redondeo, muestreo y probabilidad Para un real y, $\lfloor y \rfloor$ y $\lceil y \rceil$ son el piso y el techo, y $\lfloor y \rceil = \lfloor y + 1/2 \rfloor$ es el redondeo al entero más cercano. $v \xleftarrow{\$} X$ indica que v se elige con distribución uniforme en el conjunto finito X, y $v \leftarrow \chi$ que se muestrea de la distribución χ; aplicado a un polinomio o a un vector, cada coeficiente se muestrea de forma independiente. $\Pr[\cdot]$ denota una probabilidad, λ el parámetro de seguridad y $\mathrm{negl}(\lambda)$ una función despreciable en λ. Cuando una función hash recibe varios argumentos, $\mathsf{H}(a_1, \ldots, a_n)$, estos se codifican de forma inequívoca antes de concatenarlos (Sección 2.2.1).

La Tabla 1 reúne los símbolos principales del marco teórico.

Tabla 1: Símbolos principales del marco teórico.

Símbolo	Significado
$\mathcal{P}$, $\mathcal{V}$, Sim, Ext	Probador, verificador, simulador y extractor de una prueba genérica.
x, ω, R	Afirmación pública, testigo y relación.
a, c, z, π	Compromiso, desafío, respuesta y prueba de un protocolo Σ.
H	Función hash modelada como oráculo aleatorio.
q, n, m, χ	Módulo, dimensión del secreto, número de muestras y distribución del error de LWE.
$\mathbf{A}$, $\mathbf{s}$, $\mathbf{e}$, $\mathbf{b}$	Matriz pública, secreto, error y vector de muestras de un problema LWE.
k, η	Rango de un módulo (Module-LWE) y parámetro de la distribución CBD(η).
$\mathbf{y}$, $\mathbf{w}$	Máscara y compromiso de Fiat–Shamir con abortos.
$\tilde{c}$, κ, B_κ	Semilla del desafío, número de coeficientes no nulos del desafío y conjunto de desafíos.
γ_1, γ_2, α, β	Anchura de la máscara, mitad del paso de redondeo, paso de redondeo ($\alpha = 2\gamma_2$) y cota de rechazo.
tk, ν, Δ_ν, $\mathcal{L}_{\text{spent}}$	Ticket, marca de tiempo, ventana de validez y registro de tickets consumidos.
vpk, vsk, K, ct	Claves pública y privada de un KEM, clave encapsulada y texto cifrado.
tok_1, tok_2, tok	Texto cifrado del KEM, contenido cifrado con el AEAD y token completo.
θ, θ', v	Valores ruidosos de las dos partes de un acuerdo de clave y pista de reconciliación.
β_{BKZ}	Tamaño de bloque del algoritmo BKZ en la estimación de seguridad (distinto de la cota de rechazo β).

2.2. Zero-Knowledge Proofs (ZKP)

Los protocolos de prueba de conocimiento cero (*Zero-Knowledge Proofs* o ZKPs) son una técnica criptográfica que permite verificar la veracidad de una información sin necesidad de revelar los datos en sí [3]; en autenticación, permiten que una parte demuestre su identidad ante otra sin entregarle su secreto. En este proceso interactúan dos partes: un **probador** (*prover*), que genera y demuestra la validez de una afirmación, y un **verificador** (*verifier*), quien comprueba dicha afirmación sin obtener información adicional ni conocer los datos subyacentes.

Las ZKP pueden clasificarse principalmente en dos categorías [3]:

- **Interactivas:** donde el verificador envía múltiples retos aleatorios al probador hasta

alcanzar un alto grado de certeza.

- **No interactivas:** donde la validación se realiza en una sola interacción (*single shot*).

La diferencia fundamental radica en que las pruebas no interactivas sustituyen los retos aleatorios del verificador por un elemento al que ambas partes tienen acceso: una cadena de referencia común (*common reference string*, CRS) o una función hash modelada como oráculo aleatorio, que es lo que hace la transformación de Fiat–Shamir (Sección 2.2.1). Por eso la prueba generada es transferible y puede ser validada de forma independiente por terceros [3].

Ejemplos Intuitivos Antes de la formalización, dos analogías clásicas ilustran la idea.

- **Ejemplo interactivo: la cueva de Alí Babá.** Una cueva circular tiene una sola entrada y, en el extremo opuesto, una puerta mágica que solo se abre con una palabra secreta. La puerta divide el recorrido en dos pasillos, A y B. El probador afirma conocer la palabra y quiere demostrarlo sin decirla.

  - *Interacción:* el probador entra por uno de los dos pasillos, elegido al azar, sin que el verificador lo vea. Luego el verificador, desde la entrada, le indica en voz alta por qué pasillo debe salir.
  - *Completitud:* si el probador conoce la palabra, siempre puede cumplir la orden: si está en el pasillo equivocado, abre la puerta y cruza al otro.
  - *Solidez:* un impostor que no conoce la palabra solo cumple la orden si el pasillo pedido coincide con aquel por el que entró, lo que ocurre la mitad de las veces. Si el juego se repite 128 veces, la probabilidad de que acierte en todas es 2^{-128}, prácticamente nula.
  - *Conocimiento cero:* el verificador solo ve al probador salir por el pasillo pedido y no aprende nada sobre la palabra. Prueba de ello es que un video de la sesión no convence a un tercero: cualquiera podría fabricarlo sin conocer la palabra, grabando muchos intentos y conservando solo aquellos en los que el pasillo pedido coincidió por casualidad con el de entrada.

- **Ejemplo no interactivo: la cueva con máquina de sorteo.** Supóngase ahora que el verificador no puede estar presente. El juego se hace con 128 cuevas iguales y una cámara que solo graba las entradas. El probador entra primero en las 128 cuevas, cada vez por un pasillo elegido al azar, de modo que sus elecciones quedan fijadas en la grabación. Solo entonces introduce la grabación completa en una *máquina de sorteo* pública, que devuelve una lista de 128 pasillos de salida. La máquina es impredecible:

no hay forma de saber qué responderá a una grabación sin introducirla, y cualquier cambio en la grabación produce una lista completamente distinta. El probador sale de cada cueva por el pasillo indicado y envía el video completo.

- *Verificación:* cualquiera puede comprobar el video sin comunicarse con el probador: introduce la grabación de las entradas en la misma máquina y verifica que las salidas coinciden con la lista obtenida.
- *Solidez:* un impostor no puede elegir la lista. Solo puede repetir todas las entradas y volver a sortear hasta que, por casualidad, la máquina le pida salir siempre por donde entró, lo que tiene una probabilidad de 2^{-128} en cada intento.
- *Conocimiento cero:* el video sigue sin mostrar la palabra.
- *Diferencia con el caso interactivo:* como nadie controla la máquina, el video sí convence a un tercero; la prueba es transferible.

La máquina de sorteo representa la función hash de la transformación de Fiat–Shamir (Sección 2.2.1); la grabación de las entradas, el compromiso; la lista de pasillos, el desafío; y las salidas, la respuesta.

La utilidad principal de este paradigma en autenticación es la siguiente. En un protocolo convencional, el verificador no confía en el probador antes de que este se identifique, y el probador debe confiar en que el verificador no filtrará el secreto que le entrega para autenticarse (por ejemplo, una contraseña). En cambio, usando ZKP el probador no entrega su secreto: el verificador recibe una prueba de que el probador lo conoce, de la que no puede obtener información sobre el secreto más allá de la validez de la afirmación, en el sentido preciso de la propiedad de conocimiento cero que se define a continuación. Así se reduce la información que el probador debe confiar al verificador.

2.2.1. Fundamentos Matemáticos y Propiedades de las Pruebas de Conocimiento Cero

Formulación Formal de Relaciones NP y Sistemas de Prueba Interactivos En la teoría de la complejidad computacional y la criptografía moderna, un sistema de prueba de conocimiento cero se formaliza mediante el concepto de lenguaje en NP definido sobre una relación binaria evaluable en tiempo polinomial [4]. Sea $R \subseteq \{0,1\}^* \times \{0,1\}^*$ una relación NP. Se define el lenguaje asociado L_R como el conjunto de instancias o afirmaciones para las cuales existe un testigo (*witness*) que satisface la relación:

$$L_R = \{x \in \{0,1\}^* \mid \exists\, \omega \in \{0,1\}^* \text{ tal que } (x, \omega) \in R\} \tag{3}$$

Donde x representa la declaración pública (por ejemplo, una clave pública o el valor de un hash) y ω representa el testigo privado o secreto (por ejemplo, una clave privada o una preimagen).

Un sistema de prueba interactivo para el lenguaje L_R consta de un par de máquinas de Turing probabilísticas que interactúan mediante el intercambio de mensajes en tiempo polinomial (PPT): el **probador** ($\mathcal{P}$) y el **verificador** ($\mathcal{V}$). El probador $\mathcal{P}$ posee la instancia pública x y el testigo secreto ω, mientras que el verificador $\mathcal{V}$ solo conoce x. Al finalizar el protocolo de interacción, denotado por el flujo de transcripción $\langle \mathcal{P}(x, \omega), \mathcal{V}(x) \rangle$, el verificador emite una decisión binaria:

$$\langle \mathcal{P}(x, \omega), \mathcal{V}(x) \rangle \in \{1 \text{ (Aceptar)}, 0 \text{ (Rechazar)}\} \tag{4}$$

Definición Matemática Rigurosa de las Propiedades Fundamentales Para que la interacción $(\mathcal{P}, \mathcal{V})$ constituya un sistema de prueba de conocimiento cero (*Zero-Knowledge Proof* - ZKP), debe satisfacer de forma estricta tres propiedades fundamentales cuantificadas bajo un parámetro de seguridad $\lambda \in \mathbb{N}$ [4, 5]:

1. **Completitud (*Completeness*):** Garantiza que si la afirmación es verdadera y ambas partes siguen el protocolo honestamente, el verificador siempre aceptará la prueba. Formalmente, para todo par $(x, \omega) \in R$:

$$\Pr\left[\langle \mathcal{P}(x, \omega), \mathcal{V}(x) \rangle = 1\right] \geq 1 - \text{negl}(\lambda) \tag{5}$$

   Aquí $\text{negl}(\lambda)$ representa una cantidad *despreciable* (*negligible*): una función que, al crecer el parámetro de seguridad λ, se hace más pequeña que el inverso de cualquier polinomio. Es decir, por grande que sea un polinomio $p(\lambda)$, a partir de cierto valor de λ se cumple $\text{negl}(\lambda) < 1/p(\lambda)$. El ejemplo típico es $2^{-\lambda}$: con $\lambda = 128$ vale $2^{-128} \approx 3 \cdot 10^{-39}$, una probabilidad tan pequeña que en la práctica el evento no ocurre. Si la probabilidad de aceptar es exactamente igual a 1, es decir, si un probador honesto nunca es rechazado, el esquema posee *completitud perfecta*.

2. **Solidez (*Soundness*):** Garantiza que si la afirmación es falsa ($x \notin L_R$), ningún probador deshonesto $\mathcal{P}^*$ (incluso con recursos computacionales ilimitados) puede convencer al verificador con una probabilidad significativa. Formalmente, para todo $x \notin L_R$ y cualquier probador deshonesto $\mathcal{P}^*$:

$$\Pr\left[\langle \mathcal{P}^*(x), \mathcal{V}(x) \rangle = 1\right] \leq \epsilon(\lambda) \leq \text{negl}(\lambda) \tag{6}$$

La cantidad $\epsilon(\lambda)$ se denomina el *error de solidez* (*soundness error*). En sistemas donde la solidez se garantiza únicamente contra probadores acotados a tiempo polinomial (PPT), el sistema se denomina un *argumento*. Se habla de prueba o argumento *de conocimiento* cuando, además, existe un extractor Ext que, con acceso a cualquier probador que convenza al verificador con probabilidad no despreciable, obtiene un testigo ω con $(x, \omega) \in R$ (Sección 2.2.3).

3. **Conocimiento Cero (*Zero-Knowledge*):** Garantiza que el verificador no aprende ninguna información adicional sobre el testigo ω más allá de la veracidad de la afirmación $x \in L_R$. Se formaliza mediante la existencia de un **simulador** probabilístico en tiempo polinomial (Sim) que, teniendo acceso únicamente a la instancia pública x (y a la máquina del verificador, pero sin acceso a ω), es capaz de generar una transcripción sintética que resulta indistinguible de la interacción real.

   Para cualquier verificador PPT deshonesto $\mathcal{V}^*$ y para todo $(x, \omega) \in R$, la vista del verificador $\text{View}_{\mathcal{V}^*}[\langle \mathcal{P}(x, \omega), \mathcal{V}^*(x) \rangle]$ satisface:

$$\mathsf{Sim}^{\mathcal{V}^*}(x) \approx \text{View}_{\mathcal{V}^*}[\langle \mathcal{P}(x, \omega), \mathcal{V}^*(x) \rangle] \tag{7}$$

   Según la noción de indistinguibilidad ($\approx$), el esquema se clasifica en:

   - **Conocimiento Cero Perfecto ($\approx_p$):** Las distribuciones de probabilidad son matemáticamente idénticas.
   - **Conocimiento Cero Estadístico ($\approx_s$):** La distancia estadística entre ambas distribuciones es infinitesimal.
   - **Conocimiento Cero Computacional ($\approx_c$):** Ningún algoritmo PPT puede distinguir entre ambas distribuciones con una ventaja no despreciable.

   Por ejemplo, la cueva de Alí Babá tiene conocimiento cero perfecto: el video fabricado repitiendo intentos y conservando solo los aciertos tiene exactamente la misma distribución que el de una sesión real. Cuando la simulación solo puede fallar en un evento de probabilidad despreciable, el conocimiento cero es estadístico, y cuando la indistinguibilidad depende de que un problema matemático sea difícil, es computacional.

Clasificación de Esquemas: Protocolos Sigma vs. Argumentos No Interactivos (NIZKAoK) Los sistemas de prueba de conocimiento cero se dividen estructuralmente según el modelo de interacción entre las partes:

- **Protocolos Interactivos de Tres Movimientos (Σ-Protocolos):** Son la piedra angular de las construcciones algebraicas. Un protocolo Σ opera en tres rondas consecutivas:

  1. *Compromiso (Commitment a):* $\mathcal{P}$ genera un valor aleatorio interno, calcula un compromiso a y lo envía a $\mathcal{V}$.
  2. *Desafío (Challenge c):* $\mathcal{V}$ selecciona un desafío aleatorio $c \xleftarrow{\$} \mathsf{Ch}$ de un conjunto de desafíos Ch y lo transmite a $\mathcal{P}$.
  3. *Respuesta (Response z):* $\mathcal{P}$ calcula la respuesta z a partir de x, ω, c y su aleatoriedad interna, y la envía a $\mathcal{V}$, quien evalúa una función de verificación pública $V(x, a, c, z) \in \{0, 1\}$ [5].

  Un protocolo Σ satisface tradicionalmente la propiedad de *Conocimiento Cero ante Verificador Honesto Especial* (*Special HVZK*), lo que significa que el simulador puede generar transcripciones válidas (a, c, z) eligiendo primero el desafío c al azar y calculando (a, z) de forma inversa.

- **Argumentos No Interactivos de Conocimiento (NIZKAoK) y Heurística de Fiat-Shamir:** En aplicaciones del mundo real, la interacción de múltiples rondas genera una latencia inaceptable. Para eliminar la interacción, se aplica la **Heurística de Fiat-Shamir** [6].

  En lugar de esperar el desafío aleatorio c emitido por un verificador interactivo, el probador computa el desafío internamente mediante el uso de una función hash criptográfica modelada como un **Oráculo Aleatorio** (*Random Oracle Model* - ROM) [7]:

$$c = \mathsf{H}(x, a) \tag{8}$$

  La prueba no interactiva resultante es la tupla $\pi = (a, z)$. El verificador comprueba la validez calculando $c' = \mathsf{H}(x, a)$ y verificando que $V(x, a, c', z) = 1$. Esto transforma un protocolo Σ interactivo en un Argumento No Interactivo de Conocimiento Cero (*Non-Interactive Zero-Knowledge Argument of Knowledge* - NIZKAoK) totalmente autónomo y verificable por terceros.

  Por qué esta transformación funciona. En el protocolo interactivo, la única función del verificador es elegir el desafío de forma impredecible para el probador y en el momento oportuno: después de que el probador se haya comprometido. Si el probador conociera el desafío antes de comprometerse, podría hacer trampa sin conocer el secreto, eligiendo primero una respuesta válida para ese desafío y construyendo después un compromiso

a la medida. La transformación de Fiat–Shamir conserva esa función sin necesidad del verificador: como el desafío se obtiene aplicando la función hash al propio compromiso, el probador no puede conocerlo antes de fijar el compromiso, y cualquier cambio en el compromiso produce un desafío completamente distinto e impredecible. El probador ya no puede escoger su desafío; a lo sumo puede probar compromisos distintos hasta obtener uno favorable, lo que tiene una probabilidad despreciable cuando el conjunto de desafíos es suficientemente grande. La función hash desempeña así el papel de un verificador honesto que elige el desafío al azar, y la prueba completa puede enviarse en un solo mensaje y comprobarse después por cualquiera, recalculando el hash.

Funciones hash y funciones de salida extensible (SHA-3 y SHAKE) La transformación de Fiat–Shamir y los protocolos sobre retículos usan dos tipos de funciones de la familia SHA-3, estandarizada en FIPS 202 [8] y construida sobre la permutación Keccak con la estructura de esponja:

- **Funciones hash** de salida fija, como SHA3-256, que asigna a cualquier cadena un resumen de 256 bits. Se usan para calcular el desafío de Fiat–Shamir y para resumir objetos grandes (por ejemplo, una clave pública) en 32 bytes.
- **Funciones de salida extensible** (XOF, *extendable-output functions*), SHAKE128 y SHAKE256, que producen una salida de la longitud que se pida: de una semilla corta se obtiene un flujo de bytes tan largo como haga falta. Así se expande una matriz pública $\mathbf{A} \in \mathcal{R}_q^{k\times k}$ desde una semilla de 32 bytes, leyendo el flujo en palabras y descartando las que no representan un elemento de $\mathbb{Z}_q$ (muestreo por rechazo), y así se deriva un desafío disperso a partir de una semilla corta (Sección 2.2.2). El número del nombre indica el nivel de seguridad, en bits, frente a colisiones y preimágenes.

En los análisis de seguridad estas funciones se modelan como **oráculos aleatorios**: funciones que asignan a cada entrada nueva una salida uniforme e independiente de las demás [7]. En ese modelo, el simulador de una prueba de conocimiento cero puede **programar** el oráculo, es decir, fijar la respuesta de H en una entrada que el adversario todavía no ha consultado; esta capacidad es la que permite simular pruebas de Fiat–Shamir sin conocer el testigo. Cuando una entrada tiene varios campos, estos se codifican de forma inequívoca (por ejemplo, anteponiendo a cada campo su longitud) y con una etiqueta de dominio distinta para cada uso, de modo que dos usos de la misma función nunca compartan entradas.

Fiat–Shamir en forma comprimida En muchos protocolos Σ el compromiso queda determinado por el resto de la transcripción: existe una función pública f tal que, en toda

transcripción aceptada, $a = f(x, c, z)$. Entonces la prueba no necesita incluir a, y basta enviar

$$\pi = (c, z), \qquad \text{y el verificador acepta si } c = \mathsf{H}\big(x, f(x, c, z)\big). \tag{9}$$

Si π es válida, $a' = f(x, c, z)$ es el único compromiso compatible con la ecuación de verificación, de modo que la comprobación del hash sustituye a la comprobación de $V(x, a, c, z)$. La ventaja es de tamaño: el desafío (o una semilla de la que se deriva) ocupa unos pocos bytes, mientras que el compromiso puede ser un elemento de grupo o un vector de polinomios. Por ejemplo, en el protocolo de identificación de Schnorr, basado en el logaritmo discreto, el compromiso se recalcula exactamente a partir del desafío y la respuesta, y la prueba se reduce a (c, z) [9]. En Fiat–Shamir con abortos sobre retículos la reconstrucción no es exacta: el verificador obtiene el compromiso desplazado por un término pequeño, y por eso el hash se aplica solo a sus bits altos (Sección 2.2.2). Esta es la forma que usa Dilithium [10].

2.2.2. Construcciones de ZKP sobre Retículos: De Stern a Lyubashevsky y el Framework LNP

Evolución de los ZKP sobre Retículos: Paradigma Combinatorio vs. Fiat-Shamir con Abortos La transición de la criptografía basada en problemas algebraicos tradicionales (como la factorización de enteros o el logaritmo discreto) hacia la criptografía basada en retículos (*lattice-based cryptography*) introdujo desafíos estructurales significativos en el diseño de pruebas de conocimiento cero.

Históricamente, el primer protocolo fundamental de ZKP sobre retículos fue el esquema de Stern (1993), concebido originalmente para el problema de decodificación por síndrome (*Syndrome Decoding*) y posteriormente adaptado al problema *Short Integer Solution* (SIS). El protocolo de Stern se basa en un enfoque combinatorio de permutaciones aleatorias y compromisos ocultos. Sin embargo, su limitación principal reside en que posee un alto error de solidez por ronda de 2/3. Para alcanzar un nivel de seguridad adecuado de 128 bits, el protocolo requiere repetirse unas 219 veces ($(2/3)^{219} \leq 2^{-128}$), lo que genera pruebas extremadamente pesadas: de cientos de kilobytes a más de un megabyte incluso para relaciones básicas [5, 11, 12].

Para superar estas ineficiencias combinatorias, Cayrel et al. [11] desarrollaron una construcción mejorada basada en el problema SIS sobre retículos modulares. Mediante el uso de funciones de compromiso (COM) estadísticamente ocultas y computacionalmente vinculantes, combinadas con vectores de máscara aleatorios de $\mathbb{Z}_q^m$ y matrices de permutación, Cayrel

et al. lograron reducir el error de solidez por ronda a:

$$\epsilon = \frac{q+1}{2q} \approx \frac{1}{2} \tag{10}$$

Esta reducción de 2/3 a 1/2 disminuyó drásticamente el número de repeticiones requeridas para alcanzar la solidez deseada, con lo que se redujo la sobrecarga de almacenamiento y comunicación en retículos ideales (*Ideal-SIS*) [11].

Fiat–Shamir con Abortos No obstante, el cambio de paradigma decisivo ocurrió con la introducción de la técnica de **Fiat-Shamir con Abortos** (*Fiat-Shamir with aborts*) ideada por Lyubashevsky [13, 14]. En los protocolos Σ tradicionales sobre grupos (como el de Schnorr), la respuesta del probador es la suma de un valor aleatorio y del producto del desafío por el secreto. Si se intentara aplicar esta fórmula directamente sobre un secreto vectorial corto $\mathbf{s}$ en un retículo, la respuesta $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}$ revelaría información parcial sobre la clave secreta $\mathbf{s}$ a través de la norma y la distribución de $\mathbf{z}$.

Para evitar esa fuga sin recurrir a permutaciones, Lyubashevsky introdujo la técnica de **muestreo por rechazo** (*rejection sampling*): el probador calcula $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}$ con una máscara $\mathbf{y}$ mucho más ancha que $c\,\mathbf{s}$ y solo publica $\mathbf{z}$ si cae dentro de un conjunto fijado de antemano (por ejemplo, $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, donde β acota $\|c\,\mathbf{s}\|_\infty$), en el que la distribución de $\mathbf{z}$ ya no depende de $\mathbf{s}$. Si no, el probador **aborta** el intento y recomienza con una máscara nueva. Los intentos abortados nunca se publican: en la versión no interactiva, el probador repite internamente hasta obtener un intento aceptado. Así la respuesta publicada no aporta información sobre $\mathbf{s}$, lo que permite demostrar el conocimiento cero con respuestas algebraicas directas y compactas [13, 14]. Como no hace falta repetir el protocolo ni ocultar el secreto con permutaciones, las pruebas resultantes son de pocos kilobytes: una firma de Dilithium2, construida con esta técnica, ocupa 2 420 bytes [15]. El precio es un número variable de intentos: cuanto más ancha es la máscara frente a $c\,\mathbf{s}$, más intentos se aceptan, pero mayores son la respuesta y el margen que la verificación debe tolerar.

Descomposición en bits altos y bajos La forma comprimida de Fiat–Shamir necesita que el verificador reconstruya el compromiso. En Fiat–Shamir con abortos sobre retículos no lo consigue exactamente: obtiene el compromiso desplazado por un término pequeño, que depende del desafío y del error de la clave (por ejemplo, $\mathbf{w} - c\,\mathbf{e}$ en lugar de $\mathbf{w}$ para una clave $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$). La solución, introducida por Bai y Galbraith [16] y adoptada por Dilithium [10, 15], consiste en comprometerse solo con los *bits altos* de $\mathbf{w}$, que un desplazamiento pequeño no altera.

Sea q un primo impar y α un entero par que divide a $q-1$. Para $r \in \mathbb{Z}_q$, representado en $\{0,\ldots,q-1\}$, sea $r_0 = r \bmod^{\pm} \alpha$ y

$$\mathsf{Decompose}_q(r,\alpha) = (r_1, r_0) = \begin{cases} (0,\ r_0 - 1) & \text{si } r - r_0 = q-1, \\ \left(\dfrac{r-r_0}{\alpha},\ r_0\right) & \text{en otro caso,} \end{cases} \tag{11}$$

de modo que $r \equiv r_1\alpha + r_0 \pmod q$ con $r_1 \in \{0,\ldots,(q-1)/\alpha - 1\}$ [15]. Se escribe $\mathsf{HighBits}_q(r,\alpha) = r_1$ y $\mathsf{LowBits}_q(r,\alpha) = r_0$; sobre polinomios y vectores de polinomios ambas funciones se aplican coeficiente a coeficiente.

El caso especial existe porque $q-1$ y 0 son vecinos en $\mathbb{Z}_q$: sin él, los valores próximos a $q-1$ tendrían bits altos $(q-1)/\alpha$, distintos de los de los valores próximos a 0, aunque estén a una unidad de distancia en el círculo modular. Con el ajuste, todos los valores entre $-\alpha/2$ y $\alpha/2$ (módulo q) tienen bits altos 0 y bits bajos iguales a su representante centrado. La propiedad que se usa es el Lema 2 de la especificación de Dilithium [15]: para vectores $\mathbf{r}$ y $\boldsymbol{\delta}$ de polinomios,

$$\|\boldsymbol{\delta}\|_\infty \le \beta \ \wedge\ \left\|\mathsf{LowBits}_q(\mathbf{r},\alpha)\right\|_\infty < \tfrac{\alpha}{2} - \beta \implies \mathsf{HighBits}_q(\mathbf{r}+\boldsymbol{\delta},\alpha) = \mathsf{HighBits}_q(\mathbf{r},\alpha). \tag{12}$$

La idea de la demostración, coeficiente a coeficiente: si $r \equiv r_1\alpha + r_0$ con $|r_0| < \alpha/2 - \beta$ y $|\delta| \le \beta$, entonces $r+\delta \equiv r_1\alpha + (r_0+\delta)$ con $|r_0+\delta| < \alpha/2$; por tanto, $r_0+\delta$ es el representante centrado de $r+\delta$ módulo α y el cociente sigue siendo r_1. Alrededor de 0 la misma cuenta vale gracias al caso especial.

Por ejemplo, con los valores de ML-DSA-44 [17], $q = 8\,380\,417$, $\alpha = 190\,464$ (que deja 44 valores posibles de bits altos) y $\beta = 78$, el valor $r = 1\,000\,000 = 5 \cdot 190\,464 + 47\,680$ da $(r_1, r_0) = (5, 47\,680)$. Como $|r_0| < 95\,232 - 78$, cualquier perturbación con $|\delta| \le 78$ conserva $r_1 = 5$. En el borde, $r = q-1$ da $r_0 = 0$ y $r - r_0 = q-1$, luego $(r_1, r_0) = (0, -1)$.

Una analogía ayuda a fijar la idea. Imagínese una pista circular, como la de un estadio, dividida en 44 tramos de igual longitud, numerados del 0 al 43, con una marca en el punto medio de cada tramo. La posición de un corredor puede describirse con dos datos: el número del tramo en el que se encuentra, que corresponde a HighBits, y su distancia, con signo, al punto medio de ese tramo, que corresponde a LowBits. Si el corredor avanza o retrocede unos pocos metros, sigue en el mismo tramo, salvo que estuviera muy cerca de una de las líneas que separan los tramos. La Ecuación (12) expresa exactamente esto: si el desplazamiento es pequeño (a lo sumo β) y el corredor está lejos de los bordes (a menos de $\alpha/2 - \beta$ del punto medio), el número de tramo no cambia. Como la pista es circular, el tramo 0 tiene su punto medio justo en la línea de salida y abarca los últimos metros antes de cruzarla y

los primeros después; eso es lo que hace el caso especial de Decompose. En Fiat–Shamir con abortos, el probador incluye en el hash solo los números de tramo de los coeficientes de su compromiso, y el verificador, que obtiene el compromiso ligeramente desplazado, calcula los mismos números siempre que ningún coeficiente estuviera cerca de un borde. El probador lo garantiza con una segunda condición de rechazo: descarta los intentos en los que algún coeficiente está demasiado cerca de un borde.

Desafíos dispersos y SampleInBall En el protocolo de Schnorr el desafío es un número. En Fiat–Shamir con abortos sobre retículos es un polinomio c de d coeficientes ($d = 256$ en Dilithium), elegido de forma que sea *disperso*: casi todos sus coeficientes valen 0 y solo unos pocos, κ, valen $+1$ o -1.

Por qué coeficientes $+1$, -1 y 0. El probador debe ocultar el producto $c\,\mathbf{s}$ con su máscara, y cuanto más pequeño sea ese producto, más fácil es ocultarlo. Multiplicar un polinomio por x^i solo desplaza sus coeficientes i posiciones; los que se salen por el final reaparecen al principio con el signo cambiado, porque $x^d = -1$. Por eso, si c tiene κ coeficientes iguales a ± 1, el producto $c\,\mathbf{s}$ es la suma, con signos, de κ copias desplazadas de $\mathbf{s}$, y ninguno de sus coeficientes supera κ veces el mayor coeficiente de $\mathbf{s}$:

$$\|c\,\mathbf{s}\|_\infty \le \kappa\,\eta \quad \text{si } \|\mathbf{s}\|_\infty \le \eta. \tag{13}$$

Esa cota, $\beta = \kappa\eta$, es lo que el muestreo por rechazo debe ocultar. Con coeficientes mayores que 1 el producto crecería y el probador tendría que reintentar muchas más veces. Por ejemplo, con $d = 8$ y $\kappa = 3$, el desafío $c = x - x^4 + x^6$, de coeficientes $(0, 1, 0, 0, -1, 0, 1, 0)$, hace que cada coeficiente de $c\,\mathbf{s}$ sea una suma con signo de exactamente tres coeficientes de $\mathbf{s}$.

Cuántos coeficientes no nulos. Sea B_κ el conjunto de los polinomios con exactamente κ coeficientes iguales a ± 1 y el resto nulos. Su tamaño es el número de formas de elegir las κ posiciones no nulas y sus signos:

$$|B_\kappa| = \binom{d}{\kappa} 2^\kappa. \tag{14}$$

Un impostor que intenta adivinar el desafío acierta con probabilidad $1/|B_\kappa|$ en cada intento, de modo que este número debe ser enorme. Elegir κ es un compromiso: cada coeficiente no nulo adicional agranda el espacio de desafíos, pero también aumenta la cota $\beta = \kappa\eta$ y, con ella, el número de intentos del probador. Por ejemplo, con $d = 256$, $\kappa = 22$ da $|B_{22}| \approx 2^{126{,}7}$, que no llega a 2^{128}, y $\kappa = 23$ da $|B_{23}| \approx 2^{131{,}1}$: 23 es el menor número de coeficientes no nulos con al menos 2^{128} desafíos posibles. Dilithium deja más margen: usa $\kappa = 39$, 49 y 60, con 192, 225 y 257 bits de entropía según el nivel de seguridad [15]. Además, la diferencia de dos desafíos tiene coeficientes entre -2 y 2, lo que mantiene corto el testigo que se extrae en el

argumento de solidez.

Cómo se obtiene el desafío: SampleInBall. El hash de Fiat–Shamir produce una semilla $\tilde{c}$ de 32 bytes, que debe convertirse en un desafío de B_κ de forma determinista, para que el verificador obtenga exactamente el mismo c [15, 17]. SampleInBall lo hace como quien coloca fichas en un tablero de 256 casillas, inicialmente vacías (todos los coeficientes valen 0), usando como fuente de azar el flujo que el XOF SHAKE256 produce a partir de $\tilde{c}$ (Sección 2.2.1):

1. Los primeros bits del flujo deciden el signo, $+1$ o -1, de cada una de las κ fichas.

2. Las fichas se colocan una a una. Para cada ficha se toma una casilla nueva al final del tablero (las posiciones $d-\kappa$ a $d-1$) y se elige al azar, con el flujo, una posición j entre la primera casilla y esa casilla nueva. Lo que hubiera en la posición j se traslada a la casilla nueva y la ficha se coloca en j. Las lecturas del flujo que darían una posición sesgada se descartan.

Este procedimiento es una versión del barajado de Fisher–Yates: si el flujo se modela como aleatorio, todas las configuraciones de B_κ resultan igual de probables, y cualquiera que conozca $\tilde{c}$ obtiene el mismo desafío.

Pruebas para Afirmaciones más Complejas: el Framework LNP Cuando hay que demostrar afirmaciones más ricas que el conocimiento de una clave (por ejemplo, que una credencial está firmada por una autoridad sin revelar la credencial), se usan marcos más generales. El más eficiente sobre retículos es el de Lyubashevsky, Nguyen y Plançon (LNP), que combina compromisos sobre retículos con Fiat–Shamir con abortos: para una relación lineal básica con testigo corto, su prueba ocupa unos 14 KB, frente a más de un megabyte de las construcciones combinatorias de tipo Stern [12]. LAASSO se construye sobre este marco [18]. Por otra línea, Baum y Nof construyeron argumentos de conocimiento cero para circuitos aritméticos generales con la técnica *MPC-in-the-head* y los aplicaron a problemas de retículos [19]. Para demostrar solo el conocimiento de una clave, en cambio, basta la plantilla de Fiat–Shamir con abortos descrita arriba, con pruebas mucho más pequeñas.

2.2.3. El Obstáculo del Rebobinado Cuántico (Quantum Rewinding Barrier)

Mecanismo Clásico de Rebobinado y el Lemma de Bifurcación (*Forking Lemma*) En la criptografía clásica, las demostraciones formales de solidez (*soundness*) y la propiedad de argumento de conocimiento para protocolos Σ e identificaciones no interactivas se basan fundamentalmente en la técnica del **rebobinado** (*rewinding*) y en el *Forking Lemma*, introducido por Pointcheval y Stern [20].

Bajo este paradigma, la prueba de seguridad se construye mediante un **extractor de conocimiento** (Ext) determinista o probabilístico en tiempo polinomial que interactúa con un probador deshonesto $\mathcal{P}^*$ como una "caja negra":

1. Ext ejecuta a $\mathcal{P}^*$ hasta que este emite un compromiso inicial a.
2. Ext envía un primer desafío aleatorio c_1 y registra la respuesta válida z_1, con lo que obtiene la transcripción (a, c_1, z_1).
3. A continuación, Ext **rebobina** el estado de ejecución interno de $\mathcal{P}^*$ al punto exacto inmediatamente posterior al envío del compromiso a.
4. En este estado restaurado, Ext emite un segundo desafío diferente $c_2 \neq c_1$ y obtiene una segunda respuesta válida z_2, con lo que forma la transcripción (a, c_2, z_2).

Al disponer de dos transcripciones válidas $((a, c_1, z_1), (a, c_2, z_2))$ que comparten el mismo compromiso a, el extractor Ext aplica relaciones algebraicas directas para despejar un testigo en tiempo polinomial, de modo que la solidez de la prueba se reduce a la dureza del problema matemático subyacente. En los protocolos sobre retículos, el testigo extraído suele ser corto pero no necesariamente el secreto original: es un testigo de una relación "relajada". En Fiat–Shamir con abortos, por ejemplo, de dos respuestas $\mathbf{z}$ y $\mathbf{z}'$ a desafíos c y c' se obtiene la diferencia $\mathbf{z} - \mathbf{z}'$, un vector corto que satisface la ecuación de la clave multiplicada por $c - c'$ y con cotas más amplias que las del secreto [14]. La prueba es entonces un argumento de conocimiento para esa relación relajada.

Falla del Rebobinado Directo en el Modelo Cuántico: Colapso de la Función de Onda y No-Clonación Cuando el adversario es una entidad cuántica, el mecanismo clásico de rebobinado deja de ser aplicable de forma directa [21].

Un probador deshonesto cuántico $\mathcal{P}^*$ almacena su estado interno como un estado cuántico complejo en superposición, representado por un vector de estado $|\psi\rangle$ en un espacio de Hilbert:

$$|\psi\rangle = \sum_i \alpha_i |i\rangle, \quad \text{con} \sum_i |\alpha_i|^2 = 1 \tag{15}$$

Cuando el extractor cuántico Ext interactúa con $\mathcal{P}^*$ para obtener la respuesta z_1 al primer desafío c_1, efectúa inevitables mediciones cuánticas sobre los registros de $\mathcal{P}^*$. Por el principio de medición de von Neumann:

- **Colapso del Estado:** La medición provoca el colapso irreversible de la función de onda de $|\psi\rangle$ a un autoestado proyectado $|\psi_{c_1,z_1}\rangle$. Las fases de la superposición previa se destruyen con la medición.

- **Imposibilidad de Clonación:** Por el **Teorema de No-Clonación** (*No-Cloning Theorem*), el extractor Ext no puede realizar una copia de respaldo del estado cuántico desconocido $|\psi\rangle$ antes de enviar el primer desafío c_1.

En consecuencia, el extractor no puede, en general, "rebobinar" a $\mathcal{P}^*$ a su estado cuántico exacto $|\psi\rangle$ previo a la primera medición. Si Ext envía un segundo desafío c_2 sobre el estado alterado $|\psi_{c_1,z_1}\rangle$, la respuesta obtenida puede no guardar la coherencia necesaria con el compromiso a, lo que invalida el argumento clásico de extracción [21].

Insuficiencia de las Reducciones Clásicas para Seguridad Poscuántica La existencia del obstáculo del rebobinado cuántico conduce a un resultado crítico en la criptografía poscuántica: **la dureza poscuántica del problema matemático subyacente (como LWE o SIS) no garantiza automáticamente la seguridad poscuántica del protocolo ZKP si su demostración de solidez depende de reducciones por rebobinado** [21]. Lombardi et al. mostraron, de forma más general, que una demostración clásica de seguridad bajo LWE, incluso con reducciones de caja negra, no implica seguridad poscuántica: construyeron esquemas (deliberadamente artificiales) con esa demostración que un adversario cuántico rompe con unas pocas consultas [22].

Por este motivo, el diseño de argumentos no interactivos de conocimiento poscuánticos (NIZKAoK) exige modelos analíticos más rigurosos, tales como:

1. El **Modelo de Oráculo Aleatorio Cuántico** (*Quantum Random Oracle Model* - QROM), donde el adversario cuántico puede realizar consultas en superposición a las funciones hash.

2. La propiedad de **Extractabilidad en Línea Recta** (*Straight-Line Extractability* - SLE), la cual permite la extracción del testigo en una única ejecución sin necesidad de rebobinado. Se obtiene haciendo que la prueba contenga el testigo cifrado bajo una clave pública incluida en los parámetros comunes, cuya clave privada (la *trampilla*) solo conoce el extractor en la demostración de seguridad; el precio son pruebas más grandes. LAASSO usa esta técnica [18].

Esto no significa que el rebobinado sea imposible frente a adversarios cuánticos: Liu y Zhandry mostraron que los resultados clásicos se trasladan al caso cuántico para protocolos con propiedades adicionales, y que el protocolo de Lyubashevsky en el que se basa Fiat–Shamir con abortos es seguro en el QROM sin modificaciones, resultado que se extiende a Dilithium [21]. Kiltz, Lyubashevsky y Schaffner dieron una reducción concreta en el QROM para las firmas de esta forma [23], y Devevey et al. y Barbosa et al. corrigieron y mecanizaron después esas demostraciones [24, 25].

2.2.4. Propiedades de Privacidad: No Vinculabilidad y Denegabilidad

No Vinculabilidad de Credenciales y Tokens (*Unlinkability*) En protocolos de autenticación con privacidad, la **no vinculabilidad** (*unlinkability*) garantiza que un adversario o un intermediario no pueda determinar si dos o más ejecuciones del protocolo provienen del mismo probador [18].

Formalmente, consideremos dos sesiones de autenticación i y j realizadas por el mismo probador utilizando un secreto ω, las cuales producen los tokens de autenticación $\mathrm{tok}^{(i)}$ y $\mathrm{tok}^{(j)}$. La propiedad de no vinculabilidad establece que para cualquier adversario en tiempo polinomial probabilístico (PPT) $\mathcal{A}$, la ventaja de distinguir entre dos tokens generados por el mismo probador y dos tokens generados por probadores distintos es despreciable [18]:

$$\left|\Pr\left[\mathcal{A}\left(\mathrm{tok}^{(i)}(\omega), \mathrm{tok}^{(j)}(\omega)\right) = 1\right] - \Pr\left[\mathcal{A}\left(\mathrm{tok}^{(i)}(\omega_A), \mathrm{tok}^{(j)}(\omega_B)\right) = 1\right]\right| \leq \mathrm{negl}(\lambda) \quad (16)$$

Cuando el verificador es designado, es decir, debe saber quién se autentica, la propiedad se busca frente a terceros e intermediarios y no frente al verificador, que identifica al probador por diseño. Dos mecanismos la favorecen:

1. **Aleatoriedad nueva en cada prueba:** en Fiat–Shamir con abortos, cada respuesta $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}$ usa una máscara $\mathbf{y}$ nueva, y la respuesta aceptada tiene una distribución que no depende de $\mathbf{s}$, lo que impide correlacionar algebraicamente las respuestas de distintas sesiones [14].

2. **Cifrado híbrido con claves efímeras:** si cada token se cifra con una clave nueva, obtenida por encapsulamiento (Sección 2.2.5), un observador solo ve textos cifrados de longitud fija; si además la identidad del probador viaja cifrada, no puede asociar dos tokens a un mismo probador salvo por metadatos ajenos al protocolo, como la dirección de red o la hora de conexión.

Denegabilidad (*Deniability*) La propiedad de **denegabilidad** (*deniability*), analizada formalmente por An et al. [26], protege al probador contra coerciones o auditorías posteriores a la sesión (*post-hoc coercion*). Si un atacante o un auditor obtiene la transcripción completa tr de una sesión pasada, la denegabilidad asegura que el probador puede negar haber participado, porque cualquier tercero o el propio verificador podría haber fabricado esa transcripción [26].

Matemáticamente, la denegabilidad se fundamenta en la existencia de un simulador de transcripciones $\mathsf{Sim}_{\mathrm{den}}$ que puede ejecutarse utilizando únicamente información pública y la

clave del verificador, sin requerir el secreto del probador ω ni su cooperación [26]:

$$\mathsf{tr}_{\text{real}} = \langle \mathcal{P}(\omega), \mathcal{V}(vpk) \rangle \;\approx_c\; \mathsf{Sim}_{\text{den}}(vpk) = \mathsf{tr}_{\text{sim}} \tag{17}$$

En un protocolo con verificador designado y denegabilidad (como los esquemas de *secret handshakes* de An et al. [26]), la clave del verificador le confiere la capacidad técnica de simular transcripciones indistinguibles de las reales. De este modo, si una transcripción se expone ante un tercero, carece de valor probatorio: el verificador no puede probar que fue generada por el probador y no sintetizada por él mismo [26].

Las pruebas no interactivas obtenidas con Fiat–Shamir, en cambio, no son denegables por sí mismas: cualquiera puede verificarlas con la clave pública del probador y el reto de la sesión, de modo que son transferibles (Sección 2.2), y quien las recibe puede mostrarlas a un tercero como evidencia de que el probador participó. Cifrar la prueba limita quién puede leerla, no quién puede comprobarla una vez descifrada.

2.2.5. Protección del Canal de Comunicación: Token Hiding con ML-KEM y AES-GCM

El Problema del Verificador No Designado y la Intercepción por Intermediarios Comprometidos En los protocolos de autenticación distribuidos y sistemas de inicio de sesión único (*Single Sign-On* - SSO), las credenciales y pruebas criptográficas generadas por el probador deben transmitirse frecuentemente a través de intermediarios no confiables o redes públicas antes de alcanzar la entidad verificadora final [18].

Si las pruebas de conocimiento cero (ZKP) o los tokens de autenticación se transmitieran en texto claro a través del canal de comunicación, surgiría el problema del *verificador no designado* [18]. Un intermediario comprometido, como un nodo de la red controlado por el adversario, podría inspeccionar el contenido del token, extraer metadatos de la sesión o rastrear las pruebas enviadas en sesiones sucesivas [18]. Esta exposición destruye la propiedad de no vinculabilidad (*unlinkability*) y permite a adversarios intermedios perfilar los hábitos de uso del probador [18].

Mecanismos de Encapsulamiento de Clave (KEM), Cifrado de Datos (DEM) y AEAD Un mecanismo de encapsulamiento de clave (KEM) es una terna de algoritmos

$$(vpk, vsk) \leftarrow \mathsf{KeyGen}(), \qquad (K, \mathsf{ct}) \leftarrow \mathsf{Encaps}(vpk), \qquad K' \leftarrow \mathsf{Decaps}(vsk, \mathsf{ct}), \tag{18}$$

con $K' = K$ (salvo con probabilidad despreciable) cuando todo se ejecuta honestamente. A diferencia del cifrado de clave pública, el emisor no elige la clave: el encapsulamiento genera

K al azar y la devuelve junto con el texto cifrado ct. La seguridad estándar de un KEM es la indistinguibilidad frente a ataques adaptativos de texto cifrado elegido (IND-CCA2): un adversario que puede pedir el desencapsulamiento de cualquier texto cifrado distinto de ct no logra distinguir K de una clave uniforme. ML-KEM, estandarizado en FIPS 203 [27] a partir de Kyber [28], es un KEM IND-CCA2 basado en Module-LWE con módulo 3329 y tres juegos de parámetros, ML-KEM-512, ML-KEM-768 y ML-KEM-1024, de rangos 2, 3 y 4, asignados a las categorías de seguridad 1, 3 y 5 de NIST. En ML-KEM-768, con ruido CBD(2), la clave pública mide 1 184 bytes, el texto cifrado 1 088 bytes y la clave encapsulada 32 bytes. Obtiene la seguridad IND-CCA2 aplicando una variante de la transformación de Fujisaki–Okamoto a un cifrado IND-CPA, con rechazo implícito: un texto cifrado alterado se desencapsula en una clave sin relación con la original [27].

Un cifrado autenticado con datos asociados (AEAD) cifra un mensaje con una clave simétrica, un *nonce* y unos datos asociados que se autentican pero no se cifran. AES-256-GCM [29] usa *nonces* de 96 bits y produce una etiqueta de autenticación de 128 bits; garantiza confidencialidad e integridad, de modo que sin la clave un texto cifrado modificado o fabricado se rechaza con probabilidad abrumadora. Su seguridad exige no repetir un *nonce* con la misma clave.

El paradigma KEM-DEM combina ambos mecanismos: el KEM transporta una clave nueva y un cifrado simétrico (el DEM, *data encapsulation mechanism*, aquí un AEAD) protege los datos con ella. Si el KEM es IND-CCA2 y el DEM es seguro frente a ataques de texto cifrado elegido, el cifrado híbrido resultante es IND-CCA2 [30]. Esta garantía se refiere al contenido: el texto cifrado no revela nada sobre el mensaje salvo su longitud. No implica que el texto cifrado sea indistinguible de una cadena de bits uniforme; de hecho, los coeficientes comprimidos de un texto cifrado de ML-KEM no son uniformes, porque 3329 no es una potencia de dos.

El Mecanismo de Ocultación de Tokens (*Token Hiding*) mediante Cifrado Híbrido Para neutralizar la amenaza de intercepción en nodos intermedios y garantizar que únicamente el verificador designado pueda validar la autenticación, el sistema de inicio de sesión único anónimo LAASSO introduce la técnica de *ocultación de tokens* (*token hiding*) [18]. Este mecanismo cifra la prueba con un esquema híbrido poscuántico que combina un cifrado asimétrico sobre retículos modulares (Kyber) con el cifrado simétrico AES [18].

En LAASSO, el verificador genera un par de claves (vpk, vsk) de Kyber y custodia vsk; el probador elige una clave simétrica K, la cifra con el cifrado de clave pública de Kyber, $\mathrm{tok}_1 = \mathrm{Kyber.Enc}(vpk, K)$, y cifra con AES, bajo K, la prueba π junto con el ticket y su periodo de validez, $\mathrm{tok}_2 = \mathrm{AES.Enc}(K, \{\pi, \mathsf{tk}, \nu\})$. El verificador recupera $K = \mathrm{Kyber.Dec}(vsk, \mathrm{tok}_1)$ y

descifra tok_2 [18].

Con un KEM estándar como ML-KEM, en cambio, el probador no elige K: el encapsulamiento la genera. La misma ocultación se expresa entonces con el paradigma KEM-DEM de la sección anterior:

1. **Encapsulamiento:** $(K, \mathrm{tok}_1) \leftarrow \mathsf{KEM.Encaps}(vpk)$.
2. **Cifrado autenticado del contenido:** $\mathrm{tok}_2 = \mathsf{AEAD.Enc}_K(\pi, \mathsf{tk}, \nu, \dots)$, con un *nonce* nuevo.
3. **Token compuesto:** $\mathrm{tok} = (\mathrm{tok}_1, \mathrm{tok}_2)$.

Frescura: Retos, Tickets y Marcas de Tiempo Un adversario que observa una autenticación válida puede intentar reenviarla más tarde (ataque de repetición o *replay*). Una prueba no interactiva está especialmente expuesta, porque se verifica sin interactuar con quien la produjo. La defensa es la **frescura**: lograr que cada prueba sirva para una sola sesión. Para ello, la entrada del hash de Fiat–Shamir debe incluir valores que el verificador controla y que no se repiten:

- **Reto (*nonce*):** el verificador envía un valor aleatorio nuevo y la prueba lo incluye en la entrada del hash, de modo que una prueba calculada para un reto no verifica para otro.
- **Ticket de un solo uso:** un identificador aleatorio tk que se emite para una sesión y que el verificador anota en un registro de tickets consumidos $\mathcal{L}_{\mathrm{spent}}$ la primera vez que lo recibe; un segundo token con el mismo ticket se rechaza. Con identificadores aleatorios de 256 bits, la probabilidad de que dos de N tickets coincidan es a lo sumo $N^2/2^{257}$.
- **Marca de tiempo:** un valor ν que fija el momento de emisión, junto con una ventana de validez Δ_ν. El verificador rechaza los tokens con $|\nu_{\mathrm{actual}} - \nu| > \Delta_\nu$, lo que acota cuánto tiempo hay que recordar los tickets consumidos: los anteriores a la ventana pueden borrarse del registro, porque la propia ventana ya los rechaza.

En LAASSO, el emisor de credenciales asigna a cada ticket un identificador único, asociado a un periodo de validez, y el proveedor de servicio rechaza los tickets ya consumidos o vencidos e inserta los nuevos en el registro de consumo de forma atómica [18]. El ticket también puede emitirlo el propio verificador como reto de la sesión, de modo que una sola entidad emite los tickets y lleva su registro.

Garantías de Seguridad y el Canal Ciego de Transmisión El procesamiento del token en la entidad receptora garantiza la verificación exclusiva por el destinatario autorizado. Al recibir el token compuesto $\text{tok} = (\text{tok}_1, \text{tok}_2)$, el verificador recupera la clave con $\mathsf{Decaps}(vsk, \text{tok}_1)$ y descifra tok_2; si la etiqueta de autenticación no es válida, rechaza el token. Las propiedades de seguridad del mecanismo de *token hiding* son:

- **Seguridad de Verificador Designado (*Designated Verifier Security*):** Por la seguridad IND-CCA2 del KEM (por ejemplo, ML-KEM), cualquier intermediario que no posea la clave privada vsk es incapaz de obtener información sobre la clave del token y, por tanto, de leer tok_2 [27].
- **Confidencialidad e Integridad del Contenido:** Por la seguridad del esquema KEM-DEM [30], dos tokens tok^A y tok^B que ocultan contenidos distintos de la misma longitud son computacionalmente indistinguibles para un atacante que escuche el canal:

$$\left|\Pr[\mathcal{A}(\text{tok}^A) = 1] - \Pr[\mathcal{A}(\text{tok}^B) = 1]\right| \leq \text{negl}(\lambda), \tag{19}$$

  y cualquier modificación de tok_1 o tok_2 hace fallar la etiqueta del AEAD. El token revela su longitud, que es fija si el contenido lo es, y no es indistinguible de una cadena uniforme (Sección 2.2.5).
- **Paso a Canal Ciego de Transmisión (*Blind Transmission Channel*):** El intermediario queda relegado operacionalmente a actuar como un mero *canal ciego de transmisión*: recibe el bloque cifrado tok, no puede leer su contenido ni evaluar si la prueba ZKP es válida o falsa, y se limita a reenviarlo al verificador.

Esta ocultación impide que el intermediario lea las pruebas o los identificadores que viajan en ellas. La prevención de ataques de repetición depende, además, del registro de tickets consumidos del verificador (Sección 2.2.5) [18].

2.2.6. Estado del Arte Comparativo de Frameworks ZKP Poscuánticos

Análisis Comparativo de Paradigmas Poscuánticos y Resistencia a Rebobinado Cuántico La evolución de las pruebas de conocimiento cero en la era poscuántica está definida por la capacidad de los protocolos para mantener la solidez frente a adversarios con capacidades de computación cuántica. Mientras que los esquemas pioneros dependían exclusivamente de análisis de seguridad clásicos basados en el *Forking Lemma*, las construcciones contemporáneas se analizan en el Modelo de Oráculo Aleatorio Cuántico (QROM) o recurren a técnicas de Extractabilidad en Línea Recta [21, 18].

Tabla 2: Comparativa Teórica de Paradigmas ZKP frente a Atacantes Cuánticos y Extracción

Paradigma / Esquema	**Problema Subyacente**	**Mecanismo de Extracción**	**Modelo de Seguridad**	**Situación frente a adversarios cuánticos**
Stern (1993) / Cayrel et al. (2012) [11]	SIS / Decodificación por Síndrome	Rebobinado Clásico (*Forking*)	Clásico (ROM al aplicar Fiat–Shamir)	Solidez demostrada por rebobinado clásico; las fuentes citadas no la analizan frente a adversarios cuánticos.
Fiat-Shamir con Abortos (Lyubashevsky) [14]	SIS / LWE sobre Retículos	Rebobinado / Muestreo por Rechazo	ROM; analizado en el QROM	Segura en el QROM sin modificaciones [21]; reducción concreta para firmas de esta forma [23].
Framework LNP [12]	M-SIS / M-LWE (Anillos $\mathcal{R}_q$)	Rebobinado / compromisos sobre retículos	ROM	Análisis en el modelo de oráculo aleatorio [12].
Arquitectura LAASSO (SLE) [18]	Module-LWE / Module-SIS	**Extractabilidad en Línea Recta (SLE)**	ROM con CRS simulada	La extracción no rebobina, por lo que evita el obstáculo de la Sección 2.2.3 [18].

Evaluación Cuantitativa de Métricas de Rendimiento La viabilidad operativa de un sistema ZKP poscuántico, en particular cuando el probador es un dispositivo con recursos limitados y se busca un protocolo liviano, se mide a través del tamaño de la prueba generada, la latencia computacional de generación/verificación y la complejidad de las operaciones en el anillo $\mathbb{Z}_q$ o $\mathcal{R}_q$ [12, 18].

Las construcciones combinatorias tipo Stern sufren de una sobrecarga de tamaño inaceptable para canales de bajo ancho de banda, del orden del megabyte para relaciones básicas, debido a las múltiples repeticiones necesarias para mitigar el error de solidez de 2/3 [11, 12]. En contraste, los marcos basados en compromisos sobre polinomios modulares, como LNP, reducen la prueba de una relación lineal básica a unos 14 KB [12]. La Tabla 3 resume las cifras que reporta cada fuente; como corresponden a relaciones, parámetros y niveles de seguridad distintos, no son directamente comparables entre sí.

Síntesis La comparación anterior deja tres conclusiones para un protocolo de autenticación liviano, en el que el probador solo necesita demostrar que conoce la clave secreta asociada a su clave pública:

Tabla 3: Matriz Técnico-Cuantitativa de Frameworks ZKP Poscuánticos (valores reportados por cada fuente)

Framework / Protocolo	**Tamaño de Prueba**	**Tiempo**	**Operaciones Dominantes ($\mathbb{Z}_q/\mathcal{R}_q$)**	**Relación Soportada**
Pruebas Combinatorias (Stern / Cayrel) [11, 12]	37.5 KB por identificación con error de solidez 2^{-16} (Cayrel et al.); más de 1 MB con 2^{-128} (tipo Stern)	No reportado	Permutaciones y funciones hash	Relaciones SIS binarias/ternarias.
Circuitos Aritméticos Generales (Baum y Nof) [19]	179–357 KB (SIS binario, cuerpo de unos 32 bits)	Menos de 0.5 s	Productos internos modulares	Circuitos aritméticos arbitrarios.
Framework LNP (Lyubashevsky et al.) [12]	Unos 14 KB (relación lineal básica)	No reportado	Multiplicación Matriz-Vector NTT en $\mathcal{R}_q$	Restricciones lineales y cuadráticas.
Arquitectura LAASSO (SLE + Token Hiding) [18]	Unos 62 KB (token de acceso completo)	SignOn 492.67 ms; Verify 204.85 ms	Multiplicación NTT + Cifrado Kyber/AES	Relaciones lineales y de firmas sobre credenciales, y ocultación de tokens.

1. **La plantilla de Fiat–Shamir con abortos basta y es la más compacta.** En la forma comprimida de Dilithium (Secciones 2.2.1, 2.2.2 y 2.2.2), la prueba consiste en una respuesta $\mathbf{z}$ y una semilla del desafío de 32 bytes, sin compromisos ni pruebas auxiliares, y ocupa unos pocos kilobytes (2 420 bytes una firma de Dilithium2 [15]). Los marcos más generales, como LNP o LAASSO, demuestran relaciones más ricas a cambio de pruebas de unos 14 KB o tokens de unos 62 KB [12, 18].

2. **Su seguridad está analizada frente a adversarios cuánticos.** Es una NIZK en el modelo de oráculo aleatorio programable y un argumento de conocimiento para una relación relajada, y su seguridad en el QROM está estudiada en la literatura [21, 23, 24, 25].

3. **La privacidad frente a intermediarios requiere cifrar la prueba.** La ocultación del token con un KEM y un AEAD (Sección 2.2.5) convierte al intermediario en un canal ciego, que no puede leer la prueba ni la identidad del probador, y los tickets de un solo uso con marca de tiempo dan frescura (Sección 2.2.5).

2.3. Learning With Errors (LWE)

2.3.1. Introducción a la Criptografía Basada en Retículos

Definición Matemática de Retículo y Geometría Discreta En el ámbito del álgebra lineal y la geometría discreta, un retículo (o *lattice*) Λ en el espacio euclídeo m-dimensional $\mathbb{R}^m$ se define formalmente como un subgrupo discreto del espacio vectorial real generado por todas las combinaciones lineales enteras de n vectores linealmente independientes $\mathbf{g}_1, \mathbf{g}_2, \ldots, \mathbf{g}_n \in \mathbb{R}^m$ (con $m \geq n$) [31, 18]:

$$\Lambda = \Lambda(\mathbf{G}) = \left\{ \sum_{i=1}^{n} z_i \mathbf{g}_i \;\middle|\; z_i \in \mathbb{Z} \right\} \tag{20}$$

donde $\mathbf{G} = [\mathbf{g}_1 | \mathbf{g}_2 | \ldots | \mathbf{g}_n] \in \mathbb{R}^{m \times n}$ representa la matriz de base del retículo, mientras que los enteros n y m denotan el rango y la dimensión del retículo, respectivamente [31]. Cuando el rango iguala a la dimensión ($n = m$), se afirma que el retículo es de rango completo (*full-rank*) [18]. Geométricamente, un retículo constituye una estructura periódica discreta e infinita sobre el espacio vectorial [31].

En el diseño de protocolos criptográficos poscuánticos, adquieren especial relevancia los retículos modulares discretos definidos sobre el anillo de enteros $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ (donde q es un módulo primo o entero), conocidos como retículos q-arios (q-*ary lattices*) [31, 26]. Para una matriz modular $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, se definen los siguientes conjuntos duales fundamentales:

$$\Lambda_q^{\perp}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}\mathbf{x} \equiv \mathbf{0} \pmod{q}\} \tag{21}$$

$$\Lambda_q(\mathbf{A}) = \left\{\mathbf{x} \in \mathbb{Z}^m \;\middle|\; \mathbf{x} \equiv \mathbf{A}^T\mathbf{s} \pmod{q}, \text{ para algún } \mathbf{s} \in \mathbb{Z}_q^n\right\} \tag{22}$$

así como el retículo desplazado $\Lambda_{\mathbf{u}}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}\mathbf{x} \equiv \mathbf{u} \pmod{q}\}$ para un vector $\mathbf{u} \in \mathbb{Z}_q^n$ [26]. Asimismo, sobre anillos ciclotómicos $\mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^d + 1\rangle$, la estructura puede extenderse a retículos ideales o de tipo NTRU $\Lambda_{h,q} = \{(f_1, f_2) \in \mathcal{R}_q \times \mathcal{R}_q \mid f_1 + f_2 h \equiv 0 \pmod{q}\}$ [18].

Problemas Duros del Peor Caso en Retículos La solidez de la criptografía basada en retículos reposa sobre la intratabilidad computacional de diversos problemas geométricos multidimensionales [31, 32, 18, 26]:

- **Problema del Vector más Corto (Shortest Vector Problem - SVP):** Dada una base arbitraria $\mathbf{G}$ de un retículo Λ, el objetivo consiste en hallar el vector no nulo de menor norma euclídea posible en Λ [31].

- **Problema del Vector más Cercano (Closest Vector Problem - CVP):** Dada una base $\mathbf{G}$ de un retículo Λ y un vector objetivo $\mathbf{x}_0 \notin \Lambda$, el objetivo es encontrar el vector del retículo $\mathbf{x}' \in \Lambda$ más cercano a $\mathbf{x}_0$ [31].

- **Solución de Enteros Cortos (Short Integer Solution - SIS):** Dada una matriz distribuida uniformemente al azar $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ y una cota de norma $\mathcal{B} > 0$, el problema $\text{SIS}_{n,m,q,\mathcal{B}}$ consiste en encontrar un vector no nulo $\mathbf{x} \in \mathbb{Z}^m \setminus \{\mathbf{0}\}$ tal que [18, 26]:

$$\mathbf{A}\mathbf{x} \equiv \mathbf{0} \pmod{q} \quad \text{y} \quad \|\mathbf{x}\|_2 \leq \mathcal{B} \tag{23}$$

  Resolver SIS equivale a hallar un vector corto en el retículo ortogonal $\Lambda_q^{\perp}(\mathbf{A})$ [26].

- **Solución de Enteros Cortos en Módulos (Module-SIS - MSIS):** Es la extensión algebraica del problema SIS sobre anillos cociente $\mathcal{R}_q$. Dada una matriz de polinomios $\mathbf{A} \in \mathcal{R}_q^{n \times m}$ y un límite de norma $\mathcal{B}$, se busca un vector de polinomios $\mathbf{x} \in \mathcal{R}_q^m \setminus \{\mathbf{0}\}$ tal que $\mathbf{A}\mathbf{x} \equiv \mathbf{0} \pmod{q}$ con $\|\mathbf{x}\| \leq \mathcal{B}$, donde la norma puede ser la euclídea o la norma infinito según la aplicación [18].

Fundamento de Reducción del Peor Caso al Caso Promedio y Resistencia Cuántica Los esquemas criptográficos tradicionales como RSA, ECC y ElGamal basan su seguridad en problemas algebraicos clásicos (factorización de enteros y logaritmo discreto) [31, 33, 32, 34]. No obstante, en 1994, Peter Shor formuló un algoritmo cuántico capaz de resolver el logaritmo discreto y la factorización entera en tiempo polinomial $O(n^3)$, lo que pone en riesgo la infraestructura de seguridad precuántica [31, 32, 34].

Frente a esta amenaza, Ajtai (1996) introdujo la criptografía basada en retículos [31, 32]. Su propiedad más revolucionaria reside en las reducciones de seguridad del peor caso al caso promedio (*worst-case to average-case reductions*) [32, 26]: la capacidad de resolver instancias promedio elegidas al azar de problemas como SIS o LWE implica la capacidad de resolver instancias en el peor caso de problemas geométricos subyacentes como SVP o SIVP sobre cualquier retículo de alta dimensión [32, 26]. Dado que no existen algoritmos cuánticos conocidos capaces de resolver las versiones aproximadas de SVP o CVP con factores de aproximación polinomiales en tiempo subexponencial, los retículos ofrecen un fundamento matemático resistente a ataques de la era poscuántica [31, 33, 32, 18, 34, 35].

2.3.2. El Problema Learning with Errors (LWE) Estándar

Formulación Matemática: Versiones de Búsqueda y Decisión El problema *Learning with Errors* (LWE), introducido originalmente por Oded Regev [36], constituye una de las

primitivas fundamentales de la criptografía basada en retículos poscuánticos [34]. Matemáticamente, LWE puede entenderse como el problema de resolver un sistema de ecuaciones lineales sobredeterminado y ruidoso sobre el anillo finito $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$, donde $q \geq 2$ representa un módulo entero [36, 35].

Para definir formalmente el problema, sea $n \in \mathbb{N}$ la dimensión del vector secreto no revelado $\mathbf{s} \in \mathbb{Z}_q^n$, y sea χ una distribución de probabilidad con soporte sobre los enteros $\mathbb{Z}$ (frecuentemente una distribución Gaussiana discreta χ_σ de media cero y desviación estándar σ) [33, 35]. Una muestra individual de LWE se genera seleccionando un vector de coeficientes uniformemente al azar $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ y muestreando un escalar de ruido $e \leftarrow \chi$; el resultado es el par $(\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ definido por:

$$b = \mathbf{a}^T\mathbf{s} + e \pmod{q} \tag{24}$$

Cuando se recolectan m muestras independientes en formato matricial, la matriz pública se denota como $\mathbf{A} = [\mathbf{a}_1|\mathbf{a}_2|\dots|\mathbf{a}_m] \in \mathbb{Z}_q^{n\times m}$, el vector de errores aleatorios como $\mathbf{e} = (e_1, e_2, \dots, e_m)^T \leftarrow \chi^m$ y el de muestras como $\mathbf{b} = (b_1, b_2, \dots, b_m)^T \in \mathbb{Z}_q^m$. Como cada muestra es $b_i = \mathbf{a}_i^T\mathbf{s} + e_i$ y las filas de $\mathbf{A}^T$ son los vectores $\mathbf{a}_i^T$, el sistema completo de ecuaciones ruidosas se expresa algebraicamente como:

$$\mathbf{b} = \mathbf{A}^T\mathbf{s} + \mathbf{e} \pmod{q} \tag{25}$$

El problema LWE se formaliza analíticamente en dos variantes computacionales fundamentales [36]:

- **LWE en versión Búsqueda (Search-LWE):** Dada una matriz de coeficientes aleatorios $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$ y el vector de muestras ruidosas $\mathbf{b} = \mathbf{A}^T\mathbf{s} + \mathbf{e} \pmod{q}$, el objetivo computacional consiste en determinar de forma exacta el vector secreto $\mathbf{s} \in \mathbb{Z}_q^n$.
- **LWE en versión Decisión (Decision-LWE):** El objetivo de un algoritmo probabilístico de tiempo polinomial (PPT) es distinguir con ventaja no despreciable entre la distribución de pares genuinos LWE $(\mathbf{A}, \mathbf{A}^T\mathbf{s} + \mathbf{e} \bmod q)$ y una distribución uniformemente aleatoria $(\mathbf{A}, \mathbf{u}) \in \mathbb{Z}_q^{n\times m} \times \mathbb{Z}_q^m$.

Intuición técnica: Sin la adición de la variable de error $e \leftarrow \chi$, el sistema lineal $\mathbf{b} = \mathbf{A}^T\mathbf{s} \pmod{q}$ sería un problema algebraico trivial resoluble mediante la eliminación Gaussiana clásica en tiempo polinomial $O(n^3)$. La introducción del ruido aleatorio acotado e destruye la estructura algebraica lineal exacta del sistema, convirtiéndolo en un problema geométricamente intrincado de retículos multidimensionales [36, 34].

Parámetros Críticos y Trade-offs de Seguridad y Rendimiento El nivel de seguridad de la primitiva LWE y su eficiencia práctica en la construcción de protocolos están gobernados por el balance directo entre cuatro parámetros interdependientes [36, 35]:

- **Dimensión Secreta (n):** Determina el rango espacial del vector secreto $\mathbf{s} \in \mathbb{Z}_q^n$. Gobierna la dureza asintótica del problema frente a algoritmos de reducción de bases de retículos (ej. BKZ o algoritmos de cribado *Sieve*) [34]. Aumentar n incrementa exponencialmente el costo de los ataques conocidos, clásicos y cuánticos, pero eleva cuadráticamente el tamaño de las matrices públicas $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ [35, 18].

- **Módulo Aritmético (q):** Define la aritmética modular sobre la cual se realizan todas las sumas y multiplicaciones. La relación entre el módulo q y la magnitud del error σ (la razón módulo-ruido q/σ) es crítica. Un cociente q/σ elevado simplifica la corrección y reconciliación de errores en el receptor, pero reduce la dureza geométrica de LWE [35, 32].

- **Número de Muestras (m):** Representa el número de ecuaciones lineales expuestas al adversario, que puede usar todas o solo una parte de ellas. En aplicaciones reales, m se fija polinomial en la dimensión secreta ($m = \text{poly}(n)$) [35].

- **Distribución de Error (χ):** Define la distribución aleatoria sobre $\mathbb{Z}$ de los términos e. Mientras que el análisis teórico clásico utiliza distribuciones Gaussianas discretas χ_σ, implementaciones de alto rendimiento reemplazan la Gaussiana por la *Distribución Binomial Centrada* (CBD con parámetro η), la cual permite realizar el muestreo en tiempo constante $O(1)$ [35, 34].

Fundamentación Teórica: Reducción del Peor Caso al Caso Promedio El aspecto teórico más relevante de LWE radica en la existencia de una reducción rigurosa que conecta la dureza del problema en el *caso promedio* (average-case) con la intratabilidad de problemas geométricos de retículos en el *peor caso* (worst-case) [36, 32].

Regev [36] demostró formalmente, mediante una reducción cuántica, que si el error es gaussiano (discretizado) con parámetro $\alpha q \geq 2\sqrt{n}$, con $0 < \alpha < 1$, resolver Decision-LWE en el caso promedio para instancias aleatorias $(\mathbf{A}, \mathbf{b})$ es al menos tan difícil como aproximar cuánticamente, en el peor caso, problemas geométricos sobre cualquier retículo de dimensión n (tales como las versiones de aproximación polinomial del *Shortest Independent Vectors Problem* SIVP_γ o de GapSVP_γ, con $\gamma = \tilde{O}(n/\alpha)$) [36, 32].

Esta garantía implica que, para los parámetros que cubre la reducción, una instancia de LWE generada al azar no es más fácil que los peores casos de esos problemas, a diferencia

de lo que ocurre con los esquemas criptográficos tradicionales (RSA o ECC), cuya seguridad depende únicamente de la dificultad promedio de factorizar enteros o calcular logaritmos discretos en grupos específicos [32, 34].

La reducción, sin embargo, no da una garantía concreta para los parámetros que se usan en la práctica. Exige error gaussiano con $\alpha q \geq 2\sqrt{n}$, es decir, una desviación estándar de unas 22 unidades para $n = 768$, la dimensión de ML-KEM-768, mientras que ML-KEM usa ruido CBD(2), con desviación estándar $\sigma = 1$ (Sección 2.3.4) [27]. Para esos parámetros, la seguridad se apoya en la estimación del costo de los mejores ataques conocidos (Sección 2.3.6).

2.3.3. Variantes Algebraicas y Estructurales del Problema LWE

Ring-LWE (RLWE) y la Aceleración mediante la Transformada de Teoría de Números (NTT) El problema *Learning with Errors* (LWE) estándar impone requerimientos de almacenamiento y sobrecarga computacional prohibitivos para muchos sistemas prácticos, dado que su matriz pública exige un espacio en memoria de orden $O(n^2)$. Para resolver esta limitación, se introdujo *Ring-Learning with Errors* (RLWE), una variante algebraicamente estructurada que traslada la aritmética desde el cuerpo finito $\mathbb{Z}_q$ hacia un anillo polinomial cociente $\mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^d + 1\rangle$, donde $x^d + 1$ es un polinomio ciclotómico irreducible sobre los racionales (con d potencia de 2) y q es un módulo primo [33, 34].

La matriz pública de LWE se reemplaza en RLWE por un único polinomio $a(x) \leftarrow \mathcal{R}_q$ elegido uniformemente al azar, lo que reduce drásticamente el espacio de la clave pública de $O(n^2)$ a $O(n)$ [33]. El secreto $s(x)$ y el error $e(x)$ se definen como polinomios en $\mathcal{R}_q$, y los coeficientes del error se muestrean desde una distribución discreta Gaussiana χ_σ (o desde una distribución binomial centrada en implementaciones prácticas). Una muestra del problema decisional RLWE se genera algebraicamente como:

$$b(x) = a(x) \cdot s(x) + e(x) \pmod{q} \tag{26}$$

La principal ventaja de RLWE en cuanto a eficiencia está en la multiplicación de polinomios, que es la operación más frecuente en estos esquemas. Si dos polinomios de d coeficientes se multiplican de la forma tradicional, término a término, cada coeficiente de uno debe multiplicarse por cada coeficiente del otro: hacen falta d^2 productos, que para $d = 256$ son 65 536. La Transformada de Teoría de Números (NTT) reduce ese costo a un número de operaciones del orden de $d \log_2 d$, unas 2 048 cuando $d = 256$ [33].

La idea. Un polinomio puede representarse de dos maneras: por sus coeficientes o por los valores que toma en d puntos distintos. Con la primera representación, multiplicar es costoso; con la segunda es inmediato, porque el valor del producto en cada punto es simplemente el

producto de los dos valores, y bastan d multiplicaciones, una por punto. La NTT es la operación que pasa de la primera representación a la segunda, y la transformada inversa (INTT) hace el camino de vuelta. Se basa en la misma idea que la transformada rápida de Fourier, pero con aritmética exacta módulo q en lugar de números complejos. Evaluar un polinomio en d puntos cualesquiera volvería a costar del orden de d^2 operaciones; por eso la NTT usa como puntos las potencias de una *raíz de la unidad* de $\mathbb{Z}_q$, es decir, de un elemento que, elevado a cierta potencia, da 1. Con esos puntos, evaluar un polinomio en los d puntos se reduce a evaluar dos polinomios de la mitad de tamaño en la mitad de los puntos, porque los cuadrados de las raíces de la unidad coinciden de dos en dos, y al repetir esa división $\log_2 d$ veces se llega al costo de $d \log_2 d$. Multiplicar con la NTT se parece a traducir dos textos a un idioma en el que la cuenta es inmediata, hacerla en ese idioma y traducir el resultado de vuelta: cada traducción cuesta del orden de $d \log_2 d$ operaciones, y la cuenta, solo d.

La reducción módulo x^d+1. En $\mathcal{R}_q$, el producto se reduce con la regla $x^d = -1$: los términos de grado mayor que $d-1$ vuelven al principio con el signo cambiado, lo que se conoce como convolución *negacíclica* o envuelta negativa. Para obtener directamente ese producto, sin tener que duplicar la longitud de los polinomios rellenándolos con ceros (*zero-padding*), se usa una raíz primitiva $2d$-ésima de la unidad, ψ, que existe cuando $q \equiv 1 \pmod{2d}$, junto con su cuadrado $\zeta = \psi^2$, que es una raíz primitiva d-ésima. Primero se multiplica el i-ésimo coeficiente de cada factor, f y g, por ψ^i, con lo que se obtienen los vectores modificados f' y g'; así, la multiplicación en $\mathcal{R}_q$ se convierte en un producto elemento a elemento ($\odot$) en el dominio transformado:

$$(f \cdot g)' = \mathrm{INTT}_\zeta^d \left(\mathrm{NTT}_\zeta^d(f') \odot \mathrm{NTT}_\zeta^d(g') \right), \tag{27}$$

y el producto $f \cdot g$ se recupera multiplicando su i-ésimo coeficiente por ψ^{-i} [33]. Si q solo cumple $q \equiv 1 \pmod{d}$, la transformada se detiene un nivel antes y deja productos de polinomios de grado 1 en lugar de productos de números (NTT incompleta). ML-DSA, con $q = 8\,380\,417 \equiv 1 \pmod{512}$, usa la NTT completa para $d = 256$; ML-KEM, con $q = 3329$, que solo cumple $3329 \equiv 1 \pmod{256}$, usa la incompleta [27, 17].

Este enfoque minimiza la latencia computacional y el área de silicio requerida en hardware, y por eso RLWE se ha convertido en un pilar fundamental de la criptografía poscuántica [33, 34].

Module-LWE (MLWE) Aunque RLWE logra una reducción dramática en la sobrecarga espacial y temporal, la estructura de retículo ideal subyacente impone simetrías algebraicas rigurosas que suscitan ciertas prevenciones teóricas en materia de seguridad. *Module-Learning with Errors* (MLWE) surge como un marco algebraico intermedio que logra un balance óptimo

entre la seguridad estricta de LWE sobre retículos no estructurados y la alta eficiencia de RLWE [34, 35]. En lugar de trabajar con escalares o con un único polinomio de grado elevado, MLWE emplea matrices de dimensión reducida $k \times k$ (donde $k \geq 1$ es la dimensión del módulo) cuyos elementos son polinomios pertenecientes al anillo $\mathcal{R}_q$.

Bajo este paradigma, la matriz pública se define como $\mathbf{A} \in \mathcal{R}_q^{k \times k}$, mientras que el vector secreto $\mathbf{s} \in \mathcal{R}_q^k$ y el vector de error $\mathbf{e} \in \mathcal{R}_q^k$ están formados por k polinomios independientes muestreados de la distribución de ruido. Una muestra del problema decisional de módulo (DMLWE) se expresa como la tupla:

$$(\mathbf{A},\, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod q) \in \mathcal{R}_q^{k \times k} \times \mathcal{R}_q^k \tag{28}$$

Es la misma forma $\mathbf{b} = \mathbf{A}^T\mathbf{s} + \mathbf{e}$ del sistema de LWE con la matriz renombrada: como $\mathbf{A}$ es cuadrada y uniforme, $\mathbf{A}$ y $\mathbf{A}^T$ tienen la misma distribución. Escribirla como $\mathbf{A}\mathbf{s} + \mathbf{e}$ es la convención de Kyber y Dilithium [28, 15].

Además de atenuar las simetrías de los ideales de anillo, MLWE aporta la propiedad operativa de la escalabilidad modular [35]: permite incrementar el nivel de seguridad del sistema (por ejemplo, ajustándose a las categorías 1, 3 o 5 de estandarización del NIST) modificando únicamente el parámetro matricial k ($k = 2, 3, 4$), sin necesidad de modificar el polinomio base $x^{256}+1$ ni rediseñar los coprocesadores hardware concebidos para la NTT. Esta flexibilidad ha llevado a que estándares internacionales como *CRYSTALS-Kyber* fundamenten la seguridad de su mecanismo de encapsulación de claves (KEM) directamente en la dureza de MLWE [34].

Binary-Secret LWE (BLWE) *Binary-Secret Learning with Errors* (BLWE) constituye una especialización de LWE en la que la selección de las componentes del vector secreto $\mathbf{s}$ se restringe estrictamente al dominio binario $\{0, 1\}^n$ [26]. Restringir el secreto al alfabeto binario sustituye las multiplicaciones vectoriales de alto costo por sumas vectoriales condicionales simples y reduce sensiblemente la huella de memoria de las claves privadas, lo que lo hace muy atractivo para la construcción de protocolos para dispositivos de recursos muy restringidos y en esquemas de reconocimiento mutuo anónimo (*secret handshakes*) [26].

Learning with Rounding (LWR) y Ring-LWR A diferencia de LWE o RLWE, cuyas pruebas de reducción de seguridad dependen de la adición estocástica de un término de error e muestreado en tiempo de ejecución, la variante *Learning with Rounding* (LWR) elimina la necesidad de generar ese ruido y lo sustituye por una operación de redondeo modular determinista [34]. Específicamente, el error de cuantización inducido al escalar la muestra desde un módulo mayor q hacia un módulo menor $p < q$ mediante el redondeo al entero más

cercano $\left\lfloor \frac{p}{q} \cdot x \right\rceil$ reemplaza funcionalmente al ruido aleatorio.

Esquemas derivados como *Lizard*, que combina LWE y LWR, y su versión sobre anillos, *Ring-Lizard*, aprovechan el redondeo determinista de LWR [34]. Al eliminar el muestreo del error en tiempo de ejecución (los secretos siguen siendo aleatorios), estas variantes reducen el consumo de aleatoriedad y de energía, lo que las hace atractivas para dispositivos con recursos limitados y entornos con restricciones severas de cómputo [34].

Pairing with Error (PWE) y Decision PWE (DPWE) Las suposiciones de dureza *Pairing with Error* (PWE) y su variante decisional (DPWE) se diseñan e introducen primordialmente para respaldar las demostraciones formales de seguridad en protocolos de Intercambio de Claves Autenticado por Contraseña (PAKE) en entornos poscuánticos [35, 32]. En estos esquemas, las partes intervinientes calculan productos LWE ruidosos y utilizan mecanismos de reconciliación modular de errores (como los esquemas de Peikert o Ding) para corregir las discrepancias producidas por el error e y concertar una clave simétrica idéntica.

La hipótesis PWE/DPWE establece formalmente que la clave derivada mediante dicha función de reconciliación permanece computacionalmente indistinguible de un valor uniforme aleatorio para cualquier observador externo que intercepte tanto los mensajes LWE públicos como la señal de reconciliación enviada por el canal abierto [35, 32].

2.3.4. Muestreo y Gestión del Ruido: De la Distribución Gaussiana a la Distribución Binomial Centrada (CBD)

Distribución Gaussiana Discreta y Algoritmos Clásicos de Muestreo En la criptografía basada en retículos, la seguridad del problema *Learning with Errors* (LWE) y sus variantes algebraicas depende intrínsecamente de la inyección de un término de error no determinista e impredecible [36, 35]. Tradicionalmente, este ruido se modela mediante una distribución Gaussiana discreta χ_σ sobre el anillo de enteros $\mathbb{Z}$ o sobre un retículo n-dimensional $\Lambda \subset \mathbb{R}^n$, centrada en el origen y parametrizada por la desviación estándar $\sigma > 0$ [33, 31, 32].

La función de probabilidad Gaussiana discreta centrada en el origen asigna a cada vector $\mathbf{x} \in \Lambda$ una probabilidad proporcional a su peso exponencial euclídeo [31]:

$$D_{\Lambda,\sigma}(\mathbf{x}) = \frac{\exp\left(-\frac{\|\mathbf{x}\|^2}{2\sigma^2}\right)}{\sum_{\mathbf{x}' \in \Lambda} \exp\left(-\frac{\|\mathbf{x}'\|^2}{2\sigma^2}\right)} \tag{29}$$

El numerador da más peso a los vectores cercanos al origen, y el denominador, que suma ese mismo peso sobre todo el retículo, hace que las probabilidades sumen 1.

En la literatura, la generación de muestras pertenecientes a la distribución Gaussiana discreta se realiza históricamente mediante dos familias de algoritmos principales [31, 33]:

- **Algoritmo de Knuth-Yao:** Diseñado para el muestreo de ruido escalar sobre enteros, representa la distribución teórica a través de un Árbol de Grafos de Distribución Discreta (*Discrete Distribution Graph* - DDG) [33]. El algoritmo recorre la estructura de árbol consumiendo bits aleatorios de entropía para seleccionar el coeficiente final. Aunque Knuth-Yao demuestra una eficiencia cercana a la entropía teórica ideal (optimizando el número de bits aleatorios requeridos), depende de tablas de búsqueda precalculadas y de un número variable de comparaciones en función de los bits muestreados [33].

- **Algoritmo de Muestreo de Trapdoor GPV (`SampleD`):** Desarrollado por Gentry, Peikert y Vaikuntanathan, se utiliza para extraer vectores cortos en retículos de la forma $\Lambda_q^{\perp}(\mathbf{A})$ mediante el uso de una base de trampa corta $\mathbf{T_A}$ [31]. Dado un vector objetivo $\mathbf{u} \in \mathbb{Z}_q^n$ y una matriz pública $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, el algoritmo $\texttt{SampleD}(\mathbf{T_A}, \sigma, \mathbf{u})$ genera un vector aleatorio $\mathbf{x} \in \mathbb{Z}^m$, distribuido como una Gaussiana discreta de parámetro σ, que satisface la relación $\mathbf{Ax} \equiv \mathbf{u} \pmod{q}$ [31]. La unidireccionalidad (*one-wayness*) de este algoritmo fundamenta la seguridad de esquemas de autenticación basados en trampas [31].

A pesar de sus sólidas propiedades estadísticas, el muestreo Gaussiano discreto clásico presenta severas limitaciones en entornos físicos de ejecución [34, 35]. La necesidad de evaluar funciones trascendentes, usar tablas de búsqueda dependientes de los datos o realizar iteraciones variables en algoritmos como Knuth-Yao induce variaciones en los tiempos de respuesta y en los perfiles de consumo energético, lo que expone al dispositivo a ataques de canal lateral (*Side-Channel Attacks* - SCA) y ataques de temporización (*timing attacks*) [34].

Muestreador de Distribución Binomial Centrada (CBD) Con el objetivo de eliminar la sobrecarga computacional del muestreo Gaussiano y garantizar un comportamiento algebraico homogéneo, las construcciones modernas de retículos (como Kyber, estandarizado como ML-KEM, y Dilithium) adoptan la Distribución Binomial Centrada (*Centered Binomial Distribution* - CBD) [35, 34, 27].

Para un entero $\eta \geq 1$, la distribución CBD(η) aproxima una Gaussiana discreta centrada en cero [35]. Para generar un coeficiente e, el algoritmo toma 2η bits aleatorios independientes y uniformes $r_0, \ldots, r_{\eta-1}, r'_0, \ldots, r'_{\eta-1} \in \{0, 1\}$ y calcula la diferencia de sus pesos de Hamming:

$$e = \sum_{i=0}^{\eta-1} r_i - \sum_{i=0}^{\eta-1} r'_i \in \{-\eta, \ldots, \eta\}, \qquad \Pr[e = j] = \binom{2\eta}{\eta + j} 2^{-2\eta}. \tag{30}$$

Por simetría, la esperanza es $\mathbb{E}[e] = 0$; como cada bit tiene varianza $1/4$, la varianza es $\sigma^2 = 2\eta \cdot \frac{1}{4} = \eta/2$, con desviación estándar $\sigma = \sqrt{\eta/2}$ [35]. El muestreo de un polinomio de grado d consume $2\eta d$ bits aleatorios, procesados mediante operaciones de bits paralelas sobre palabras del procesador.

Por ejemplo, ML-KEM-768 y ML-KEM-1024 muestrean su secreto y su error con $\eta = 2$ ($\sigma = 1$, coeficientes en $\{-2, \ldots, 2\}$), y ML-KEM-512 usa $\eta = 3$ para el secreto ($\sigma \approx 1{,}22$) [27]. El valor más probable es siempre 0: con $\eta = 2$, con probabilidad $\binom{4}{2}/16 = 0{,}375$. Aumentar η ensancha el ruido, porque σ crece como $\sqrt{\eta}$; para un módulo fijo eso endurece LWE, al reducir la razón q/σ, a cambio de errores mayores que el resto del esquema debe tolerar.

Ventajas de Implementación de la CBD La sustitución de la distribución Gaussiana discreta por la CBD facilita implementaciones seguras en dispositivos físicos [35, 34]:

- **Ejecución en Tiempo Constante:** A diferencia del algoritmo Knuth-Yao o del muestreo por rechazo Gaussiano, la CBD no contiene bucles condicionales de duración indeterminada ni saltos de ejecución (*if-else*) dependientes de datos secretos [35, 34]. Cada coeficiente se calcula mediante una secuencia fija y acotada de instrucciones booleanas (`AND`, `XOR`, desplazamientos de bits y conteo de peso de Hamming), de modo que el tiempo de cómputo puede hacerse independiente de los valores muestreados [35].

- **Eliminación de Tablas de Búsqueda y Protección contra SCA:** Los accesos a memoria indexados por secretos en tablas de probabilidad Gaussiana constituyen una fuente principal de fugas por emisiones electromagnéticas y análisis diferencial de potencia (DPA) [34]. Al operar directamente sobre registros del procesador mediante operaciones binarias, la CBD permite una firma de consumo de energía uniforme y predecible, que dificulta los ataques de canal lateral de traza única (*single-trace side-channel attacks*) [34].

- **Eficiencia y Compactación de Memoria:** El algoritmo CBD elimina por completo la necesidad de almacenar tablas de precisión en coma flotante. Como la CBD no es una Gaussiana, la reducción del peor caso no se le aplica directamente; la seguridad de los esquemas que la usan se evalúa estimando los ataques sobre esa distribución concreta (Sección 2.3.6).

Estas son propiedades del algoritmo cuando se implementa en un dispositivo objetivo con instrucciones de tiempo constante; una implementación en un lenguaje de alto nivel

no las hereda automáticamente. Además, en los esquemas que usan muestreo por rechazo (Sección 2.2.2), el probador repite su cálculo un número variable de veces, lo que se refleja en su tiempo de ejecución aunque la distribución de ese número no dependa de la clave.

2.3.5. Mecanismos de Reconciliación de Errores y Mapeo Criptográfico

El Problema de la Dispersión de Ruido en la Extracción de Claves Deterministas En la criptografía basada en el problema *Learning with Errors* (LWE) y sus variantes algebraicas (RLWE y MLWE), las operaciones interactivas entre dos partes involucran la inyección deliberada de errores Gaussianos o binomiales cortos para garantizar la seguridad frente a adversarios clásicos y cuánticos [36, 35]. Sin embargo, esta perturbación estocástica introduce un desafío algebraico fundamental: cuando ambas partes calculan un producto escalar o polinomial compartido sobre el anillo $\mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^d + 1\rangle$, los valores resultantes $\theta \in \mathbb{Z}_q$ y $\theta' \in \mathbb{Z}_q$ no son exactamente iguales, sino que difieren por un término de error acumulado no nulo $\epsilon_\theta \in \mathbb{Z}_q$, tal que $\theta' = \theta + \epsilon_\theta \pmod q$ [35, 32].

En un esquema de autenticación o acuerdo de claves determinista, esta pequeña discrepancia aritmética impediría que ambas entidades obtengan exactamente la misma secuencia de bits de secreto, lo que provocaría fallos continuos de autenticación [35, 34]. Para resolver esta dispersión sin transmitir información en texto claro que comprometa la clave, se emplean técnicas de mapeo discreto y mecanismos de reconciliación de errores [35, 32].

Codificación de Bits en Coeficientes Modulares Para procesar datos binarios discretos dentro del espacio modular $\mathbb{Z}_q$, se requiere una función de codificación que mapee los símbolos del alfabeto binario $\{0, 1\}$ hacia coeficientes polinomiales con la máxima separación modular posible [33, 34].

Dado un módulo entero q, la función de codificación mapea cada bit $\mu_i \in \{0, 1\}$ a un coeficiente polinomial $m_i \in \mathbb{Z}_q$ mediante la siguiente regla determinista [33]:

$$m_i = \text{Encode}(\mu_i) = \begin{cases} 0 & \text{si } \mu_i = 0 \\ \frac{q-1}{2} & \text{si } \mu_i = 1 \end{cases} \pmod q \tag{31}$$

Esta asignación ubica la representación del bit 1 en el punto medio del espectro modular $\mathbb{Z}_q$ (a distancia máxima de 0), lo que maximiza la tolerancia geométrica del sistema frente a las desviaciones causadas por el ruido LWE durante el proceso de descifrado y reconciliación [33, 32].

Mecanismo de Reconciliación de Peikert y Funciones de Redondeo Para permitir que dos entidades concilien el mismo bit secreto a partir de valores ruidosos aproximados

$\theta, \theta' \in \mathbb{Z}_q$, se utiliza el mecanismo de reconciliación de errores de Peikert [37], en la forma en que lo recogen Yang et al. [35]. Para un módulo q se definen los siguientes subconjuntos de $\mathbb{Z}_q$, escritos con representantes centrados:

$$I_0 = \left\{0, 1, \ldots, \left\lfloor \tfrac{q}{4} \right\rceil - 1\right\}, \qquad I_1 = \left\{-\left\lfloor \tfrac{q}{4} \right\rceil, \ldots, -1\right\}, \qquad E = \left[-\tfrac{q}{8}, \tfrac{q}{8}\right) \cap \mathbb{Z} \tag{32}$$

A partir de esta partición, Peikert define tres funciones matemáticas fundamentales [37, 35]:

1. **Función de Redondeo Modular (bit de clave):** Extrae el bit de clave redondeando el valor modular al entero más cercano:

$$\lfloor \theta \rceil_{q,2} = \left\lfloor \frac{2}{q} \cdot \theta \right\rceil \bmod 2 \tag{33}$$

2. **Función de Redondeo Cruzado (*Cross-Rounding*, pista):** Extrae una señal de sincronización $v \in \{0, 1\}$ que indica en qué par de cuadrantes se encuentra el valor:

$$\langle \theta \rangle_{q,2} = \left\lfloor \frac{4}{q} \cdot \theta \right\rceil \bmod 2 \tag{34}$$

3. **Función de Reconciliación** $\mathrm{rec}(\theta', v)$**:** Utiliza el valor ruidoso propio θ' y la pista v enviada por la otra parte para recuperar el bit redondeado de esta:

$$\mathrm{rec}(\theta', v) = \begin{cases} 0 & \text{si } \theta' \in I_v + E \pmod q \\ 1 & \text{en otro caso} \end{cases} \tag{35}$$

La parte que posee θ toma como bit de clave $\lfloor \theta \rceil_{q,2}$ y envía la pista $v = \langle \theta \rangle_{q,2}$; la otra parte, con un valor cercano θ', calcula $\mathrm{rec}(\theta', v)$.

Garantía de Corrección: para un módulo q par, si los dos valores difieren en un error $\theta - \theta' \in E$, el mecanismo garantiza que $\mathrm{rec}(\theta', \langle \theta \rangle_{q,2}) = \lfloor \theta \rceil_{q,2}$, con concordancia total de los bits entre ambas partes [35]. Para un módulo q impar, Peikert aplica antes una duplicación aleatoria $\theta \mapsto 2\theta - \bar{e}$, con $\bar{e} \in \{-1, 0, 1\}$, que lleva los valores a $\mathbb{Z}_{2q}$ y hace uniforme el bit de clave [37, 35]. De forma complementaria, variantes como la función auxiliar modular $f_2(\theta, \mu) = \left(\theta + \mu \cdot \frac{q-1}{2}\right) \bmod q \bmod 2$ permiten obtener resultados idénticos ajustando la paridad modular [32].

Los Cuatro Cuadrantes La pista indica en cuál de dos pares de cuadrantes opuestos se encuentra θ. Para el módulo $q = 3329$ de ML-KEM [27], los cuadrantes de $\mathbb{Z}_{3329}$ son los de

la Tabla 4.

Tabla 4: Cuadrantes de $\mathbb{Z}_{3329}$, bit de clave y pista.

Cuadrante	**Valores de θ**	**Bit de clave $\lfloor\theta\rceil_{q,2}$**	**Pista $\langle\theta\rangle_{q,2}$**
Q_0	$0,\ldots,832$	0	0
Q_1	$833,\ldots,1664$	1	1
Q_2	$1665,\ldots,2496$	1	0
Q_3	$2497,\ldots,3328$	0	1

El bit de clave vale 0 cerca de 0 ($Q_3 \cup Q_0$) y 1 cerca de $q/2$ ($Q_1 \cup Q_2$). La pista vale 0 en Q_0 y Q_2 y 1 en Q_1 y Q_3: los dos cuadrantes de cada par están a media vuelta del círculo modular y tienen bits distintos, de modo que la pista por sí sola no revela el bit. En cada par, el cuadrante cuyo bit es 0 es Q_0 (pista 0) o Q_3 (pista 1). A ellos corresponden $I_0 = \{0,\ldots,831\}$, que es Q_0 sin su último valor, 832, e $I_1 = \{-832,\ldots,-1\} = Q_3$; la tolerancia es $E = \{-416,\ldots,416\}$.

La reconciliación elige, entre los dos cuadrantes que permite la pista, el que está cerca de θ'. Recorriendo los cuatro cuadrantes se comprueba que, si $|\theta' - \theta| \leq 415$, el valor θ' cae en $I_v + E$ exactamente cuando el bit de θ es 0. Por ejemplo, $\theta = 1000 \in Q_1$ da bit 1 y pista $v = 1$; un valor $\theta' = 1000 + \epsilon_\theta$ con $|\epsilon_\theta| \leq 415$ no cae en $I_1 + E = \{-1248,\ldots,415\}$ y la otra parte obtiene también 1. El valor 832, que pertenece a Q_0 pero no a I_0, es el que impide llegar a 416: con $\theta = 832$ y un error de 416, $\theta' = 1248 \notin I_0 + E = \{-416,\ldots,1247\}$.

Módulo Impar sin Duplicación Las tres funciones también pueden aplicarse directamente sobre un módulo impar, sin la duplicación aleatoria. Para $q = 3329$ hay dos consecuencias, comprobadas exhaustivamente sobre los 3 329 valores de $\mathbb{Z}_{3329}$. La concordancia está garantizada para todo error $|\theta' - \theta| \leq 415$, y no hasta $q/8 \approx 416{,}1$. Además, el bit de clave tiene un sesgo pequeño: 1 665 de los 3 329 valores dan bit 0, y dada la pista $v = 0$ el bit vale 1 con probabilidad 832/1665 en lugar de 1/2. Quien use esta variante debe, por tanto, acotar la probabilidad de que el error entre θ y θ' supere 415 y aceptar ese sesgo.

2.3.6. Estimación Concreta de Seguridad

Las reducciones del peor caso establecen que LWE es difícil asintóticamente, pero no indican cuántos bits de seguridad tiene un juego de parámetros concreto. Para ello se estima el costo de los mejores ataques conocidos, siguiendo la metodología de los esquemas estandarizados [38, 28, 15] y la herramienta *lattice-estimator* [39, 40].

Reducción de Bases: BKZ Los ataques a LWE y a SIS reducen la base de un retículo construido a partir de la instancia. El algoritmo BKZ (*Block-Korkine-Zolotarev*) con tamaño de bloque β_{BKZ} recorre la base en pasadas (*tours*) y, en cada posición, resuelve el problema del vector más corto (SVP) en una ventana de dimensión β_{BKZ}. Cuanto mayor es β_{BKZ}, más corta es la base resultante y más caro el ataque. La calidad de la base se modela con la hipótesis de la serie geométrica: tras la reducción, las longitudes de los vectores de Gram–Schmidt decrecen geométricamente, con una razón determinada por el factor de Hermite

$$\delta = \left(\frac{(\pi \beta_{\mathrm{BKZ}})^{1/\beta_{\mathrm{BKZ}}}\, \beta_{\mathrm{BKZ}}}{2\pi \mathrm{e}} \right)^{\frac{1}{2(\beta_{\mathrm{BKZ}}-1)}}, \tag{36}$$

donde e es la base del logaritmo natural [28].

En términos sencillos. Una base de un retículo es un conjunto de vectores con cuyas combinaciones enteras se obtienen todos sus puntos, y un mismo retículo tiene muchas bases. Unas son "buenas", con vectores cortos y casi perpendiculares entre sí; otras son "malas", con vectores largos y casi paralelos. Con una base buena es fácil encontrar un vector corto del retículo o el punto más cercano a un objetivo; con una mala es muy difícil, y lo que el atacante tiene a la vista es, en la práctica, una base mala. BKZ es el algoritmo que intenta convertir una base mala en una mejor: recorre la base mirando ventanas de β_{BKZ} vectores consecutivos y, en cada ventana, resuelve por completo el problema del vector más corto, que es la parte cara del trabajo. Cuanto mayor es la ventana, mejor queda la base, pero el costo crece exponencialmente con β_{BKZ}. Por eso el tamaño de bloque que un ataque necesita es la medida natural de su costo, y el factor δ, más cercano a 1 cuanto mayor es la ventana, resume la calidad de la base que se consigue.

El Modelo Core-SVP El modelo Core-SVP resume un ataque en el tamaño de bloque β_{BKZ} que necesita y le asigna el costo de una sola llamada a SVP en esa dimensión mediante algoritmos de criba: $2^{0{,}292\,\beta_{\mathrm{BKZ}}}$ operaciones clásicas y $2^{0{,}265\,\beta_{\mathrm{BKZ}}}$ operaciones cuánticas [38]. Es una cota inferior deliberadamente conservadora, pensada para comparar juegos de parámetros entre sí.

Ataques a LWE: Primal, Dual e Híbrido Sobre una instancia LWE $(\mathbf{A}, \mathbf{b} = \mathbf{A}^T\mathbf{s} + \mathbf{e} \bmod q)$, con $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$, se conocen tres familias de ataques basados en la reducción de retículos. La estimación de seguridad es el costo del más barato.

Ataque primal. Convierte la instancia en un problema de vector único más corto: el retículo $\{\mathbf{x} \in \mathbb{Z}^{N_\Lambda} : (\mathbf{A}^T \mid \mathbf{I}_m \mid -\mathbf{b})\,\mathbf{x} \equiv \mathbf{0} \pmod q\}$, de dimensión $N_\Lambda = m + n + 1$ y volumen q^m, contiene el vector corto $(\mathbf{s}, \mathbf{e}, 1)$, porque $\mathbf{A}^T\mathbf{s} + \mathbf{e} - \mathbf{b} \equiv \mathbf{0}$. Si σ es la desviación estándar de

los coeficientes del secreto y del error, BKZ lo encuentra cuando [38, 28]

$$\sigma\sqrt{\beta_{\mathrm{BKZ}}} \leq \delta^{\,2\beta_{\mathrm{BKZ}}-N_\Lambda-1}\, q^{m/N_\Lambda}, \tag{37}$$

donde m es el número de muestras que el atacante decide usar. En términos sencillos: el atacante arma un retículo en el que el secreto y el error, juntos, forman un vector excepcionalmente corto, mucho más que cualquier otro vector del retículo. Si BKZ mejora la base lo suficiente, ese vector aparece en ella y el atacante lee $\mathbf{s}$ directamente. La desigualdad compara dos longitudes: a la izquierda, el tamaño del vector secreto tal como lo ve BKZ, que crece con el ruido σ; a la derecha, la longitud por debajo de la cual BKZ con bloque β_{BKZ} logra detectar un vector, que aumenta con el bloque y con el módulo q. El ataque funciona cuando la izquierda queda por debajo de la derecha; más ruido o un módulo menor obligan al atacante a usar un bloque mayor.

Ataque dual. En lugar de buscar el secreto, busca un vector corto $\mathbf{w} \in \mathbb{Z}^m$ con $\mathbf{A}\mathbf{w} \equiv \mathbf{0}$ (mód q), es decir, un vector corto del retículo $\Lambda_q^{\perp}(\mathbf{A})$ (una instancia de SIS). Para ese vector,

$$\langle \mathbf{w}, \mathbf{b} \rangle = (\mathbf{A}\mathbf{w})^T\mathbf{s} + \langle \mathbf{w}, \mathbf{e} \rangle \equiv \langle \mathbf{w}, \mathbf{e} \rangle \pmod{q}, \tag{38}$$

que es pequeño porque $\mathbf{w}$ y $\mathbf{e}$ lo son, mientras que, si $\mathbf{b}$ fuera uniforme, $\langle \mathbf{w}, \mathbf{b} \rangle$ sería un valor cualquiera de $\mathbb{Z}_q$. Con suficientes vectores $\mathbf{w}$, el atacante distingue las muestras LWE de muestras uniformes (Decision-LWE) [39]. En términos sencillos: el atacante busca una combinación de las ecuaciones, con pesos pequeños, en la que los términos con el secreto se cancelan por completo; lo que queda es una suma de errores pequeños y, por tanto, un número cercano a cero. Con datos aleatorios, la misma combinación da un número cualquiera. La dificultad está en encontrar pesos que cancelen el secreto y sean a la vez pequeños, y para eso también se usa BKZ.

Ataques híbridos. Dividen el secreto en dos partes: adivinan los coeficientes de una de ellas (por búsqueda exhaustiva o con técnicas de encuentro a mitad de camino) y aplican el ataque primal o el dual al problema que queda, de dimensión menor [40]. En términos sencillos: cada coeficiente adivinado achica el retículo que hay que reducir, pero multiplica el número de casos que hay que probar; el atacante busca el punto de equilibrio entre adivinar y reducir. Son especialmente eficaces cuando los coeficientes del secreto toman muy pocos valores, como en LWE con secreto binario (Sección 2.3.3), porque entonces adivinar es barato; por eso esos esquemas necesitan dimensiones mayores para el mismo nivel de seguridad.

Ataques a SIS Para SIS y MSIS, BKZ busca directamente un vector corto en $\Lambda_q^{\perp}(\mathbf{A})$; el costo depende de la norma que se exige a la solución (euclídea o infinito), de la cota $\mathcal{B}$ y del

número de columnas que el atacante decide usar.

La Herramienta El *lattice-estimator* implementa estos modelos y elige, para cada ataque, los parámetros más favorables al atacante [39, 40]. En este trabajo se usó la versión correspondiente al *commit* `53da598` con el modelo de costo Core-SVP. Con esa versión, la herramienta reproduce los tamaños de bloque publicados por los diseñadores de los esquemas estandarizados: 406 para el ataque primal sobre Kyber512, 624 para Kyber768 (publicado: 625) y 423 para el ataque a SIS de Dilithium2 [28, 15].

Lo que Core-SVP No Cuenta Un ataque real paga más que una llamada a SVP. Las estimaciones refinadas de Kyber y Dilithium cuentan además: (i) el número de llamadas: BKZ progresivo cuesta en total unas $C_{\mathrm{BKZ}}\,(N_\Lambda - \beta_{\mathrm{BKZ}})$ veces una criba en dimensión β_{BKZ}, con $C_{\mathrm{BKZ}} = 1/(1-2^{-0{,}292}) \approx 5{,}46$ (unas 3 340 veces para Kyber512); (ii) el costo de cada criba medido en compuertas lógicas, descontando las "dimensiones gratis", que permiten cribar en una dimensión algo menor que β_{BKZ}; y (iii) la memoria: la criba almacena del orden de $2^{0{,}2075\,\beta_{\mathrm{BKZ}}}$ vectores [28]. En el caso cuántico, el exponente 0,265 supone acceso cuántico a una memoria de tamaño exponencial (QRAM), y la especificación de Kyber considera tenue la ventaja cuántica real [28]. Con estas cuentas, Kyber512 pasa de 2^{118} en Core-SVP a unas $2^{151{,}5}$ compuertas y $2^{93{,}8}$ bits de memoria [28], y Dilithium2 pasa de 2^{123} a 2^{159} compuertas y 2^{98} de memoria [15]. Estas penalizaciones por memoria y por recorridos de ventana son un margen adicional sobre la cifra Core-SVP.

Categorías de Seguridad de NIST NIST define sus categorías de seguridad por comparación con primitivas simétricas: la categoría 1 exige que romper el esquema sea al menos tan difícil como buscar exhaustivamente una clave de AES-128 (unas 2^{143} compuertas clásicas), la categoría 3 como buscar una clave de AES-192 y la categoría 5 como buscar una de AES-256 [41]. Kyber512 (ML-KEM-512) se propuso en la categoría 1 con 2^{118} en Core-SVP, apoyándose en el margen de su costo refinado [28]. Una estimación Core-SVP es, por tanto, una herramienta de comparación: no certifica por sí sola una categoría.

2.3.7. Elección del Módulo: Pruebas, Cifrado y Multiplicación Rápida

Los esquemas estandarizados sobre retículos no comparten módulo: ML-KEM usa $q = 3329$, de 12 bits, y ML-DSA usa $q = 8\,380\,417$, de 23 bits [27, 17]. La diferencia responde a tres condiciones que tiran en sentidos opuestos y que debe tener en cuenta cualquier construcción que combine una prueba de Fiat–Shamir con abortos con un cifrado o un acuerdo de clave.

El Muestreo por Rechazo Exige un Módulo Grande En Fiat–Shamir con abortos (Secciones 2.2.2 y 2.2.2), sea n el número de coeficientes de la respuesta y β una cota de $\|c\,\mathbf{s}\|_\infty$. Si la máscara es uniforme en $\{-(\gamma_1-1),\ldots,\gamma_1-1\}$, la probabilidad de que un intento cumpla la condición sobre la respuesta es aproximadamente $\mathrm{e}^{-n\beta/\gamma_1}$; si los bits bajos se comportan como uniformes, la de que cumpla la condición sobre ellos es $\left(\frac{2(\gamma_2-\beta)-1}{2\gamma_2}\right)^n \approx \mathrm{e}^{-n\beta/\gamma_2}$. Ambas exigen que γ_1 y γ_2 sean al menos del orden de $n\beta$; además, γ_1 debe ser menor que $q/2$, y $2\gamma_2$ debe dividir a $q-1$ dejando al menos dos valores de bits altos (Sección 2.2.2). Con $q = 3329$, el mayor paso posible es $2\gamma_2 = 1664$ y, aun con la dimensión y la cota de ML-DSA-44 ($n = 1024$, $\beta = 78$) [17], la condición de los bits bajos se cumpliría con probabilidad

$$\left(\frac{2(832-78)-1}{1664}\right)^{1024} \approx 8{,}5\cdot 10^{-45}\,: \tag{39}$$

la prueba no terminaría nunca. Por eso ML-DSA usa un módulo de 23 bits, que deja espacio para una máscara ancha y un paso de redondeo grande.

Con un Ruido Fijo, un Módulo Mayor Debilita LWE La dificultad de LWE depende de la razón entre el módulo y el ruido: para una dimensión y un ruido fijos, aumentar el módulo facilita el ataque primal, porque el lado derecho de su condición de éxito crece con q (Sección 2.3.6). Un módulo grande, elegido por las necesidades de la prueba, obliga a compensar con más dimensión o con más ruido. En sentido contrario, un cifrado de tipo ML-KEM necesita un ruido pequeño frente al módulo para descifrar correctamente, y un módulo pequeño le da textos cifrados más cortos.

La Multiplicación Rápida Condiciona el Módulo La NTT completa sobre $\mathbb{Z}_q[x]/\langle x^{256}+1\rangle$ necesita una raíz primitiva 512-ésima de la unidad, es decir, $q \equiv 1 \pmod{512}$ (Sección 2.3.3). El primo de ML-DSA, $2^{23}-2^{13}+1$, la tiene, porque $q-1 = 2^{13}\cdot 1023$; en cambio, $3329-1 = 2^8\cdot 13$ solo da $3329 \equiv 1 \pmod{256}$, y por eso ML-KEM usa una NTT incompleta [27, 17]. Ambos módulos admiten multiplicación rápida, aunque de forma distinta.

Consecuencia Las tres condiciones explican por qué los estándares usan módulos distintos para la firma y para el encapsulamiento de clave: un módulo pequeño hace impracticable el muestreo por rechazo de Fiat–Shamir con abortos, y uno grande agranda los textos cifrados y exige más ruido para mantener la dificultad de LWE. Un único módulo no satisface bien las tres a la vez.

3. Propuesta del Protocolo de Autenticación Poscuántica: ML-ZKAuth

3.1. Visión General y Modelo de Amenazas

Objetivo del Protocolo Se propone *ML-ZKAuth*, un protocolo de autenticación en el que un dispositivo cliente demuestra a un servidor verificador designado que conoce la clave secreta asociada a su clave pública, mediante un argumento no interactivo de conocimiento cero sobre retículos, a través de una terminal intermedia que no es de confianza [35, 18]. Tras una autenticación aceptada, cliente y servidor pueden acordar además una clave de sesión de 256 bits. El cliente es el probador y el servidor, el verificador. El protocolo busca ser liviano: una autenticación consta de cuatro mensajes, la prueba viaja en un token de unos 3.5 KB y el probador solo necesita aritmética de polinomios, funciones de la familia SHA-3, ML-KEM y AES, operaciones al alcance de dispositivos con recursos limitados. El protocolo combina los elementos del marco teórico: la prueba de Fiat–Shamir con abortos en la forma comprimida de Dilithium (Secciones 2.2.1, 2.2.2 y 2.2.2), la ocultación del token con ML-KEM y AES-GCM (Sección 2.2.5), los tickets y marcas de tiempo para la frescura (Sección 2.2.5) y la reconciliación de Peikert para el acuerdo de clave (Sección 2.3.5). Este capítulo describe el protocolo tal como está implementado en el repositorio del trabajo; la Sección 3.13 indica dónde se encuentra cada paso en el código.

Dos Capas sobre Dos Módulos El protocolo trabaja sobre dos anillos de polinomios con el mismo grado $d = 256$ y módulos distintos (Sección 3.2.1):

1. **Capa de prueba:** $\mathcal{R}_{q'} = \mathbb{Z}_{q'}[x]/\langle x^{256} + 1\rangle$, con $q' = 8\,380\,417$ (el primo de 23 bits de ML-DSA) y rango $k' = 4$. Aquí viven la clave de identidad del cliente y la prueba de conocimiento cero.

2. **Capa de transporte:** $\mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^{256} + 1\rangle$, con $q = 3329$ (el primo de 12 bits de ML-KEM) y rango $k = 3$. Aquí viven el cifrado del token con ML-KEM-768 y AES-256-GCM y el acuerdo de clave de sesión.

Entidades y Actores del Sistema

1. **Cliente ($\mathcal{C}$), el probador:** dispositivo, posiblemente con recursos limitados, que custodia la clave secreta de retículo $(\mathbf{s}, \mathbf{e})$ y su identificador $\mathsf{id}_{\mathcal{C}}$, y conoce los parámetros públicos del sistema, incluida la clave pública de cifrado vpk del servidor [34, 18]. Genera la prueba y el token sin exportar el material secreto. Su aritmética consiste en

productos de polinomios en $\mathcal{R}_{q'}$ y en $\mathcal{R}_q$, y ambos anillos son compatibles con la NTT: completa sobre q', como en ML-DSA, e incompleta sobre q, como en ML-KEM (Sección 2.3.3). Un dispositivo cliente puede, por tanto, reutilizar las implementaciones o los coprocesadores de NTT de esos dos estándares; el prototipo, en cambio, usa la convolución directa (Sección 3.11).

2. **Terminal intermedia ($\mathcal{T}$):** cualquier intermediario que retransmite los mensajes entre $\mathcal{C}$ y $\mathcal{S}$. Se considera potencialmente hostil: puede leer, modificar, retener, reordenar o reenviar mensajes [18].

3. **Servidor ($\mathcal{S}$), el verificador designado:** entidad central que custodia la clave privada vsk de ML-KEM-768, la base de datos de claves públicas de los clientes registrados y el registro de tickets consumidos $\mathcal{L}_{\text{spent}}$ [35, 18].

Modelo de Amenazas El adversario controla la terminal y el canal: ve y manipula todos los mensajes. Puede usar un computador cuántico, por lo que las primitivas deben resistir ataques cuánticos. Se supone que el servidor sigue el protocolo y protege vsk, que el dispositivo del cliente protege $(\mathbf{s}, \mathbf{e})$ y que los relojes del servidor son confiables. Los ataques de canal lateral sobre el dispositivo y la denegación de servicio quedan fuera del alcance.

Propiedades Con esas hipótesis, el protocolo ofrece las siguientes propiedades, cuyos enunciados precisos y supuestos están en la Sección 3.11:

- **Correctitud:** toda prueba honesta es aceptada.
- **Conocimiento cero:** la prueba es una NIZK con conocimiento cero estadístico en el modelo de oráculo aleatorio programable, también frente al servidor, que la descifra.
- **Resistencia a la suplantación:** la prueba es un argumento de conocimiento para una relación relajada; suplantar a un cliente exige resolver instancias de Module-LWE y de SelfTargetMSIS con los parámetros de la capa de prueba.
- **Verificador designado y no vinculabilidad frente a la terminal:** solo $\mathcal{S}$ puede leer la prueba y el identificador del cliente; la terminal solo ve mensajes cifrados de longitud fija.
- **Frescura:** cada prueba sirve para un único reto y dentro de una ventana de tiempo.
- **Acuerdo de clave ligado a la autenticación:** la clave de sesión se acuerda con el cliente que se autenticó, y la terminal no puede sustituir las cuotas del intercambio.

El protocolo no ofrece denegabilidad (Sección 2.2.4) y su implementación no es de tiempo constante.

Notación del Protocolo Además de la notación general de la Sección 2.1, el protocolo usa los símbolos de la Tabla 5. Cuando las dos capas usan objetos análogos, los de la capa de prueba llevan un apóstrofo (por ejemplo, q y q').

Tabla 5: Símbolos propios de ML-ZKAuth.

Símbolo	Significado
$\mathcal{C}$, $\mathcal{T}$, $\mathcal{S}$	Cliente (probador), terminal intermedia y servidor (verificador designado).
q, k, η, $\mathbf{A}$	Módulo, rango, parámetro de ruido y matriz pública de la capa de transporte.
q', k', η', $\mathbf{A}'$	Módulo, rango, parámetro de ruido y matriz pública de la capa de prueba.
ρ	Semilla pública de la que se expanden $\mathbf{A}$ y $\mathbf{A}'$.
$\mathbf{s}$, $\mathbf{e}$, $\mathbf{t}$, $\mathsf{id}_{\mathcal{C}}$	Secreto, error, clave pública e identificador del cliente.
$\mathbf{y}$, $\mathbf{w}$, $\mathbf{w}_1$, $\mathbf{z}$	Máscara, compromiso, bits altos del compromiso y respuesta de la prueba.
vpk, vsk	Claves pública y privada de ML-KEM-768 del servidor.
K_{tok}, K_{resp}	Clave encapsulada en el token y clave derivada de ella para cifrar la respuesta del servidor.
$\mathbf{b}_{\mathcal{C}}$, $\mathbf{b}_{\mathcal{S}}$, $\theta_{\mathcal{C}}$, $\theta_{\mathcal{S}}$, $\mathbf{v}$	Cuotas del acuerdo de clave, valores ruidosos de cada lado y pista de reconciliación.
K_{ses}	Clave de sesión de 256 bits.
H	SHA3-256 sobre la codificación inequívoca de sus argumentos, con una etiqueta de dominio para cada uso.

3.2. Justificación de las Decisiones de Diseño

Esta sección reúne las decisiones de diseño del protocolo. Para cada una explica por qué se tomó, es decir, qué concepto del marco teórico la justifica, y para qué, es decir, qué propiedad o beneficio busca.

Tipo de Prueba Lo único que el cliente debe demostrar es que conoce la clave secreta que corresponde a su clave pública. Para algo tan concreto no se necesitan marcos generales como LNP o LAASSO, pensados para demostrar afirmaciones más complejas a costa de pruebas de unos 14 KB o de tokens de unos 62 KB [12, 18]; es suficiente con el esquema de Fiat–Shamir

con abortos (Sección 2.2.6). Por eso el protocolo usa un argumento no interactivo de conocimiento cero de este tipo, en la versión comprimida que emplea Dilithium (Secciones 2.2.1, 2.2.2 y 2.2.2). La prueba está formada solo por una respuesta $\mathbf{z}$ y una semilla $\tilde{c}$ de 32 bytes, sin compromisos ni pruebas adicionales, y con los parámetros elegidos el token cifrado completo ocupa 3 524 bytes (Sección 3.8).

Para qué: para que la autenticación sea liviana. Un token de unos 3.5 KB puede enviarse por canales con poco ancho de banda y guardarse en la memoria de dispositivos con recursos limitados, y el probador solo necesita aritmética de polinomios y funciones hash. Además, como la seguridad se basa en problemas de retículos, la autenticación resiste los ataques de un adversario que disponga de un computador cuántico, algo que no ofrece ECDSA, el esquema de referencia de la comparación.

Forma Comprimida: el Compromiso No Se Envía En la forma comprimida de Fiat–Shamir, la prueba no incluye el compromiso (Sección 2.2.1). En este protocolo no es solo una cuestión de tamaño: como la clave es de tipo LWE, $\mathbf{t} = \mathbf{A}'\mathbf{s} + \mathbf{e}$, un verificador que recibiera $\mathbf{w}$ podría recuperar $c\,\mathbf{e}$ y, con él, la clave completa. No enviar el compromiso es necesario para el conocimiento cero (Sección 3.7).

Para qué: para proteger la confidencialidad de la clave secreta frente al propio verificador y, de paso, para reducir el token: el compromiso $\mathbf{w}$ ocuparía $k'd = 1024$ coeficientes de 23 bits, 2 944 bytes, frente a los 32 bytes de la semilla $\tilde{c}$.

Garantías de la Prueba Con esa plantilla, la prueba es una NIZK con conocimiento cero estadístico en el modelo de oráculo aleatorio programable, también frente al verificador designado, y un argumento de conocimiento para una relación relajada (Sección 2.2.3), cuya resistencia a la suplantación se reduce a Module-LWE y a SelfTargetMSIS. Su seguridad frente a adversarios cuánticos se apoya en los análisis publicados de la plantilla en el QROM [21, 23, 24, 25]. Los enunciados exactos están en la Sección 3.11.

Para qué: para obtener autenticidad, porque solo quien conoce la clave secreta puede producir una prueba aceptada (resistencia a la suplantación); privacidad del secreto, porque ni siquiera el servidor aprende algo de él (conocimiento cero); y que ambas garantías se mantengan frente a un adversario cuántico.

Ocultación del Token con KEM-DEM Para que la terminal sea un canal ciego, que no pueda leer la prueba ni el identificador del cliente, el protocolo adopta la ocultación de tokens de LAASSO (Sección 2.2.5) [18]. A diferencia de LAASSO, que cifra con Kyber una clave elegida por el probador, usa ML-KEM tal como está estandarizado en FIPS 203: el

encapsulamiento genera la clave, de modo que el token sigue el paradigma KEM-DEM, con AES-256-GCM como cifrado autenticado [27, 30]. Los parámetros son los de ML-KEM-768, cuyo rango $k = 3$ es el que fijaba el manuscrito original del protocolo. El contenido exacto del token se describe en la Sección 3.8.

Para qué: para dar confidencialidad e integridad al contenido frente a la terminal, que no puede leer la prueba ni el identificador del cliente, ni modificarlos sin que falle la etiqueta de autenticación; para que la terminal no pueda relacionar dos autenticaciones del mismo cliente; y para usar primitivas estandarizadas, con implementaciones ya disponibles, en lugar de una construcción propia. El cifrado del contenido es simétrico: AES es rápido y muchos procesadores lo aceleran en hardware.

Frescura con Tickets Emitidos por el Verificador Siguiendo el mecanismo de tickets de LAASSO (Sección 2.2.5), cada prueba se liga a un ticket de un solo uso y a una marca de tiempo. A diferencia de LAASSO, donde un emisor de credenciales asigna los tickets, aquí el ticket lo emite el propio servidor verificador como reto de la sesión, de modo que una sola entidad emite los tickets y lleva el registro de consumo (Secciones 3.3 y 3.6).

Para qué: para evitar ataques de repetición, porque una prueba capturada no sirve en otra sesión, y para acotar con la ventana de tiempo cuánto debe recordarse cada ticket, de modo que el registro de tickets consumidos tenga un tamaño acotado (Sección 3.9). Que el propio verificador emita el ticket evita depender de un tercero y de sincronizar el reloj del cliente.

Privacidad Buscada La no vinculabilidad (Sección 2.2.4) se busca frente a terceros y frente a la terminal, no frente al servidor, que identifica al cliente por diseño. Se apoya en dos elementos: cada prueba usa una máscara nueva, uniforme en una caja, y la respuesta aceptada tiene una distribución que no depende de $\mathbf{s}$ (Sección 3.7); y cada token se cifra con una clave nueva, obtenida por encapsulamiento, con el identificador del cliente dentro de la parte cifrada (Sección 3.8). El protocolo no busca denegabilidad: una vez descifrada, su prueba es una prueba no interactiva que cualquiera puede verificar con la clave pública del cliente y el reto de la sesión, es decir, es transferible, y el servidor podría mostrarla a un tercero como evidencia de que el cliente participó (Sección 2.2.4).

Para qué: para impedir que intermediarios y observadores del canal sigan o perfilen a los clientes. La transferibilidad tiene además una contrapartida: da una forma de no repudio, porque el servidor puede demostrar ante un tercero que el cliente produjo la prueba de esa sesión. Denegabilidad y no repudio son incompatibles, y el protocolo queda del lado del no repudio.

Acuerdo de Clave con Reconciliación sin Duplicación El acuerdo de clave aplica las funciones de Peikert directamente sobre $q = 3329$, sin la duplicación aleatoria (Sección 2.3.5). En consecuencia, la concordancia está garantizada para errores de hasta 415, y no hasta $q/8$, y cada bit de la clave tiene un sesgo de 832/1665 frente a 1/2 cuando la pista vale 0. La Sección 3.10 acota la probabilidad de que el error supere 415.

Para qué: para que, tras autenticarse, cliente y servidor compartan una clave de sesión y se comuniquen con cifrado simétrico, que es rápido y barato, sin un segundo intercambio de claves; las cuotas viajan dentro del token para que la terminal no pueda interponerse (Sección 3.11). Aplicar las funciones sin duplicación mantiene la reconciliación determinista, sin bits aleatorios adicionales, a cambio de ese sesgo pequeño.

Implementación en Python, sin Tiempo Constante El prototipo, escrito en Python con NumPy, no es de tiempo constante: no aprovecha las propiedades de la CBD descritas en la Sección 2.3.4, y el número de intentos del probador se refleja en su tiempo de ejecución, aunque su distribución no dependa de la clave (Sección 3.11).

Para qué: el prototipo sirve para medir y comparar los seis candidatos del benchmark bajo un mismo marco, no para desplegarse. Python con NumPy, dentro de un entorno con SageMath que el experimento usa para estimar la dureza con el *lattice-estimator*, permite prototipar rápido, reutilizar bibliotecas de álgebra y de criptografía y mantener un código compacto y legible, comparable con el de los demás candidatos, en lugar de una base de código extensa en C. El costo es no tener tiempo constante ni protección contra canales laterales, que una implementación de producción debería añadir.

3.2.1. Arquitectura de Módulo Dual

El manuscrito original del protocolo trabajaba sobre un único módulo, $q = 3329$, el de ML-KEM. La prueba de Fiat–Shamir con abortos no es practicable sobre ese módulo (Sección 2.3.7): aun con los parámetros más favorables del manuscrito ($n = 768$, $\beta = 46$), la condición de los bits bajos se cumpliría con probabilidad

$$\left(\frac{2(832-46)-1}{1664}\right)^{768} \approx 6{,}6 \cdot 10^{-20}, \tag{40}$$

y con los parámetros actuales ($n = k'd = 1024$, $\beta = 138$), con una probabilidad del orden de 10^{-81}: la prueba no terminaría nunca. Por eso este trabajo separa el protocolo en dos capas con el mismo grado y módulos distintos:

$$\mathcal{R}_{q'} = \mathbb{Z}_{q'}[x]/\langle x^{256}+1\rangle \quad (q' = 8\,380\,417) \qquad \text{y} \qquad \mathcal{R}_q = \mathbb{Z}_q[x]/\langle x^{256}+1\rangle \quad (q = 3329). \tag{41}$$

La prueba usa $q' = 2^{23} - 2^{13} + 1$, el primo de ML-DSA estandarizado en FIPS 204 [17, 15], que admite los pasos de redondeo de Dilithium, una máscara ancha y la NTT completa ($q' \equiv 1$ (mód 512); Sección 2.3.3). El cifrado del token y el acuerdo de clave conservan $q = 3329$, el módulo de ML-KEM estandarizado en FIPS 203 [27], con el que los textos cifrados son compactos.

El módulo grande tiene un costo para la clave de identidad, porque con un ruido fijo un módulo mayor debilita LWE (Sección 2.3.7): con el ruido CBD(2) de ML-KEM, una clave Module-LWE de rango 4 sobre q' alcanza solo $2^{114,9}$ en el modelo Core-SVP, y con CBD(6) alcanza $2^{128,2}$ (Tabla 7). Por eso la capa de prueba usa su propio parámetro de ruido, $\eta' = 6$ ($\sigma = \sqrt{3}$, coeficientes en $\{-6, \ldots, 6\}$, con el valor 0 como el más probable, de probabilidad $\binom{12}{6}/4096 \approx 0{,}226$; Sección 2.3.4), mientras que la capa de transporte conserva el $\eta = 2$ de ML-KEM.

Para qué: para que la prueba termine en unos 13 intentos, cuando con un único módulo no terminaría; para mantener compactos los textos cifrados y las cuotas de la capa de transporte; y para usar en cada capa un módulo estandarizado, cuyas implementaciones y coprocesadores de NTT ya existen. El ruido $\eta' = 6$ sirve para que la clave alcance la meta común de 2^{128} en Core-SVP, la misma de los demás candidatos de retículos.

3.3. Elementos del Protocolo: Identificador, Ticket y Marca de Tiempo

Antes de describir los mensajes conviene definir los valores que intervienen en ellos.

- **Identificador del cliente.** Es el resumen de su clave pública, $\mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t})$, de 32 bytes. El servidor indexa su base de datos por $\mathsf{id}_{\mathcal{C}}$ y el cliente lo guarda junto a su clave secreta, de modo que no necesita almacenar $\mathbf{t}$ completo (Sección 3.5). Solo viaja cifrado dentro del token. Como el resumen tiene 256 bits, hay $2^{256} \approx 1{,}2 \cdot 10^{77}$ identificadores posibles, y la probabilidad de que dos de N clientes compartan identificador es a lo sumo $N^2/2^{257}$ (cota del cumpleaños). Con $N = 10^{30}$ clientes, más de 10^{20} veces la población mundial, esa probabilidad es de unos $4 \cdot 10^{-18}$; recién con unos $1{,}5 \cdot 10^{31}$ clientes llegaría a 10^{-15}. Además, un despliegue puede rechazar en el registro un identificador repetido.

- **Ticket.** Es un valor aleatorio de 256 bits, $\mathsf{tk} \xleftarrow{\$} \{0,1\}^{256}$, que el servidor emite para un único intento de autenticación. Sirve para que la prueba no pueda reutilizarse: la prueba se calcula sobre tk y el servidor anota cada ticket recibido en $\mathcal{L}_{\text{spent}}$ (Sección 2.2.5).

- **Marca de tiempo.** Es el instante de emisión del ticket según el reloj del servidor, ν, expresado en milisegundos de tiempo Unix. El servidor solo acepta un token dentro de

la ventana $|\nu_{\text{actual}} - \nu| \leq \Delta_\nu$, con $\Delta_\nu = 300$ s. Como la fija el propio servidor, no hace falta sincronizar el reloj del cliente.

- **Reto.** El par (tk, ν) forma el reto de la sesión: es la entrada del verificador que la transformación de Fiat–Shamir incluye en el hash.

3.4. Vector de Parámetros y su Obtención

La Tabla 6 reúne los parámetros e indica de dónde sale cada valor. Los de la capa de transporte son los de ML-KEM-768; los de la capa de prueba combinan el módulo y los parámetros de redondeo de ML-DSA-44 con un rango y un ruido elegidos para que la clave de identidad alcance 2^{128} en el modelo Core-SVP (Sección 2.3.6), la misma meta que los demás candidatos de retículos del benchmark.

Tabla 6: Vector de parámetros de ML-ZKAuth y origen de cada valor.

Parámetro	**Valor**	**Cómo se obtiene**
d	256	Grado de ambos anillos. Potencia de 2, de modo que $x^{256}+1$ es un polinomio ciclotómico; es el grado de ML-KEM y ML-DSA.
q'	8 380 417	Primo $2^{23}-2^{13}+1$ de ML-DSA: cumple $q' \equiv 1$ (mód 512) (NTT completa), cabe en 23 bits y $q'-1$ admite los pasos de redondeo de Dilithium (Sección 3.2.1).
k'	4	Rango de la capa de prueba. Junto con η', el menor que lleva la clave de identidad a 2^{128} en Core-SVP (Tabla 7); la dimensión $k'd = 1024$ es la de ML-DSA-44.
η'	6	Ruido CBD(η') del secreto y del error de identidad. Compensa el módulo grande: con $\eta' = 2$ la clave quedaría en $2^{114{,}9}$.
κ	23	Número de coeficientes ± 1 del desafío: el menor con $\lvert B_\kappa \rvert \geq 2^{128}$ ($\lvert B_{23} \rvert \approx 2^{131{,}1}$; Sección 2.2.2).
β	$\eta'\kappa = 138$	Derivado, no ajustable: cota exacta de $\lVert c\,\mathbf{s} \rVert_\infty$ y $\lVert c\,\mathbf{e} \rVert_\infty$.
γ_1	$2^{17} = 131\,072$	Anchura de la máscara; valor de ML-DSA-44.
γ_2	$(q'-1)/88 = 95\,232$	Mitad del paso de redondeo $\alpha = 2\gamma_2 = 190\,464$, que divide a $q'-1$ y deja 44 valores de bits altos; valor de ML-DSA-44.
q	3329	Módulo de ML-KEM (FIPS 203).
k	3	Rango de ML-KEM-768 y del acuerdo de clave.
η	2	Ruido CBD(2) de ML-KEM-768 y del acuerdo de clave.
Δ_ν	300 s	Decisión de ingeniería: tiempo máximo entre la emisión del reto y la verificación del token.
tk	256 bits	Colisiones despreciables: a lo sumo $N^2/2^{257}$ entre N tickets.
Cifrado	ML-KEM-768 y AES-256-GCM	Encapsulamiento de clave (IND-CCA2) y cifrado autenticado del contenido del token (Sección 2.2.5).

Búsqueda del Rango y del Ruido de la Capa de Prueba Con el módulo q' fijado, la dureza de la clave de identidad depende del rango k' y del ruido η', y el costo para el probador depende de $\beta = \eta'\kappa$ a través de la probabilidad de aceptar cada intento,

$$p_{\text{acep}} = \left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1 - 1}\right)^{k'd} \cdot \left(\frac{2(\gamma_2 - \beta) - 1}{2\gamma_2}\right)^{k'd}, \tag{42}$$

que se justifica en la Sección 3.7. La Tabla 7 recoge la búsqueda: para cada combinación, la dureza de la clave estimada con el *lattice-estimator* en el modelo Core-SVP y el número medio de intentos $1/p_{\text{acep}}$, con γ_1 y γ_2 fijos. La combinación elegida, $k' = 4$ y $\eta' = 6$, es la de menor costo que alcanza 2^{128}.

La tabla puede reproducirse con el repositorio del trabajo, desde la carpeta del experimento (`lwe-auth-project`):

- `make proof-layer-search` calcula la tabla completa. Descarga el *lattice-estimator* en el *commit* `53da598`, si no está presente, y ejecuta el cálculo dentro del contenedor del experimento.
- `./scripts/proof-layer-search.sh --without-hardness` calcula solo las dos últimas columnas, sin el estimador.

El cálculo está en el módulo `backend/benchmarking/proof_layer_search.py`. Para cada par (k', η') construye la misma instancia de la clave que usa la estimación de dureza de los candidatos y calcula p_{acep} con la misma función que usa el protocolo; imprime una fila por par. La prueba `backend/tests/test_proof_layer_search.py` comprueba que los valores coinciden con los de la tabla.

Tabla 7: Búsqueda de (k', η') para la clave de identidad ($q' = 8\,380\,417$, $\kappa = 23$, $\gamma_1 = 2^{17}$, $\gamma_2 = 95\,232$).

k'	η'	**Clave (Core-SVP clásico)**	**Aceptación por intento**	**Intentos medios**
3	2	$2^{78,8}$	0.525	1.9
4	2	$2^{114,9}$	0.423	2.4
4	4	$2^{122,6}$	0.180	5.6
4	5	$2^{126,1}$	0.117	8.5
4	**6**	$\mathbf{2^{128,2}}$	**0.077**	**13.1**
4	7	$2^{129,9}$	0.050	20.0
4	8	$2^{131,4}$	0.033	30.7
5	2	$2^{152,4}$	0.341	2.9

Con rango 5 y $\eta' = 2$ la clave quedaría unos 24 bits por encima del resto de candidatos, lo que rompería la comparabilidad del benchmark. También se descartó usar los parámetros de redondeo de ML-DSA-65 ($\gamma_1 = 2^{19}$, $\gamma_2 = (q'-1)/32$): el probador aceptaría cerca del 44 % de los intentos, pero la solidez caería a unos 2^{115}.

3.5. Configuración del Sistema y Enrolamiento

Configuración El servidor ejecuta una única vez:

$$\rho \xleftarrow{\$} \{0,1\}^{256}, \qquad (vpk, vsk) \leftarrow \mathsf{ML\text{-}KEM\text{-}768.KeyGen}(). \tag{43}$$

Las matrices públicas se expanden de forma determinista desde ρ con el XOF SHAKE128 (Sección 2.2.1):

$$\mathbf{A}' = \mathsf{ExpandA}_{q'}\big(\mathsf{H}(\rho)\big) \in \mathcal{R}_{q'}^{k' \times k'}, \qquad \mathbf{A} = \mathsf{ExpandA}_{q}(\rho) \in \mathcal{R}_{q}^{k \times k}, \tag{44}$$

donde $\mathsf{ExpandA}$ lee, para cada entrada (i, j), el flujo $\mathsf{SHAKE128}$ de la semilla seguida de i y j, lo interpreta en palabras de 23 bits (12 bits para q) y descarta las palabras mayores o iguales que el módulo, de modo que los coeficientes son exactamente uniformes. La semilla de $\mathbf{A}'$ se deriva de ρ con una etiqueta propia para que ambas matrices provengan de flujos independientes. Los parámetros públicos del sistema son (ρ, vpk); las matrices no se transmiten ni se almacenan, porque cualquiera puede regenerarlas.

Enrolamiento Cada cliente genera su clave de identidad sobre $\mathcal{R}_{q'}$:

$$\mathbf{s}, \mathbf{e} \leftarrow \mathrm{CBD}(\eta')^{k'}, \qquad \mathbf{t} = \mathbf{A}'\mathbf{s} + \mathbf{e} \bmod q', \qquad \mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t}). \tag{45}$$

La clave pública es $\mathbf{t} \in \mathcal{R}_{q'}^{k'}$ y la privada es $(\mathbf{s}, \mathbf{e}, \mathsf{id}_{\mathcal{C}})$. El servidor registra el par $(\mathsf{id}_{\mathcal{C}}, \mathbf{t})$. El cliente necesita $\mathbf{e}$, y no solo $\mathbf{s}$, porque la segunda condición de rechazo de la prueba usa $c\,\mathbf{e}$ (Sección 3.7). Que $\mathbf{t}$ no revele $\mathbf{s}$ es exactamente el supuesto de Module-LWE.

Por Qué Siempre Se Usa el Mismo e El par $(\mathbf{s}, \mathbf{e})$ se muestrea una sola vez, en el enrolamiento, y forma la clave de largo plazo del cliente: $\mathbf{t} = \mathbf{A}'\mathbf{s} + \mathbf{e}$ es la clave pública que el servidor registró, y cambiar $\mathbf{e}$ cambiaría $\mathbf{t}$, con lo que las pruebas dejarían de verificar frente a la clave registrada. Un error nuevo en cada sesión exigiría registrar una clave nueva cada vez. Lo que cambia en cada prueba, y en cada intento dentro de una prueba, es la máscara $\mathbf{y}$ y, con ella, el compromiso y el desafío c.

El error $\mathbf{e}$ tampoco interviene en lo que se envía: no aparece en la respuesta $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}$ ni viaja en el token. El cliente lo usa solo para calcular $\mathbf{w} - c\,\mathbf{e}$ en la segunda condición de rechazo, es decir, para prever el valor que reconstruirá el verificador, $\mathbf{A}'\mathbf{z} - c\,\mathbf{t} = \mathbf{w} - c\,\mathbf{e}$, y descartar los intentos en los que sus bits altos cambiarían (Sección 3.9). Reutilizar $\mathbf{e}$ no filtra información: los intentos rechazados no se publican, y la decisión de aceptar uno depende solo de $\mathbf{z}$ y de $\mathbf{A}'\mathbf{z} - c\,\mathbf{t}$, que cualquiera puede calcular a partir de datos públicos; por eso el simulador de la Sección 3.11 reproduce las pruebas sin conocer $\mathbf{e}$.

Por Qué el Cliente No Almacena $\mathbf{t}$ El cliente solo necesita $\mathbf{t}$ dentro del hash del desafío, y allí $\mathbf{t}$ entra a través de su resumen $\mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t})$ (Sección 3.7, paso 3). Guardar $\mathsf{id}_{\mathcal{C}}$, de 32 bytes, en lugar de $\mathbf{t}$, de 2 944 bytes, ahorra memoria en el dispositivo, y si alguna vez hiciera falta $\mathbf{t}$ podría recalcularse como $\mathbf{A}'\mathbf{s} + \mathbf{e}$. Por la misma razón el cliente no guarda $\mathbf{A}'$, de $k'^2 d = 4\,096$ coeficientes de 23 bits (11 776 bytes), sino que la regenera desde ρ cada vez que la necesita. Quien sí almacena $\mathbf{t}$ es el servidor, que lo necesita para verificar: calcula $\mathbf{A}'\mathbf{z} - c\,\mathbf{t}$ y comprueba que $\mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t})$ (Sección 3.9).

3.6. Flujo de Mensajes

Una autenticación consta de los mensajes de la Tabla 8. Todos pasan por la terminal $\mathcal{T}$, que se limita a retransmitirlos.

Tabla 8: Mensajes de una autenticación con acuerdo de clave.

Paso	Sentido	Contenido	Tamaño
1	$\mathcal{C} \to \mathcal{S}$	Hello: solicitud de autenticación, sin identificador del cliente.	—
2	$\mathcal{S} \to \mathcal{C}$	Reto (tk, ν).	40 bytes
3	$\mathcal{C} \to \mathcal{S}$	Token $\mathrm{tok} = (\mathrm{tok}_1, \mathrm{tok}_2)$ con la prueba, el reto, $\mathsf{id}_{\mathcal{C}}$ y, si se pide clave de sesión, la cuota $\mathbf{b}_{\mathcal{C}}$.	3 524 bytes (4 676 con $\mathbf{b}_{\mathcal{C}}$)
4	$\mathcal{S} \to \mathcal{C}$	Resultado y, si se pidió clave de sesión, la respuesta cifrada con $(\mathbf{b}_{\mathcal{S}}, \mathbf{v})$.	1 212 bytes

1. **Hello.** El cliente pide iniciar una autenticación. El mensaje no lleva identificador: así la terminal no sabe quién se autentica.

2. **Reto.** El servidor genera tk con un generador aleatorio criptográfico, toma ν de su reloj y envía (tk, ν). Solo aceptará después un token que responda a un reto que él emitió.

3. **Prueba y token.** El cliente calcula la prueba sobre (tk, ν) (Sección 3.7) y la oculta en el token (Sección 3.8). Si quiere acordar una clave de sesión, genera además su cuota efímera $\mathbf{b}_{\mathcal{C}}$ y la incluye dentro del token.

4. **Verificación y respuesta.** El servidor descifra, verifica (Sección 3.9) y comunica el resultado. Si la autenticación fue aceptada y el token traía $\mathbf{b}_{\mathcal{C}}$, responde con su cuota $\mathbf{b}_{\mathcal{S}}$ y la pista de reconciliación $\mathbf{v}$, cifradas con una clave derivada de la del token, y ambos obtienen K_{ses} (Sección 3.10).

3.7. Generación de la Prueba

El cliente demuestra que conoce $(\mathbf{s}, \mathbf{e})$ tales que $\mathbf{t} = \mathbf{A}'\mathbf{s} + \mathbf{e}$, con la plantilla de Dilithium sin compresión de la clave pública (Figura 1 de su especificación) [10, 15], en la que el reto (tk, ν) ocupa el lugar del mensaje firmado. Cada intento consta de los pasos siguientes; si un intento se rechaza, el cliente lo descarta por completo y empieza otro con una máscara nueva.

Paso 1: Máscara Uniforme en una Caja El cliente toma

$$\mathbf{y} \xleftarrow{\$} S^{k'}_{\gamma_1 - 1}, \tag{46}$$

es decir, cada uno de los $k'd = 1024$ coeficientes de $\mathbf{y}$ se elige de forma independiente y uniforme en $\{-(\gamma_1 - 1), \ldots, \gamma_1 - 1\}$: $2\gamma_1 - 1 = 262\,143$ valores equiprobables, con media 0 y desviación estándar $\sqrt{\gamma_1(\gamma_1 - 1)/3} \approx 75\,674$. Esta distribución, $S_{\gamma_1 - 1}$ en la notación de la Sección 2.1, no es Gaussiana ni binomial: es plana sobre una "caja". Se elige así porque es la que hace que la respuesta aceptada sea exactamente uniforme e independiente del secreto, con una sola comparación de normas, como muestra el lema siguiente.

El problema. La respuesta será $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}$. Cada coeficiente de $\mathbf{z}$ es un coeficiente de la máscara, uniforme en la caja, desplazado por el coeficiente correspondiente ς de $c\,\mathbf{s}$, que depende del secreto y cumple $|\varsigma| \le \beta$. El conjunto de valores posibles de z es, por tanto, la caja corrida ς posiciones, y los valores cercanos a sus bordes delatan hacia dónde se corrió. La solución es publicar z solo cuando cae en la zona central, que la caja cubre sea cual sea el desplazamiento.

Un ejemplo pequeño. Tómense $\gamma_1 = 5$ y $\beta = 2$: la máscara y es uniforme en $\{-4, \ldots, 4\}$, 9 valores con probabilidad $1/9$ cada uno, y solo se publica z si $|z| < \gamma_1 - \beta = 3$. La Tabla 9 compara dos secretos distintos, con $\varsigma = 2$ y $\varsigma = -1$.

Tabla 9: Distribución de $z = y + \varsigma$ con y uniforme en $\{-4, \ldots, 4\}$ para dos desplazamientos.

z	-5	-4	-3	-2	-1	0	1	2	3	4	5	6
$\Pr[z]$ con $\varsigma = 2$	0	0	0	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\Pr[z]$ con $\varsigma = -1$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$	0	0	0
¿Se publica?	no	no	no	sí	sí	sí	sí	sí	no	no	no	no

Sin rechazo, ver $z = 6$ revelaría que $\varsigma = 2$, porque con $\varsigma = -1$ ese valor es imposible, y ver $z = -5$ revelaría que $\varsigma = -1$. En cambio, las columnas de -2 a 2 tienen la misma probabilidad, 1/9, en las dos filas, y lo mismo ocurre para cualquier ς entre -2 y 2: es la zona que la caja cubre en todas sus posiciones posibles. El probador publica solo si z cae en ella y, si no, repite con otra máscara. Un intento se publica con probabilidad 5/9 en ambos casos, y, dado que se publica, cada uno de los 5 valores aparece con probabilidad $(1/9)/(5/9) = 1/5$: el valor publicado no dice nada de ς. Es como recortar los bordes de una foto movida: lo que queda no muestra hacia dónde se movió la cámara. Con los parámetros reales ocurre lo mismo a mayor escala: cada coeficiente de la máscara toma $2\gamma_1 - 1 = 262\,143$ valores, de los que $2(\gamma_1 - \beta) - 1 = 261\,867$ caen en la zona central.

El lema siguiente enuncia esta idea en general.

Lema (rechazo con máscara uniforme). Sea $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}$ con $\|c\,\mathbf{s}\|_\infty \le \beta$ y $\mathbf{y} \xleftarrow{\$} S^{k'}_{\gamma_1 - 1}$. Condicionado a $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, el vector $\mathbf{z}$ es uniforme en $S^{k'}_{\gamma_1 - \beta - 1}$, sea cual sea $c\,\mathbf{s}$:

$$\Pr\left[\mathbf{z} = \mathbf{z}_0 \mid \|\mathbf{z}\|_\infty < \gamma_1 - \beta\right] = \frac{1}{(2(\gamma_1 - \beta) - 1)^{k'd}} \qquad \text{para todo } \mathbf{z}_0 \in S^{k'}_{\gamma_1 - \beta - 1}. \tag{47}$$

Demostración. Coeficiente a coeficiente, como en el ejemplo: sea ς el coeficiente correspondiente de $c\,\mathbf{s}$, con $|\varsigma| \le \beta$. Para cada z_0 con $|z_0| < \gamma_1 - \beta$ existe exactamente un valor de la máscara que lo produce, $y = z_0 - \varsigma$, y ese valor está dentro de la caja porque $|z_0 - \varsigma| \le \gamma_1 - \beta - 1 + \beta = \gamma_1 - 1$. Así, cada z_0 del conjunto aceptado tiene probabilidad $1/(2\gamma_1 - 1)$, independiente de ς; como hay $2(\gamma_1 - \beta) - 1$ valores aceptados y los coeficientes son independientes, se obtiene la fórmula del lema, lo que termina la demostración.

En palabras: la condición $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$ recorta, en cada coeficiente, una franja de anchura β en cada borde de la caja, justo lo que el secreto puede desplazarla; lo que queda tiene la misma distribución con cualquier secreto. En consecuencia, la condición sobre la respuesta se cumple con probabilidad $p_z = \left(\frac{2(\gamma_1 - \beta) - 1}{2\gamma_1 - 1}\right)^{k'd} \approx \mathrm{e}^{-k'd\beta/\gamma_1}$, que tampoco depende de la clave. Por eso la máscara debe ser mucho más ancha que β: cada coeficiente se acepta con probabilidad $261\,867/262\,143 \approx 0{,}99895$, pero hay $k'd = 1024$ coeficientes, y $(261\,867/262\,143)^{1024} \approx 0{,}340$. Con una máscara más estrecha, la franja recortada sería una

fracción mayor de la caja y casi ningún intento se publicaría.

Paso 2: Compromiso y Bits Altos El cliente calcula el compromiso y sus bits altos respecto del paso $\alpha = 2\gamma_2$ (Sección 2.2.2):

$$\mathbf{w} = \mathbf{A}'\mathbf{y} \bmod q', \qquad \mathbf{w}_1 = \mathsf{HighBits}_{q'}(\mathbf{w}, 2\gamma_2) \in \{0, \ldots, 43\}^{k'd}. \tag{48}$$

Paso 3: Desafío Con la transformación de Fiat–Shamir, el desafío depende de la matriz, de la clave pública, de los bits altos del compromiso y del reto:

$$\tilde{c} = \mathsf{H}\big(\mathbf{A}', \mathbf{t}, \mathbf{w}_1, \mathsf{tk}, \nu\big) \in \{0,1\}^{256}, \qquad c = \mathsf{SampleInBall}(\tilde{c}) \in B_{23}. \tag{49}$$

H es SHA3-256 sobre la codificación inequívoca de sus argumentos, precedida de una etiqueta de dominio y de un resumen de los parámetros; $\mathbf{A}'$ y $\mathbf{t}$ entran a través de sus resúmenes de 32 bytes, $\mathsf{H}(\mathbf{A}')$ y $\mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t})$, lo que en el modelo de oráculo aleatorio es equivalente y permite que el cliente no almacene $\mathbf{t}$. $\mathsf{SampleInBall}$ expande $\tilde{c}$ con SHAKE256 en un polinomio con exactamente 23 coeficientes iguales a ± 1 (Sección 2.2.2).

Paso 4: Respuesta y Doble Rechazo La respuesta se calcula sobre los enteros:

$$\mathbf{z} = \mathbf{y} + c\,\mathbf{s}. \tag{50}$$

El intento se publica solo si cumple las dos condiciones

$$\|\mathbf{z}\|_\infty < \gamma_1 - \beta = 130\,934 \qquad \text{y} \qquad \big\|\mathsf{LowBits}_{q'}(\mathbf{w} - c\,\mathbf{e}, 2\gamma_2)\big\|_\infty < \gamma_2 - \beta = 95\,094. \tag{51}$$

La primera es la del lema anterior y garantiza que $\mathbf{z}$ no revela $\mathbf{s}$. La segunda garantiza que el verificador, que reconstruirá $\mathbf{w} - c\,\mathbf{e}$ en lugar de $\mathbf{w}$, obtenga los mismos bits altos (Sección 3.9). Si los bits bajos de $\mathbf{w} - c\,\mathbf{e}$ se tratan como uniformes, la segunda condición se cumple con probabilidad $p_{\text{low}} = \left(\frac{2(\gamma_2 - \beta) - 1}{2\gamma_2}\right)^{k'd} \approx 0{,}225$, y un intento se publica con probabilidad

$$p_{\text{acep}} = p_z\, p_{\text{low}} \approx 0{,}340 \times 0{,}225 \approx 0{,}077. \tag{52}$$

El número de intentos sigue una distribución geométrica de media $1/p_{\text{acep}} \approx 13{,}1$. La implementación devuelve un error si no obtiene una prueba en 1 000 intentos, lo que ocurre con probabilidad $(1 - p_{\text{acep}})^{1000} \approx 2{,}4 \cdot 10^{-35}$. La prueba es el par

$$\pi = (\mathbf{z}, \tilde{c}). \tag{53}$$

Por Qué los Coeficientes del Desafío Valen ± 1 y Por Qué Son 23 Con coeficientes en $\{-1, 0, 1\}$, cada coeficiente de $c\,\mathbf{s}$ es una suma con signo de 23 coeficientes de $\mathbf{s}$, de modo que $\|c\,\mathbf{s}\|_\infty \leq \kappa\eta' = 138 = \beta$, y lo mismo vale para $c\,\mathbf{e}$; esa cota es la que ocultan las dos condiciones de rechazo. Con coeficientes mayores, β crecería y p_{acep} caería. El número de coeficientes no nulos fija el tamaño del espacio de desafíos: $|B_{22}| \approx 2^{126,7}$ no alcanza 2^{128} y $|B_{23}| \approx 2^{131,1}$ sí, y no conviene más porque cada coeficiente adicional aumenta β (Sección 2.2.2).

Por Qué No Se Transmite el Compromiso El token contiene $(\mathbf{z}, \tilde{c})$, no $\mathbf{w}$, y no es una decisión de tamaño sino de seguridad. Si el verificador recibiera $\mathbf{w}$, podría calcular

$$\mathbf{w} - (\mathbf{A}'\mathbf{z} - c\,\mathbf{t}) = \mathbf{w} - (\mathbf{w} - c\,\mathbf{e}) = c\,\mathbf{e}. \tag{54}$$

Como c es invertible en $\mathcal{R}_{q'}$ con alta probabilidad, obtendría $\mathbf{e} = c^{-1}(c\,\mathbf{e})$ y luego $\mathbf{s}$ a partir de $\mathbf{A}'\mathbf{s} = \mathbf{t} - \mathbf{e}$, porque $\mathbf{A}'$ es cuadrada e invertible con alta probabilidad: un solo token entregaría la clave completa. En la forma comprimida, el compromiso solo interviene a través de sus bits altos, dentro del hash, y el verificador los reconstruye sin conocer $\mathbf{w}$.

3.8. Ocultación del Token

El cliente oculta la prueba con el paradigma KEM-DEM de la Sección 2.2.5:

$$(K_{\text{tok}}, \text{tok}_1) \leftarrow \mathsf{ML\text{-}KEM\text{-}768.Encaps}(vpk), \tag{55}$$

$$\text{tok}_2 = \mathsf{AES\text{-}256\text{-}GCM.Enc}_{K_{\text{tok}}}\big(\mathbf{z}, \tilde{c}, \mathsf{tk}, \nu, \mathsf{id}_{\mathcal{C}}\,[\,, \mathbf{b}_{\mathcal{C}}\,]\big), \tag{56}$$

$$\text{tok} = (\text{tok}_1, \text{tok}_2). \tag{57}$$

tok_2 lleva un *nonce* aleatorio de 96 bits y, como datos asociados, un resumen de los parámetros del sistema, de modo que un token no se acepta bajo otra configuración. La cuota $\mathbf{b}_{\mathcal{C}}$ solo se incluye si el cliente pide una clave de sesión (Sección 3.10); al viajar dentro de tok_2 queda cifrada y autenticada junto con la prueba.

Con los parámetros de la Tabla 6, tok_1 mide 1 088 bytes y tok_2 contiene $\mathbf{z}$ (2 304 bytes, a 18 bits por coeficiente), $\tilde{c}$, tk e $\mathsf{id}_{\mathcal{C}}$ (32 bytes cada uno), ν (8 bytes), el *nonce* (12 bytes) y la etiqueta (16 bytes): en total, 3 524 bytes, o 4 676 con los 1 152 bytes de $\mathbf{b}_{\mathcal{C}}$. Todos los tokens del mismo tipo miden lo mismo. La terminal no puede descifrar K_{tok} ni leer o modificar el contenido sin que falle la etiqueta, pero el token no es indistinguible de una cadena aleatoria: revela su longitud y, con ella, si lleva cuota de acuerdo de clave.

3.9. Verificación en el Servidor

Al recibir un token, el servidor ejecuta, en este orden:

1. **Descifrado.** $K_{\text{tok}} = \mathsf{ML\text{-}KEM\text{-}768.Decaps}(vsk, \text{tok}_1)$ y descifrado de tok_2. Si la etiqueta de GCM no es válida, o si el contenido no es una respuesta bien formada, rechaza.

2. **Identidad y reto.** Busca $\mathbf{t}$ por $\mathsf{id}_{\mathcal{C}}$ y comprueba que (tk, ν) sea un reto que él emitió. En el prototipo, quien invoca la verificación entrega $\mathbf{t}$ y el reto emitido, y el verificador exige que $\mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t})$ y que el reto coincida con el que trae el token. Un fallo en este paso rechaza el token sin consumir el ticket.

3. **Ventana temporal.** Rechaza si $|\nu_{\text{actual}} - \nu| > \Delta_\nu$.

4. **Registro de consumo.** Si $\mathsf{tk} \in \mathcal{L}_{\text{spent}}$, rechaza; si no, añade tk a $\mathcal{L}_{\text{spent}}$ de forma atómica. El ticket se consume antes de comprobar la prueba, de modo que un token inválido también lo gasta. Los tickets anteriores a la ventana pueden borrarse del registro, porque la propia ventana ya los rechaza; por eso el registro tiene un tamaño acotado (ver el párrafo siguiente).

5. **Cota de la respuesta.** Rechaza si $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$.

6. **Bits altos y hash.** Calcula $c = \mathsf{SampleInBall}(\tilde{c})$, reconstruye

$$\mathbf{w}_1' = \mathsf{HighBits}_{q'}\left(\mathbf{A}'\mathbf{z} - c\,\mathbf{t} \bmod q',\, 2\gamma_2\right) \tag{58}$$

   y acepta si y solo si $\mathsf{H}(\mathbf{A}', \mathbf{t}, \mathbf{w}_1', \mathsf{tk}, \nu) = \tilde{c}$.

Tamaño del Registro de Tickets Consumidos El registro $\mathcal{L}_{\text{spent}}$ no crece con el historial del sistema. Un ticket solo debe recordarse mientras su marca de tiempo esté dentro de la ventana: después, el paso 3 ya rechaza cualquier token que lo use, y el ticket puede borrarse. En cada momento, el registro contiene a lo sumo los tickets emitidos en los últimos Δ_ν segundos. Si hay N clientes y cada uno hace en promedio f intentos de autenticación por hora (los intentos con una prueba inválida también cuentan, porque también consumen su ticket), el número de entradas es del orden de

$$E \approx \frac{N \cdot f \cdot \Delta_\nu}{3600} = \frac{N \cdot f}{12} \qquad \text{con } \Delta_\nu = 300 \text{ s}. \tag{59}$$

Cada entrada guarda el ticket (32 bytes) y su marca de tiempo (8 bytes): 40 bytes de datos. El prototipo guarda el registro en un diccionario de Python y borra los tickets vencidos cada vez

que el registro alcanza 8 192 entradas; con la estructura incluida, cada entrada ocupa entre 130 y 155 bytes, según una medición con `tracemalloc` en CPython 3.13 sobre registros de 10^4 a 10^6 entradas. La Tabla 10 aplica la fórmula a escenarios con números realistas, con unos 150 bytes por entrada para el prototipo; como este solo borra al llegar a 8 192 entradas, puede guardar además hasta ese número de tickets vencidos, alrededor de 1.2 MB. En el escenario de pico, cada uno de un millón de clientes se autentica una vez dentro de los mismos cinco minutos.

Tabla 10: Tamaño del registro de tickets consumidos con $\Delta_\nu = 300$ s.

Escenario	**Clientes** N	**Intentos/hora**	**Entradas** E	**Datos (40 B)**	**Prototipo**
Pequeño	10 000	2	1 667	67 KB	0.25 MB
Mediano	1 000 000	2	166 667	6.7 MB	25 MB
Grande	10 000 000	2	1 666 667	67 MB	250 MB
Pico (todos en 5 min)	1 000 000	—	1 000 000	40 MB	150 MB

Incluso en el pico, el registro es mucho menor que la base de claves públicas que el servidor guarda de todos modos: con un millón de clientes, sus claves $\mathbf{t}$, de 2 944 bytes cada una, ocupan unos 2.9 GB. El tamaño depende de la tasa de autenticaciones y de Δ_ν, no del tiempo que lleve funcionando el sistema: reducir la ventana a la mitad reduce el registro a la mitad. Si un despliegue guarda además los tickets emitidos que aún no se usaron, para comprobar en el paso 2 que el reto es suyo, esa tabla obedece la misma cota y a lo sumo duplica el total. Un atacante que pidiera tickets en masa con mensajes Hello inflaría esa segunda tabla; es una forma de denegación de servicio, fuera del modelo de amenazas, que se mitiga limitando la emisión de tickets por conexión.

Por Qué una Prueba Honesta Siempre Verifica Sustituyendo $\mathbf{z} = \mathbf{y} + c\,\mathbf{s}$ y $\mathbf{t} = \mathbf{A}'\mathbf{s} + \mathbf{e}$:

$$\begin{aligned} \mathbf{A}'\mathbf{z} - c\,\mathbf{t} &\equiv \mathbf{A}'(\mathbf{y} + c\,\mathbf{s}) - c\,(\mathbf{A}'\mathbf{s} + \mathbf{e}) \pmod{q'} \\ &\equiv \mathbf{A}'\mathbf{y} + c\,\mathbf{A}'\mathbf{s} - c\,\mathbf{A}'\mathbf{s} - c\,\mathbf{e} \pmod{q'} \\ &\equiv \mathbf{w} - c\,\mathbf{e} \pmod{q'}, \end{aligned} \tag{60}$$

donde se usó que c es un escalar del anillo y conmuta con $\mathbf{A}'$. El verificador obtiene así el compromiso desplazado por $-c\,\mathbf{e}$, con $\|c\,\mathbf{e}\|_\infty \le \beta$. La segunda condición de rechazo de (51) es exactamente la hipótesis del lema de la Ecuación (12), aplicado a $\mathbf{r} = \mathbf{w} - c\,\mathbf{e}$ y $\boldsymbol{\delta} = c\,\mathbf{e}$:

$$\mathbf{w}_1' = \mathsf{HighBits}_{q'}(\mathbf{w} - c\,\mathbf{e}, 2\gamma_2) = \mathsf{HighBits}_{q'}(\mathbf{w}, 2\gamma_2) = \mathbf{w}_1, \tag{61}$$

de modo que el verificador calcula el hash sobre la misma entrada que el probador y obtiene $\tilde{c}$. La primera condición garantiza la cota del paso 5. Toda prueba publicada verifica: no hay error de completitud, y la única aleatoriedad está en cuántos intentos necesita el probador. Sin la segunda condición, una prueba honesta fallaría cada vez que $c\,\mathbf{e}$ cambiara algún bit alto: con los parámetros actuales, en torno al 3.5 % de las veces (estimación $1-(1-\mathbb{E}|(c\mathbf{e})_i|/(2\gamma_2))^{k'd}$, con $\mathbb{E}|(c\mathbf{e})_i| \approx 6{,}6$).

3.10. Acuerdo de Clave de Sesión

Tras una autenticación aceptada, cliente y servidor pueden acordar una clave de sesión K_{ses} de 256 bits sobre la capa de transporte, con cuotas efímeras de Module-LWE y la reconciliación de Peikert (Sección 2.3.5) [35, 37].

Cuotas Efímeras El cliente, al generar el token, y el servidor, al responder, muestrean secretos y errores nuevos de CBD(η) con $\eta = 2$:

$$\mathbf{b}_{\mathcal{C}} = \mathbf{A}\mathbf{s}_{\mathcal{C}} + \mathbf{e}_{\mathcal{C}} \bmod q, \qquad \mathbf{b}_{\mathcal{S}} = \mathbf{A}^T\mathbf{s}_{\mathcal{S}} + \mathbf{e}_{\mathcal{S}} \bmod q, \qquad \mathbf{s}_{\mathcal{C}}, \mathbf{e}_{\mathcal{C}}, \mathbf{s}_{\mathcal{S}}, \mathbf{e}_{\mathcal{S}} \leftarrow \text{CBD}(2)^k. \tag{62}$$

Las cuotas se generan nuevas en cada sesión; reutilizarlas permitiría ataques de fuga de señal sobre la reconciliación.

Valores Ruidosos de Cada Lado El cliente calcula $\theta_{\mathcal{C}}$ con la cuota del servidor y su propio secreto; el servidor calcula $\theta_{\mathcal{S}}$ con la cuota del cliente, su secreto y un error adicional $e^*_{\mathcal{S}} \leftarrow \text{CBD}(2)$, un único polinomio:

$$\theta_{\mathcal{C}} = \mathbf{b}_{\mathcal{S}}^T\mathbf{s}_{\mathcal{C}} \bmod q, \qquad \theta_{\mathcal{S}} = \mathbf{b}_{\mathcal{C}}^T\mathbf{s}_{\mathcal{S}} + e^*_{\mathcal{S}} \bmod q. \tag{63}$$

Por Qué $\theta_{\mathcal{C}}$ y $\theta_{\mathcal{S}}$ Casi Coinciden Desarrollando cada lado:

$$\theta_{\mathcal{C}} = (\mathbf{A}^T\mathbf{s}_{\mathcal{S}} + \mathbf{e}_{\mathcal{S}})^T\mathbf{s}_{\mathcal{C}} = \mathbf{s}_{\mathcal{S}}^T\mathbf{A}\,\mathbf{s}_{\mathcal{C}} + \mathbf{e}_{\mathcal{S}}^T\mathbf{s}_{\mathcal{C}}, \tag{64}$$

$$\theta_{\mathcal{S}} = (\mathbf{A}\mathbf{s}_{\mathcal{C}} + \mathbf{e}_{\mathcal{C}})^T\mathbf{s}_{\mathcal{S}} + e^*_{\mathcal{S}} = \mathbf{s}_{\mathcal{C}}^T\mathbf{A}^T\mathbf{s}_{\mathcal{S}} + \mathbf{e}_{\mathcal{C}}^T\mathbf{s}_{\mathcal{S}} + e^*_{\mathcal{S}}. \tag{65}$$

El término $\mathbf{s}_{\mathcal{C}}^T\mathbf{A}^T\mathbf{s}_{\mathcal{S}}$ es un único elemento de $\mathcal{R}_q$ y, por tanto, igual a su traspuesto, $\mathbf{s}_{\mathcal{S}}^T\mathbf{A}\,\mathbf{s}_{\mathcal{C}}$, porque $\mathcal{R}_q$ es conmutativo. Ese término, que es grande y uniforme a la vista de un observador, aparece en ambos lados y se cancela al restar:

$$\theta_{\mathcal{C}} - \theta_{\mathcal{S}} = \mathbf{e}_{\mathcal{S}}^T\mathbf{s}_{\mathcal{C}} - \mathbf{e}_{\mathcal{C}}^T\mathbf{s}_{\mathcal{S}} - e^*_{\mathcal{S}} \pmod q. \tag{66}$$

La diferencia solo contiene productos de valores pequeños. Cada coeficiente es la suma de $2kd = 1536$ productos de dos variables CBD(2) independientes, de varianza 1 cada uno, más un término CBD(2); su varianza es 1537 y su desviación estándar $\sigma_\theta \approx 39{,}2$. Calculando su distribución exacta,

$$\Pr\left[\,|(\theta_{\mathcal{C}} - \theta_{\mathcal{S}})_i| > 415\,\right] \approx 8{,}4 \cdot 10^{-26} \tag{67}$$

por coeficiente, de modo que la probabilidad de que alguno de los 256 bits de la clave difiera es a lo sumo $2{,}2 \cdot 10^{-23}$. Como referencia, el error supera 150 con probabilidad $1{,}3 \cdot 10^{-4}$ por coeficiente y 300 con probabilidad $2{,}4 \cdot 10^{-14}$.

Del Valor Ruidoso a la Clave: los Cuatro Cuadrantes El servidor extrae, coeficiente a coeficiente, el bit de clave y la pista:

$$K_{\mathcal{S}}^{(i)} = \left\lfloor \theta_{\mathcal{S}}^{(i)} \right\rceil_{q,2}, \qquad v^{(i)} = \left\langle \theta_{\mathcal{S}}^{(i)} \right\rangle_{q,2}, \qquad i = 0, \ldots, 255, \tag{68}$$

y el cliente reconcilia con su propio valor: $K_{\mathcal{C}}^{(i)} = \mathrm{rec}\big(\theta_{\mathcal{C}}^{(i)}, v^{(i)}\big)$. La pista $v^{(i)}$ indica en cuál de los dos pares de cuadrantes opuestos de la Tabla 4 está $\theta_{\mathcal{S}}^{(i)}$, y la reconciliación elige, dentro de ese par, el cuadrante más cercano a $\theta_{\mathcal{C}}^{(i)}$. Como el error es como mucho 415 salvo con la probabilidad calculada arriba, ambos obtienen el mismo bit (Sección 2.3.5):

$$K_{\mathcal{C}} = K_{\mathcal{S}} = K_{\mathrm{ses}} \in \{0,1\}^{256}. \tag{69}$$

Por ejemplo, si $\theta_{\mathcal{S}}^{(i)} = 1000$ (cuadrante Q_1), el servidor obtiene el bit 1 y envía $v^{(i)} = 1$; el cliente tiene $\theta_{\mathcal{C}}^{(i)} = 1000 + \epsilon$ con $|\epsilon| \le 415$, que no pertenece a $I_1 + E = \{-1248, \ldots, 415\}$, y obtiene también 1. Como q es impar y no se usa la duplicación aleatoria de Peikert, el bit de clave tiene un sesgo de 832/1665 frente a 1/2 cuando $v^{(i)} = 0$.

Cómo Viajan las Cuotas La cuota del cliente $\mathbf{b}_{\mathcal{C}}$ viaja dentro de tok_2, cifrada y autenticada con K_{tok} junto con la prueba. Tras aceptar el token, el servidor genera $\mathbf{b}_{\mathcal{S}}$ y $\mathbf{v}$ y los envía cifrados con AES-256-GCM bajo la clave $K_{\mathrm{resp}} = \mathsf{H}(K_{\mathrm{tok}})$, derivada con una etiqueta propia, y con el ticket como dato asociado del cifrado; el cliente, que conoce K_{tok} porque la generó al encapsular, descifra la respuesta y reconcilia. El servidor responde una sola vez por ticket aceptado.

3.11. Análisis de Seguridad

Correctitud Toda prueba honesta verifica con probabilidad 1 (Sección 3.9), y las dos partes acuerdan la misma clave de sesión salvo con probabilidad menor que $2{,}2 \cdot 10^{-23}$ (Sección 3.10).

Conocimiento Cero La prueba es una NIZK en el modelo de oráculo aleatorio programable (Sección 2.2.1). Un simulador Sim que solo conoce $(\mathbf{A}', \mathbf{t}, \mathsf{tk}, \nu)$ produce pruebas con la misma distribución que las reales:

1. elige $\tilde{c} \xleftarrow{\$} \{0,1\}^{256}$, calcula $c = \mathsf{SampleInBall}(\tilde{c})$ y toma $\mathbf{z} \xleftarrow{\$} S^{k'}_{\gamma_1-\beta-1}$;
2. repite el paso anterior si $\|\mathsf{LowBits}_{q'}(\mathbf{A}'\mathbf{z} - c\,\mathbf{t}, 2\gamma_2)\|_\infty \geq \gamma_2 - \beta$;
3. programa el oráculo para que $\mathsf{H}\big(\mathbf{A}', \mathbf{t}, \mathsf{HighBits}_{q'}(\mathbf{A}'\mathbf{z} - c\,\mathbf{t}, 2\gamma_2), \mathsf{tk}, \nu\big) = \tilde{c}$ y devuelve $(\mathbf{z}, \tilde{c})$.

Las pruebas reales tienen esa distribución: por el lema de la Sección 3.7, el $\mathbf{z}$ aceptado es uniforme en su caja sea cual sea el secreto, y la segunda condición de rechazo depende solo de $(\mathbf{z}, c, \mathbf{t})$, porque $\mathbf{w} - c\,\mathbf{e} = \mathbf{A}'\mathbf{z} - c\,\mathbf{t}$. Los intentos rechazados nunca se publican, lo que encaja con la noción de conocimiento cero "sin abortos" (naHVZK) de Kiltz, Lyubashevsky y Schaffner, en la que el simulador no necesita reproducir los compromisos de los intentos abortados [23]. Como el servidor ve exactamente $(\mathbf{z}, \tilde{c})$ tras descifrar, el argumento también lo cubre: el servidor no aprende de una prueba nada que no pudiera haber generado solo.

Tipo de conocimiento cero. Las pruebas simuladas y las reales tienen exactamente la misma distribución, salvo en un caso: que el simulador tenga que programar el oráculo en una entrada que ya había sido consultada. Como los bits altos del compromiso tienen una entropía muy alta, ese caso tiene probabilidad despreciable [15]. Por eso el conocimiento cero de la prueba es **estadístico** (Sección 2.2): no es perfecto, por ese margen despreciable, y tampoco es solo computacional, porque la indistinguibilidad no depende de que ningún problema matemático sea difícil, sino únicamente de que la función hash se comporte como un oráculo aleatorio. Lo que sí es computacional es la confidencialidad del token frente a terceros, que descansa en ML-KEM y AES-GCM (Sección 3.8).

Solidez: Relación Relajada y SelfTargetMSIS Si un probador produce dos pruebas aceptadas con los mismos bits altos y desafíos distintos $c \neq c'$, se cumple $\mathbf{A}'\mathbf{z} - c\,\mathbf{t} = \alpha\mathbf{w}_1 + \mathbf{r}_0$ y $\mathbf{A}'\mathbf{z}' - c'\,\mathbf{t} = \alpha\mathbf{w}_1 + \mathbf{r}'_0$ con bits bajos acotados por γ_2, y al restar:

$$\mathbf{A}'(\mathbf{z} - \mathbf{z}') + \mathbf{u} = (c - c')\,\mathbf{t} \pmod{q'}, \qquad \|\mathbf{z} - \mathbf{z}'\|_\infty < 2(\gamma_1 - \beta), \quad \|\mathbf{u}\|_\infty \leq 2\gamma_2, \tag{70}$$

con $\mathbf{u} = \mathbf{r}'_0 - \mathbf{r}_0$. Es un testigo corto de una relación relajada, no necesariamente $(\mathbf{s}, \mathbf{e})$: la prueba es un argumento de conocimiento para esa relación. Siguiendo el análisis de Dilithium [15, 23], la resistencia a la suplantación se reduce a dos problemas con los parámetros de la capa de prueba:

- **Module-LWE:** que $\mathbf{t} = \mathbf{A}'\mathbf{s} + \mathbf{e}$ sea indistinguible de un vector uniforme.
- **SelfTargetMSIS:** encontrar un vector de coeficientes pequeños $(\mathbf{z}, c, \mathbf{u})$ y un mensaje μ tales que $\mathsf{H}\big(\mu, [\mathbf{A}' \mid \mathbf{t} \mid \mathbf{I}]\,(\mathbf{z}, c, \mathbf{u})\big) = c$ [15].

En el modelo de oráculo aleatorio clásico, SelfTargetMSIS se reduce a MSIS con el *forking lemma*, mediante una reducción no ajustada (*non-tight*) [20, 15]. En el cuántico (QROM) esa reducción no se conoce: la seguridad se reduce a Module-LWE y SelfTargetMSIS [23], con las correcciones posteriores de la literatura [24, 25], y la dureza de SelfTargetMSIS se estima como la de MSIS sobre la matriz $[\mathbf{A}' \mid \mathbf{t} \mid \mathbf{I}]$, de $k' \times (2k'+1)$ polinomios (1024×2304 enteros), con cota $\gamma_1 - \beta = 130\,934$ en norma infinito [15]. Existe además un ataque genérico que no usa la estructura del retículo: elegir $\mathbf{z}$ corto y un desafío c, calcular los bits altos de $\mathbf{A}'\mathbf{z} - c\,\mathbf{t}$ y esperar que el hash devuelva una semilla de la que se derive exactamente c; cada intento tiene éxito con probabilidad $1/|B_{23}| \approx 2^{-131{,}1}$.

Dureza Concreta La Tabla 11 resume las estimaciones, obtenidas con el *lattice-estimator* (*commit* `53da598`) en el modelo Core-SVP (Sección 2.3.6).

Tabla 11: Seguridad estimada de ML-ZKAuth con los parámetros por defecto.

Instancia	Ataque más barato	β_{BKZ}	Clásico	Cuántico
Clave de identidad: MLWE, dimensión 1024, q', CBD(6)	primal (uSVP)	439	$2^{128{,}2}$	$2^{116{,}3}$
Ídem, ataque dual híbrido	dual híbrido	447	$2^{130{,}5}$	$2^{118{,}5}$
Solidez: MSIS sobre $[\mathbf{A}' \mid \mathbf{t} \mid \mathbf{I}]$, cota 130 934	reducción de retículo	516	$2^{150{,}7}$	$2^{136{,}7}$
Adivinar el desafío: $\lvert B_{23}\rvert$	búsqueda genérica	—	$2^{131{,}1}$	$\approx 2^{65{,}5}$ (Grover)
Capa de transporte: MLWE, dimensión 768, q, CBD(2)	dual híbrido	597	$2^{174{,}3}$	$2^{160{,}0}$

La seguridad frente a ataques de retículo es la menor de las tres primeras filas: $2^{128{,}2}$ clásico y $2^{116{,}3}$ cuántico en Core-SVP, al nivel de los demás candidatos de retículos del benchmark, que se ajustaron a la misma meta. La búsqueda genérica del desafío cuesta del orden de 2^{131} evaluaciones del hash, o unas $2^{65{,}5}$ con el algoritmo de Grover: el mismo orden que la búsqueda de Grover sobre una clave de AES-128, la referencia de la categoría 1 de NIST

(Sección 2.3.6). Los demás candidatos de retículos del benchmark tienen espacios de desafíos del mismo tamaño ($2^{129,6}$ a $2^{131,6}$); Dilithium usa espacios mayores (2^{192} en su nivel más bajo) [15]. La capa de transporte usa los parámetros de ML-KEM-768, que FIPS 203 asigna a la categoría 3 [27], y su estimación no limita la del protocolo.

Por Qué los 128 Bits No Dependen de Penalizaciones Adicionales El modelo Core-SVP cuenta una sola llamada a SVP. Un ataque real paga además el número de pasadas y ventanas de BKZ, la memoria exponencial de la criba ($2^{0,2075 \cdot 439} \approx 2^{91}$ vectores para el bloque de la clave) y, en el caso cuántico, el costo de una memoria cuántica de acceso aleatorio (Sección 2.3.6). Esas penalizaciones de memoria y de recorridos de ventana llevan, por ejemplo, a Kyber512 de 2^{118} en Core-SVP a unas $2^{151,5}$ compuertas [28], pero no se calcularon para ML-ZKAuth: la afirmación de este trabajo es la de Core-SVP, en la que la clave ya alcanza $2^{128,2}$ con $\eta' = 6$. Las penalizaciones son margen adicional, no parte del argumento, y ninguna de estas cifras certifica una categoría de NIST.

Verificador Designado, No Vinculabilidad y Transferibilidad Solo el servidor, que posee vsk, puede obtener K_{tok} y leer tok_2 (Sección 2.2.5). Como el identificador del cliente viaja dentro de tok_2, el Hello no lleva identificador, las máscaras son nuevas en cada prueba y la respuesta del servidor también va cifrada, la terminal y los observadores del canal no pueden saber qué cliente se autentica ni relacionar dos autenticaciones del mismo cliente, salvo por metadatos ajenos al protocolo (dirección de red, horario) o por la longitud del token, que solo indica si se pidió clave de sesión. Frente al servidor no hay anonimato: identifica al cliente por diseño. Una vez descifrada, la prueba $(\mathbf{z}, \tilde{c})$ es verificable por cualquiera con $\mathbf{t}$ y (tk, ν): es transferible y, por tanto, el protocolo no es denegable (Sección 2.2.4).

Repetición y Frescura Una prueba está ligada a su reto porque (tk, ν) entra en el hash: volver a cifrar una prueba válida bajo otro ticket u otra marca de tiempo produce un token que no verifica. Reenviar un token completo falla en el registro de consumo, que es común a todas las verificaciones del servidor, y un token antiguo falla en la ventana temporal. Un token con un reto que el servidor no emitió, o con el identificador de otro cliente, se rechaza sin consumir ningún ticket.

Acuerdo de Clave Ligado a la Autenticación Si la cuota del cliente viajara aparte, la terminal podría sustituirla por una suya: el servidor aceptaría la prueba legítima del cliente y acordaría la clave de sesión con la terminal. Como $\mathbf{b}_{\mathcal{C}}$ viaja dentro de tok_2, autenticada con K_{tok}, la terminal no puede sustituirla sin que falle la etiqueta, ni extraer la prueba para volver a cifrarla con otra cuota, porque no conoce K_{tok}. En sentido inverso, solo quien desencapsuló

tok_1 conoce K_{tok} y, con ella, K_{resp}, de modo que la terminal no puede responder en nombre del servidor ni alterar $(\mathbf{b}_{\mathcal{S}}, \mathbf{v})$: el servidor queda autenticado ante el cliente de forma implícita. La clave de sesión es pseudoaleatoria para un observador bajo el supuesto de que las cuotas de Module-LWE y la pista no revelan el bit reconciliado (la hipótesis PWE de la Sección 2.3.3) [35].

Limitaciones Para cada limitación se indica por qué ocurre y por qué se acepta en este trabajo o cómo se mitigaría.

- **El prototipo no es de tiempo constante.** *Por qué ocurre:* está escrito en Python con NumPy, cuyas operaciones con enteros y arreglos no garantizan un tiempo independiente de los datos, y multiplica polinomios con la convolución negacíclica directa, sin NTT. Además, el número de intentos del probador, unos 13 en promedio, se refleja en su tiempo de ejecución. *Por qué se acepta:* el prototipo sirve para medir y comparar candidatos bajo el mismo marco (Sección 3.2), y el número de intentos no filtra la clave, porque su distribución es geométrica con una probabilidad de aceptación que no depende de ella (Sección 3.7). *Mitigación:* una implementación de producción en un lenguaje compilado, con aritmética de tiempo constante y NTT, que ambos anillos admiten (Sección 2.3.3).

- **No se analizaron canales laterales ni fallos inducidos.** *Por qué ocurre:* esos ataques dependen del dispositivo físico (consumo, emisiones, inyección de fallos) y no del protocolo, y el trabajo no evalúa hardware. *Por qué se acepta:* quedan fuera del modelo de amenazas (Sección 3.1). *Mitigación:* implementación en tiempo constante, enmascaramiento de los valores secretos y elementos seguros o módulos criptográficos certificados para guardar $(\mathbf{s}, \mathbf{e})$.

- **No hay una demostración propia de seguridad en el QROM.** *Por qué ocurre:* demostrar la seguridad de una variante concreta frente a adversarios cuánticos es un trabajo de investigación en sí mismo. *Por qué se acepta:* la prueba sigue la plantilla de Dilithium, cuya seguridad en el QROM está analizada [21, 23, 24, 25], y la reducción clásica se apoya en el *forking lemma*. *Mitigación:* una demostración específica queda como trabajo futuro.

- **El estado del verificador vive en la memoria de un proceso.** *Por qué ocurre:* la clave *vsk* y el registro de tickets consumidos se guardan en el proceso que ejecutó la configuración, porque el contrato común del benchmark construye un objeto nuevo en cada llamada y no tiene una base de datos de sesiones. *Por qué se acepta:* el benchmark

ejecuta un solo proceso de verificación. *Mitigación:* un despliegue con varios procesos o servidores necesita un almacén compartido con inserción atómica de tickets, y una custodia de vsk en un módulo criptográfico; el tamaño de ese almacén está acotado (Sección 3.9).

- **La clave de sesión tiene un sesgo pequeño.** *Por qué ocurre:* la reconciliación se aplica sobre el módulo impar $q = 3329$ sin la duplicación aleatoria de Peikert, y cuando la pista vale 0 el bit vale 1 con probabilidad 832/1665 en lugar de 1/2 (Sección 2.3.5). *Por qué se acepta:* cada bit sesgado conserva al menos $\log_2(1665/833) \approx 0{,}9991$ bits de entropía mínima, de modo que, aun si los 256 bits fueran sesgados, la clave tendría más de 255.7 bits. *Mitigación:* derivar la clave de sesión con una función hash a partir de los bits reconciliados, como hacen los KEM estandarizados, eliminaría el sesgo; el prototipo entrega los bits reconciliados directamente.

- **No hay confirmación de clave.** *Por qué ocurre:* el protocolo no incluye un mensaje con el que cada parte compruebe que la otra obtuvo la misma K_{ses}. *Por qué se acepta:* un desacuerdo ocurre con probabilidad menor que $2{,}2 \cdot 10^{-23}$ (Sección 3.10) y solo hace fallar el primer mensaje cifrado, tras lo cual se repite la autenticación. *Mitigación:* añadir un código de autenticación de mensaje calculado con K_{ses}, como los mensajes de finalización de TLS, sería un cambio de protocolo.

- **No hay protección específica contra relés.** *Por qué ocurre:* un intermediario que reenvía en tiempo real los mensajes entre un cliente legítimo y el servidor completa la autenticación dentro de la ventana de 300 s, porque el protocolo no mide distancias ni tiempos de respuesta. *Por qué se acepta:* si la sesión se protege con K_{ses}, el intermediario no obtiene la clave, porque la cuota del cliente viaja dentro del token y la respuesta del servidor va cifrada con una clave que solo conoce el cliente; el relé solo afecta a usos que tratan la aceptación como una autorización sin cifrar la sesión (Sección 3.12). *Mitigación:* usar siempre la clave de sesión o añadir medición de distancia (*distance bounding*) en el canal físico.

3.12. Eventos Improbables, Comparación y Amenazas Prácticas

A lo largo del documento aparecen eventos de probabilidad despreciable: en las definiciones de completitud y solidez (Sección 2.2), en la corrección de un KEM (Sección 2.2.5), en las colisiones de tickets y de identificadores (Secciones 2.2.5 y 3.3), en el límite de intentos del probador (Sección 3.7), en el error de reconciliación (Sección 3.10) y en la búsqueda genérica del desafío (Sección 3.11). Despreciable no significa imposible: significa tan pequeño que, en

la práctica, el evento no ocurre. Para dar una escala, si mil millones de dispositivos hicieran mil millones de intentos por segundo durante toda la edad del universo, unos $4{,}3 \cdot 10^{17}$ s, harían unos $4{,}3 \cdot 10^{35}$ intentos, y un evento de probabilidad $2^{-128} \approx 2{,}9 \cdot 10^{-39}$ por intento ocurriría con probabilidad de alrededor de 10^{-3}.

Esta sección reúne esos valores en un solo lugar. Los separa según su consecuencia: los de la Tabla 12 solo hacen que una autenticación honesta falle y deba repetirse, mientras que los de la Tabla 13 darían ventaja a un atacante. Después los compara con mecanismos de autenticación en uso (Tabla 14) y resume cómo se comporta el protocolo frente a las amenazas prácticas más comunes (Tabla 15).

Eventos que No Comprometen la Seguridad Son fallos de corrección: le ocurren a un cliente honesto y su única consecuencia es que la autenticación, o el acuerdo de clave, no se completa. No dan información ni ventaja a un atacante.

Tabla 12: Eventos improbables que solo afectan la disponibilidad.

Evento	**Probabilidad**	**Qué ocurre si sucede**
Una prueba honesta es rechazada	0	Nunca ocurre: la completitud es perfecta, porque el cliente solo publica intentos que pasan las dos condiciones de rechazo (Sección 3.9).
El probador no obtiene una prueba en 1 000 intentos	$2{,}4 \cdot 10^{-35}$	El prototipo devuelve un error y no envía nada; el cliente pide un reto nuevo y repite.
Falla el desencapsulado de ML-KEM-768	$\approx 2^{-164}$ [28]	El servidor obtiene otra clave, la etiqueta de AES-GCM no verifica y el token se rechaza sin gastar el ticket; el cliente repite.
El servidor emite dos veces el mismo ticket	$\leq N^2/2^{257}$	A lo sumo, el segundo token honesto se rechaza por ticket gastado y el cliente repite. Un token antiguo no sirve, porque su marca de tiempo es otra.
Dos clientes tienen el mismo identificador	$\leq N^2/2^{257}$	Un despliegue lo detecta al registrar al segundo cliente, que genera otra clave (Sección 3.3).
El error de reconciliación supera 415	$\leq 2{,}2 \cdot 10^{-23}$ por sesión	Cliente y servidor obtienen claves de sesión distintas; lo notan en el primer mensaje cifrado que no se puede descifrar y repiten la autenticación. Las cuotas son de un solo uso, de modo que un observador no aprende nada.

Eventos que Darían Ventaja a un Atacante Son los que mide el error de solidez y los supuestos de dureza: si ocurrieran, un impostor se autenticaría como otro cliente en esa sesión o alteraría el acuerdo de clave. La Tabla 13 muestra por qué no son practicables.

Tabla 13: Eventos improbables que comprometerían la seguridad.

Evento	**Probabilidad o costo**	**Por qué no es practicable**
Enviar un token al azar y que sea aceptado	$\approx 2^{-256}$ por intento	Hay que acertar la semilla $\tilde{c}$ de 256 bits del hash, y cada intento debe enviarse al servidor y consume un ticket.
Búsqueda genérica del desafío (Sección 3.11)	$2^{-131,1}$ por evaluación del hash	El hash incluye el ticket, de modo que la búsqueda solo puede empezar al recibirlo y debe terminar dentro de la ventana de 300 s. Con 10^{12} evaluaciones por segundo, la probabilidad de éxito en esa ventana es de unos 2^{-83}; con el algoritmo de Grover harían falta unos $2^{65,5}$ pasos secuenciales.
Falsificar el contenido cifrado del token, por ejemplo para cambiar la cuota $\mathbf{b}_{\mathcal{C}}$	2^{-128} por intento	Hay que acertar la etiqueta de 128 bits de AES-GCM [29], y cada intento debe enviarse al servidor.
Romper Module-LWE o SelfTargetMSIS con los parámetros de la capa de prueba	$\geq 2^{128,2}$ operaciones (Core-SVP)	No es un evento de suerte sino de cómputo (Tabla 11). Si se lograra, rompería la clave atacada; el riesgo que hay que vigilar es un avance en el criptoanálisis de retículos.
Que la respuesta filtre el secreto por mala suerte	0	No existe ese evento: el lema de rechazo es exacto y la respuesta publicada es siempre uniforme en su caja, sea cual sea la clave (Sección 3.7). El secreto solo podría filtrarse por canales laterales.

Comparación con Mecanismos de Autenticación en Uso La Tabla 14 compara esas cifras con las de mecanismos de autenticación de uso extendido. Las probabilidades de los factores numéricos se obtienen contando los valores posibles; en la práctica se compensan limitando el número de intentos y vigilando el fraude.

Tabla 14: Probabilidad de que un atacante acierte por intento, o costo de un ataque, en mecanismos en uso y en ML-ZKAuth.

Mecanismo	**Evento**	**Probabilidad o costo**	**Resistencia cuántica**
PIN numérico de 4 dígitos	adivinarlo	10^{-4} por intento	No aplica (factor humano).
Código de un solo uso de 6 dígitos	adivinarlo	10^{-6} por intento	No aplica (factor humano).
Criptograma de 64 bits de las tarjetas de pago con chip (EMV) [42]	falsificarlo	$2^{-64} \approx 5{,}4 \cdot 10^{-20}$ por intento	Parcial: el criptograma usa cifrado simétrico, y la autenticación de datos de la tarjeta usa RSA, que el algoritmo de Shor rompe [42].
Etiqueta de 128 bits de AES-GCM, como en TLS [29]	falsificarla	$2^{-128} \approx 2{,}9 \cdot 10^{-39}$ por intento	Se mantiene: Grover no reduce la probabilidad de un intento único.
Firma ECDSA sobre P-256, como en FIDO2 [2, 43]	romper la clave	2^{128} operaciones	No: el algoritmo de Shor la rompe en tiempo polinomial.
ML-ZKAuth	adivinar el desafío	$2^{-131{,}1}$ por evaluación del hash	Grover: unas $2^{65{,}5}$ evaluaciones secuenciales.
ML-ZKAuth	aceptación de un token al azar	$\approx 2^{-256}$ por intento	Se mantiene.
ML-ZKAuth	romper la clave	$2^{128{,}2}$ clásico, $2^{116{,}3}$ cuántico (Core-SVP)	Sí, bajo los supuestos de Module-LWE y SelfTargetMSIS.

Desde el punto de vista criptográfico, ML-ZKAuth está al nivel de los mecanismos de 128 bits en uso o por encima, y a diferencia de ellos resiste a un adversario cuántico. Sus cifras no son el punto débil: en la práctica, las brechas de los sistemas de autenticación casi nunca rompen la criptografía, sino que explotan a las personas, a los dispositivos o a la operación.

Amenazas Prácticas La Tabla 15 resume cómo se comporta el protocolo frente a esas amenazas: para cada una indica si el protocolo la cubre y por qué, o cómo se mitigaría.

Tabla 15: Amenazas prácticas y cobertura de ML-ZKAuth.

Amenaza	**Cobertura**	**Razón o mitigación**
Suplantación de identidad (*phishing*)	Parcial	No hay una contraseña ni un código que el usuario teclee y pueda entregar a un sitio falso, y el token solo puede leerlo el servidor que posee vsk. Si el atacante roba el dispositivo con $(\mathbf{s}, \mathbf{e})$, puede suplantar al cliente: el protocolo no incluye un segundo factor local, como un PIN o la biometría, que desbloquee la clave.
Terminal comprometida y captura de datos (*skimming*)	Sí	Es el caso de diseño: la terminal es un canal ciego que solo ve tokens cifrados de longitud fija y de un solo uso, sin el identificador del cliente (Sección 3.8).
Repetición de mensajes (*replay*)	Sí	Ticket de un solo uso, ventana de 300 s y registro de tickets consumidos (Sección 3.9).
Relé en tiempo real	Parcial	El intermediario completa la autenticación, pero no obtiene la clave de sesión; el riesgo se limita a usos que no cifran la sesión con K_{ses} (Sección 3.11).
Robo de la base de datos del servidor	Sí, salvo vsk	El servidor solo guarda claves públicas $\mathbf{t}$, con las que no se puede suplantar a un cliente. Quien robe vsk puede leer tokens y suplantar al servidor ante los clientes, no a los clientes: vsk debe custodiarse en un módulo criptográfico.
Malware o robo del dispositivo del cliente	No	Quien obtiene $(\mathbf{s}, \mathbf{e})$ suplanta al cliente; se mitiga con almacenamiento seguro de la clave en el dispositivo, fuera del protocolo.
Canales laterales y fallos inducidos	No	El prototipo no es de tiempo constante (Sección 3.11).
Adversario con computador cuántico	Sí	La seguridad descansa en problemas de retículos (Tabla 11); los mecanismos basados en RSA o curvas elípticas no la tienen.
Denegación de servicio	Fuera del modelo	Un atacante puede pedir tickets en masa; se mitiga limitando la emisión de tickets por conexión (Sección 3.9).

3.13. Correspondencia con la Implementación

El protocolo está implementado en el módulo `crypto_core/protocols/proposed_lwe.py` del repositorio, con el cifrado del token en `crypto_core/token_hiding.py` y las primitivas de retículo (muestreadores, codificación, aritmética de módulos, descomposición y reconciliación) en `crypto_core/lwe/`. Implementa el contrato común con el que el benchmark evalúa a los seis candidatos, más tres operaciones propias del acuerdo de clave. La Tabla 16 relaciona cada paso con la función que lo ejecuta.

Tabla 16: Pasos del protocolo y funciones del prototipo.

Función	**Paso del protocolo**
`generate_system_parameters`	Configuración: ρ y el par ML-KEM-768 (vpk, vsk); publica (ρ, vpk) y guarda vsk en el almacén del proceso, indexado por $\mathsf{H}(vpk)$.
`generate_keypair`	Enrolamiento: $\mathbf{s}, \mathbf{e} \leftarrow \mathrm{CBD}(\eta')^{k'}$, $\mathbf{t} = \mathbf{A}'\mathbf{s} + \mathbf{e}$ y $\mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t})$.
`generate_challenge`	Respuesta del servidor al Hello: reto (tk, ν).
`solve_challenge`	Prueba (Sección 3.7) y token sin cuota de acuerdo de clave. Es la operación que mide el benchmark.
`verify_response`	Verificación completa (Sección 3.9); devuelve aceptación o rechazo.
`solve_challenge_with_key_agreement`	Prueba y token con la cuota $\mathbf{b}_{\mathcal{C}}$; devuelve además el estado secreto del cliente $(\mathbf{s}_{\mathcal{C}}, K_{\mathrm{resp}})$.
`generate_session_reply`	Respuesta cifrada del servidor con $(\mathbf{b}_{\mathcal{S}}, \mathbf{v})$ y su mitad de la clave, una sola vez por ticket aceptado.
`reconcile_session_key`	Descifrado de la respuesta y reconciliación del cliente.

Los objetos se codifican empaquetando los coeficientes al número mínimo de bits: 23 bits para $\mathbf{t}$ (2 944 bytes), 18 bits para $\mathbf{z}$ desplazado por γ_1, 4 bits para $\mathbf{s}$ y $\mathbf{e}$ desplazados por η' (512 bytes cada uno) y 12 bits para las cuotas de la capa de transporte (1 152 bytes). Toda la aleatoriedad proviene de generadores criptográficos: el del sistema operativo para los muestreos, los tickets y los *nonces*, y el de la biblioteca criptográfica para ML-KEM. El formato de expansión de las matrices es propio del prototipo, y $\mathsf{SampleInBall}$ elige las posiciones leyendo palabras de 16 bits con rechazo (FIPS 204 lee bytes): la distribución es la misma, pero la misma semilla no produce el mismo desafío que ML-DSA. El rango k', el ruido η', κ, γ_1, γ_2 y Δ_ν son ajustables; la implementación rechaza cualquier combinación con

un espacio de desafíos menor que 2^{128}, con $2\gamma_2$ que no divida a $q' - 1$ o con una probabilidad de aceptación por intento menor que 0,05, y fija los demás parámetros.

4. Evaluación experimental y benchmarking de protocolos de autenticación

Este capítulo presenta la metodología experimental utilizada para evaluar y comparar las diferentes alternativas criptográficas consideradas en este trabajo. El objetivo no consiste únicamente en determinar cuál implementación presenta el menor tiempo de ejecución, sino en estudiar los compromisos existentes entre rendimiento, consumo de recursos, comunicación, seguridad y requisitos operacionales.

4.1. Objetivos de la evaluación

La evaluación experimental persigue los siguientes objetivos:

1. Comparar el costo computacional de los diferentes protocolos durante las principales etapas de autenticación.
2. Comparar el consumo de memoria y almacenamiento requerido por cada alternativa.
3. Determinar el volumen de información que debe transmitirse durante una autenticación y proyectar su efecto sobre el tiempo de autenticación en distintas condiciones de red.
4. Analizar las condiciones de seguridad asociadas con los parámetros utilizados por cada protocolo.
5. Examinar los principales ataques relevantes para cada familia criptográfica y las condiciones necesarias para mitigarlos.
6. Comparar los requisitos de configuración y despliegue de cada alternativa.
7. Analizar las dependencias de confianza y los requisitos de infraestructura de cada protocolo.
8. Evaluar el impacto que tendría cada alternativa sobre la experiencia de un usuario final.
9. Identificar los principales compromisos entre seguridad, rendimiento, comunicación, memoria y facilidad de despliegue.
10. Comparar el protocolo propuesto en este trabajo con las demás alternativas bajo las mismas condiciones experimentales.

4.2. Principio de comparabilidad

Para garantizar una comparación significativa, todos los candidatos implementan una misma funcionalidad abstracta de autenticación.

El flujo experimental se divide conceptualmente en dos momentos: registro (*enrollment*) y autenticación.

4.2.1. Registro

Durante el registro se ejecutan las operaciones necesarias para preparar al usuario y al verificador antes de realizar una autenticación.

De manera abstracta, esta etapa se representa como:

$$(pk, sk, \theta) \leftarrow \text{Setup}() \tag{71}$$

donde:

- pk representa la información pública necesaria para verificar al usuario;
- sk representa el material secreto conservado por el usuario o demostrador;
- θ representa parámetros adicionales requeridos por el protocolo.

Dependiendo de la construcción, esta operación puede incluir generación de claves, muestreo de vectores, construcción de matrices, generación de parámetros públicos o precomputaciones.

4.2.2. Autenticación

Una autenticación se modela mediante las siguientes operaciones:

$$c \leftarrow \text{Challenge}(pk), \tag{72}$$
$$r \leftarrow \text{Respond}(sk, c), \tag{73}$$
$$b \leftarrow \text{Verify}(pk, c, r), \tag{74}$$

donde:

- c es el reto generado por el verificador;
- r es la respuesta generada por el usuario;

- $b \in \{0, 1\}$ representa el resultado de la verificación.

Para una ejecución válida se espera que:

$$\text{Verify}(pk, c, \text{Respond}(sk, c)) = 1. \tag{75}$$

Esta abstracción permite medir las diferentes construcciones utilizando las mismas etapas funcionales aun cuando las operaciones matemáticas internas sean diferentes.

4.2.3. Separación entre registro y autenticación

El tiempo de generación de claves no se incorpora automáticamente al tiempo de cada autenticación, puesto que normalmente corresponde a una operación realizada durante el registro inicial.

Por lo tanto, se reportan por separado:

$$T_{\text{enrollment}} \tag{76}$$

y

$$T_{\text{authentication}} = T_{\text{challenge}} + T_{\text{response}} + T_{\text{verification}}. \tag{77}$$

Cuando una implementación requiera operaciones adicionales en alguna de estas etapas, estas deberán documentarse y contabilizarse explícitamente.

4.3. Protocolos evaluados

La Tabla 17 resume las alternativas estudiadas.

Tabla 17: Protocolos incluidos en la evaluación experimental.

Protocolo	**Fundamento matemático**	**Tipo de construcción**	**Rol en la comparación**
ECDSA	Problema del logaritmo discreto sobre curvas elípticas	Criptografía tradicional	Referencia de industria
LWE estándar	Learning With Errors	Retículos	Candidato poscuántico
Binary-LWE	Learning With Errors con secreto restringido	Retículos	Variante LWE
Ring-LWE	Learning With Errors sobre estructuras algebraicas de anillos	Retículos estructurados	Variante estructurada
LWR	Learning With Rounding	Retículos	Variante basada en redondeo
ML-ZKAuth (propuesto)	Module-LWE y SelfTargetMSIS (prueba); ML-KEM-768 y AES-256-GCM (token)	Retículos estructurados (módulos)	Propuesta de este trabajo

ECDSA se utiliza como referencia de una alternativa ampliamente adoptada en sistemas criptográficos tradicionales. Las construcciones basadas en LWE y problemas relacionados permiten estudiar alternativas fundamentadas en retículos y analizar sus propiedades frente a los requisitos considerados en este proyecto.

La comparación se realiza utilizando un mismo modelo funcional de autenticación y un entorno experimental controlado. De esta manera, las diferencias reportadas buscan reflejar las características de cada alternativa criptográfica y no diferencias artificiales introducidas por procedimientos de medición distintos.

4.4. Entorno experimental

Todos los experimentos fueron ejecutados bajo el mismo entorno de hardware y software. La Tabla 18 documenta la configuración utilizada con el objetivo de facilitar la reproducibilidad de los resultados.

Los experimentos se ejecutan dentro de un contenedor Docker, sobre un computador de escritorio con Windows. En Windows, Docker Desktop ejecuta los contenedores en una máquina virtual ligera de WSL2 (*Windows Subsystem for Linux*), que corre sobre el hipervisor del sistema. No hay emulación: el código se ejecuta directamente sobre el procesador del anfitrión, y el contenedor ve el mismo modelo de procesador y sus 16 núcleos lógicos, sin límite de CPU. Hay tres diferencias con el anfitrión, por las que la tabla informa ambos niveles:

- **Memoria:** la máquina virtual de WSL2 recibe por omisión la mitad de la memoria del anfitrión, de modo que el contenedor ve unos 30 GiB de los 61.6 GiB que ve Windows, sin un límite adicional del contenedor.
- **Sistema operativo y kernel:** el contenedor usa el espacio de usuario de Ubuntu de la imagen de SageMath sobre el kernel Linux de WSL2, no el de Windows.
- **Planificación:** los núcleos virtuales los planifica Windows, de modo que otros procesos del anfitrión y los cambios de frecuencia del procesador pueden introducir variación en los tiempos. Se mitiga con las ejecuciones de calentamiento, el orden aleatorio de los candidatos y la repetición de cada medición (Sección 4.6). Opcionalmente, el contenedor puede fijarse a núcleos concretos con la variable `BENCHMARK_CPUSET` del script del benchmark, que también queda registrada en los metadatos.

Todas las mediciones de tiempo y memoria se toman dentro del contenedor. Además, el benchmark registra estos datos de forma automática en los metadatos de cada corrida, junto con el *commit* evaluado y el identificador de la imagen.

Tabla 18: Entorno utilizado para ejecutar los experimentos.

Elemento	**Configuración**
Procesador	AMD Ryzen 7 9700X, frecuencia base de 3.8 GHz
Arquitectura	x86_64 (AMD64)
Núcleos físicos	8
Núcleos lógicos	16 (SMT); el contenedor ve los 16, sin límite de CPU
Memoria RAM	Anfitrión: 61.6 GiB visibles para Windows. Contenedor: 30.2 GiB, asignados a la máquina virtual de WSL2 (valor por omisión), sin límite adicional
Sistema operativo	Anfitrión: Windows 11 Pro 10.0.26200, con WSL 2.6.1. Contenedor: Ubuntu 24.04.5 LTS
Kernel	Linux 6.6.87.2-microsoft-standard-WSL2 (kernel de WSL2 que ejecuta los contenedores)
Docker	Docker Desktop con Docker Engine 28.4.0 y Docker Compose 2.39.2; cgroups v2
Imagen base	`sagemath/sagemath:latest`; imagen del experimento construida el 30 de septiembre de 2026 (`sha256:e59db430fb1a`)
SageMath	10.10
Python	CPython 3.12.3 (entorno de SageMath)
Biblioteca criptográfica	`cryptography` 50.0.2, con OpenSSL 4.0.3, para ECDSA P-256, ML-KEM-768 y AES-256-GCM; NumPy 2.4.4 para la aritmética de retículos
Código del experimento	*Commit* `05a59f7` (Sección 4.22)
Fecha de ejecución	20 de octubre de 2026, de 13:50 a 13:52 UTC

Todos los candidatos fueron ejecutados sobre el mismo hardware y utilizando el mismo mecanismo de instrumentación. Durante la toma principal de resultados se evitó ejecutar aplicaciones ajenas al experimento que pudieran introducir carga significativa sobre el sistema.

4.5. Configuración criptográfica

Los parámetros criptográficos empleados para cada candidato se presentan en la Tabla 19.

Tabla 19: Parámetros criptográficos utilizados en cada implementación.

Protocolo	**Dimensión**	**Módulo / curva**	**Distribución / error**	**Otros parámetros**
ECDSA	N/A	P-256 (`secp256r1`)	N/A	Hash SHA-256; reto de 32 bytes firmado por el cliente.
LWE estándar	$n = m = 1116$; $\ell = 128$ columnas secretas	$q = 8\,380\,417$	Secreto y error CBD(2)	Fiat–Shamir con abortos: desafío con $\kappa = 31$ coeficientes ± 1, máscara uniforme de anchura $\gamma = 2^{17}$, SHA3-256 y SHAKE256.
Binary-LWE	$n = m = 1196$; $\ell = 128$	$q = 8\,380\,417$	Secreto uniforme en $\{0, 1\}$; error CBD(2)	Igual que LWE estándar ($\kappa = 31$, $\gamma = 2^{17}$).
Ring-LWE	$n = 1024$	$q = 8\,380\,417$; polinomio $x^{1024} + 1$	Secreto y error CBD(6)	Desafío polinomial con $\kappa = 16$ coeficientes ± 1, $\gamma = 2^{17}$; producto negacíclico directo.
LWR	$n = m = 1112$; $\ell = 128$	$q = 2^{23}$, $p = 2^{21}$	Secreto CBD(2); sin error aleatorio	Redondeo al entero más cercano; $\kappa = 31$, $\gamma = 2^{17}$.
LWE propuesto (ML-ZKAuth)	Prueba: $k' = 4$, $d = 256$ (1024); transporte: $k = 3$ (768)	$q' = 8\,380\,417$ (prueba); $q = 3329$ (transporte)	CBD(6) en la clave de identidad, CBD(2) en el transporte; máscara uniforme	$\kappa = 23$, $\beta = 138$, $\gamma_1 = 2^{17}$, $\gamma_2 = 95\,232$, $\Delta_\nu = 300$ s; ML-KEM-768 y AES-256-GCM (Tabla 6).

Los parámetros fueron seleccionados buscando un nivel de seguridad comparable entre los candidatos. Los de los cinco candidatos de retículos se ajustaron para que la clave alcance entre $2^{128{,}0}$ y $2^{128{,}6}$ en el modelo Core-SVP (Sección 2.3.6); el resultado de esa estimación se presenta en la Sección 4.13.

4.5.1. Tamaños y costos analíticos de ML-ZKAuth

Los tamaños de los objetos de ML-ZKAuth se derivan de sus parámetros y de la codificación del prototipo (Sección 3.13):

1. **Memoria del cliente:**

   - Clave privada $(\mathbf{s}, \mathbf{e}, \mathsf{id}_{\mathcal{C}})$: 1 056 bytes. Cada uno de $\mathbf{s}$ y $\mathbf{e}$ tiene $k' \times d = 4 \times 256 = 1\,024$ coeficientes muestreados de CBD(6), que toman 13 valores, de -6 a 6, y caben en 4 bits ($2^4 = 16$): $1\,024 \times 4/8 = 512$ bytes por vector. El identificador $\mathsf{id}_{\mathcal{C}} = \mathsf{H}(\mathbf{t})$ es un resumen SHA3-256 de 32 bytes. En total, $2 \times 512 + 32 = 1\,056$ bytes.
   - Clave pública $\mathbf{t}$: 2 944 bytes, porque sus 1 024 coeficientes son enteros módulo $q' = 8\,380\,417 < 2^{23}$ y ocupan 23 bits cada uno: $1\,024 \times 23/8 = 2\,944$. El cliente no necesita guardarla, porque usa su resumen $\mathsf{id}_{\mathcal{C}}$.
   - Matrices públicas: $\mathbf{A}'$ tiene $k'^2 d = 4\,096$ coeficientes de 23 bits (11 776 bytes) y $\mathbf{A}$, $k^2 d = 2\,304$ coeficientes de 12 bits (3 456 bytes). Se regeneran desde la semilla ρ de 32 bytes y no ocupan almacenamiento permanente.

2. **Comunicación por autenticación:**

   - Reto (tk, ν): 40 bytes, el ticket aleatorio de 32 bytes (256 bits) y la marca de tiempo ν, en milisegundos, como entero de 64 bits (8 bytes).
   - Token: 3 524 bytes. tok_1 es el texto cifrado de ML-KEM-768, de 1 088 bytes [27]. tok_2 tiene 2 436 bytes: el *nonce* de AES-GCM (12), el contenido cifrado, que ocupa lo mismo que el contenido en claro (2 408), y la etiqueta de autenticación (16). Ese contenido es la respuesta $\mathbf{z}$, cuyos 1 024 coeficientes cumplen $|z_i| < \gamma_1 = 2^{17}$ y se codifican con 18 bits ($1\,024 \times 18/8 = 2\,304$ bytes), más la semilla del desafío $\tilde{c}$ (32), el ticket (32), la marca de tiempo (8) y el identificador $\mathsf{id}_{\mathcal{C}}$ (32): $2\,304 + 32 + 32 + 8 + 32 = 2\,408$ bytes.
   - Token con acuerdo de clave: 4 676 bytes, porque tok_2 lleva además la cuota $\mathbf{b}_{\mathcal{C}} \in \mathcal{R}_q^3$, con $3 \times 256 = 768$ coeficientes módulo $q = 3329 < 2^{12}$ de 12 bits cada uno: $768 \times 12/8 = 1\,152$ bytes.
   - Respuesta del servidor con $(\mathbf{b}_{\mathcal{S}}, \mathbf{v})$, solo si se acuerda clave de sesión: 1 212 bytes. La cuota $\mathbf{b}_{\mathcal{S}}$ ocupa 1 152 bytes, como $\mathbf{b}_{\mathcal{C}}$, y la pista $\mathbf{v}$, un bit por cada uno de los 256 bits de la clave de sesión, 32 bytes; cifrados con AES-GCM, se suman el *nonce* y la etiqueta: $12 + 1\,152 + 32 + 16 = 1\,212$ bytes.

3. **Cómputo:**

- Aritmética: los coeficientes de $\mathcal{R}_{q'}$ caben en 23 bits, pero sus productos ocupan hasta 46 bits y las sumas de productos algo más, de modo que un microcontrolador de 32 bits necesita multiplicaciones de $32 \times 32 \to 64$ bits, sin bibliotecas de multiprecisión [34]. El prototipo usa enteros de 64 bits de NumPy y la convolución negacíclica directa, sin NTT.
- Probador: en promedio, unos 13 intentos de muestreo por rechazo por autenticación.
- Tiempo constante: el muestreo CBD admite implementaciones de tiempo constante en un dispositivo objetivo (Sección 2.3.4); el prototipo en Python no lo es, y el número de intentos del probador se refleja en su tiempo de ejecución.

Estos valores son analíticos; las mediciones del prototipo se reportan en la Sección 4.10. Los tamaños medidos son mayores, 1 450 bytes la clave privada y 3 936 la pública, porque la serialización JSON codifica los bytes en Base64, que añade un tercio, y agrega los nombres de los campos.

4.6. Diseño de los experimentos

4.6.1. Número de ejecuciones

Cada operación fue ejecutada:

$$N = 200 \tag{78}$$

veces por configuración: 200 configuraciones del sistema, 200 generaciones de claves y 200 autenticaciones completas, cada una medida por etapas (reto, respuesta y verificación). Las autenticaciones alternan entre 5 pares de claves generados de antemano, para que el resultado no dependa de una sola clave. La memoria se mide en una fase aparte, con 20 repeticiones por operación.

Antes de iniciar la toma de datos se realizaron:

$$N_{\text{warm-up}} = 20 \tag{79}$$

ejecuciones de calentamiento por candidato, cada una con generación de claves, reto, respuesta y verificación, que no fueron incluidas en los resultados finales. Los candidatos se ejecutan en un orden aleatorio fijado por una semilla registrada en la configuración, con una pausa de 1 s entre candidatos.

4.6.2. Estadísticos reportados

Para evitar representar el rendimiento mediante una única medición, para cada operación se calcularon la media aritmética, la mediana, la desviación estándar, el mínimo, el máximo y los percentiles 95 y 99.

4.7. Métricas evaluadas

Las métricas utilizadas se agrupan en seis dimensiones: rendimiento computacional, consumo de recursos, comunicación, seguridad, características operacionales y experiencia de usuario, y madurez y facilidad de despliegue. Las subsecciones siguientes detallan qué se mide en cada una.

4.7.1. Rendimiento computacional

Para cada protocolo se mide, en milisegundos, el tiempo de la configuración del sistema, de la generación de claves, de la generación del reto, de la generación de la respuesta, de la verificación y de la autenticación completa. Adicionalmente se calcula el *throughput*:

$$R = \frac{N_{\text{auth}}}{T_{\text{total}}}, \tag{80}$$

expresado en autenticaciones por segundo.

4.7.2. Consumo de memoria y almacenamiento

Para cada implementación se estudian el pico de memoria asignada durante cada operación y el incremento de la memoria residente (RSS) del proceso; el tamaño de la clave pública, de la clave privada y de los parámetros públicos adicionales; y el almacenamiento persistente requerido por usuario. Los tamaños de estructuras criptográficas se calculan utilizando su representación serializada real y no únicamente su representación matemática abstracta.

4.7.3. Comunicación

El volumen de comunicación se calcula utilizando los bytes efectivamente serializados por la aplicación.

Para una autenticación:

$$B_{\text{auth}} = B_{\text{challenge}} + B_{\text{response}}, \tag{81}$$

donde cada término incluye la carga criptográfica del mensaje y el sobre que la contiene, cuyo tamaño conjunto se informa aparte como B_{metadata}. Además se registran el número de mensajes y de viajes de ida y vuelta (*round trips*) de la autenticación medida.

Para estudiar el efecto de la red, el tiempo de una autenticación se proyecta en cinco escenarios sintéticos: local (0.2 ms de ida y vuelta y 1000 Mbit/s), baja latencia (10 ms y 100 Mbit/s), latencia media (50 ms y 25 Mbit/s), alta latencia (100 ms y 10 Mbit/s) y red restringida (250 ms y 1 Mbit/s). La proyección suma al tiempo medio de autenticación y de serialización un viaje de ida y vuelta y el tiempo de transmisión de B_{auth}:

$$T_{\text{red}} = T_{\text{auth}} + T_{\text{ser}} + \text{RTT} + \frac{8\,B_{\text{auth}}}{W}, \tag{82}$$

donde W es el ancho de banda en bits por segundo. Es un modelo determinista para comparar candidatos, no una emulación de red: no incluye el establecimiento de la conexión, las cabeceras de TCP, TLS o HTTP, la congestión ni las retransmisiones.

4.7.4. Seguridad

La evaluación de seguridad considera el problema matemático subyacente y los parámetros utilizados, las estimaciones de seguridad clásica y frente a adversarios cuánticos, los ataques conocidos relevantes, la sensibilidad a una selección incorrecta de parámetros, los requisitos de aleatoriedad, la probabilidad de fallo cuando sea aplicable y las vulnerabilidades derivadas de la implementación.

4.7.5. Características operacionales

Para estudiar las implicaciones en un sistema real se consideran el número de pasos del registro y de una autenticación; la necesidad de almacenar secretos en el dispositivo del usuario, de contar con un servidor en línea y de mantener estado por usuario; la posibilidad de que un tercero verifique una autenticación; los procedimientos de renovación de claves y ante el compromiso de credenciales; y los requerimientos de infraestructura adicionales. A ello se suma la madurez de cada alternativa, es decir, si está estandarizada y si existen implementaciones consolidadas.

4.8. Resultados de rendimiento

4.8.1. Tiempo de generación de claves

Tabla 20: Tiempo de generación de claves, en milisegundos.

Protocolo	Media	Mediana	Desv.	P95	P99	Máx.
ECDSA	0.0180	0.0143	0.0148	0.0395	0.101	0.138
LWE	17.36	17.24	0.639	18.47	19.33	21.43
Binary-LWE	15.58	15.46	0.713	16.31	17.47	22.78
Ring-LWE	0.472	0.463	0.0496	0.566	0.612	0.828
LWR	15.41	15.13	1.126	16.87	18.19	24.59
ML-ZKAuth	0.812	0.785	0.0929	0.998	1.133	1.287

Los resultados de esta sección corresponden a la corrida oficial descrita en la Sección 4.22, con 200 mediciones por operación y candidato. Los tres candidatos de matriz (LWE, Binary-LWE y LWR) generan sus claves en 15 a 17 ms, porque calculan el producto de una matriz pública de más de mil filas por las 128 columnas del secreto. Los dos candidatos sobre anillos generan las suyas en menos de 1 ms: 0.47 ms Ring-LWE y 0.81 ms ML-ZKAuth. ECDSA lo hace en 0.018 ms. La configuración del sistema, que se ejecuta una vez por despliegue, cuesta entre 11 y 14 ms en los candidatos de matriz, que muestrean y codifican una matriz pública de más de un millón de coeficientes, y menos de 0.1 ms en los demás (Figura 1).

4.8.2. Tiempo de autenticación

Tabla 21: Tiempo promedio por etapa del proceso de autenticación.

Protocolo	Challenge	Response	Verify	Total	Throughput
	ms	ms	ms	ms	auth/s
ECDSA	0.0009	0.0320	0.0438	0.0728	13 740
LWE	0.0387	6.870	6.626	13.41	74.6
Binary-LWE	0.0504	7.539	7.848	15.53	64.4
Ring-LWE	0.0273	4.985	0.788	5.849	171.0
LWR	0.0718	8.596	8.201	15.68	63.8
ML-ZKAuth	0.0363	9.739	0.978	9.723	102.8

El tiempo total se mide en una autenticación completa independiente de las mediciones por etapa, con un reto nuevo, de modo que no coincide exactamente con la suma de las tres columnas anteriores. La diferencia es mayor en ML-ZKAuth (9.723 ms frente a 10.753 ms de la suma), porque su probador repite un número aleatorio de intentos (Sección 3.7) y cada medición toma una muestra distinta de ese número. El *throughput* es secuencial: el número de autenticaciones dividido por la suma de sus tiempos, sobre las 200 autenticaciones completas, y no representa la capacidad concurrente de un servidor.

4.8.3. Distribución de los tiempos

La Figura 1 compara, para cada protocolo, el tiempo medio de la configuración del sistema, del registro (generación de claves) y de una autenticación completa, y la Figura 2 desglosa la autenticación en sus etapas.

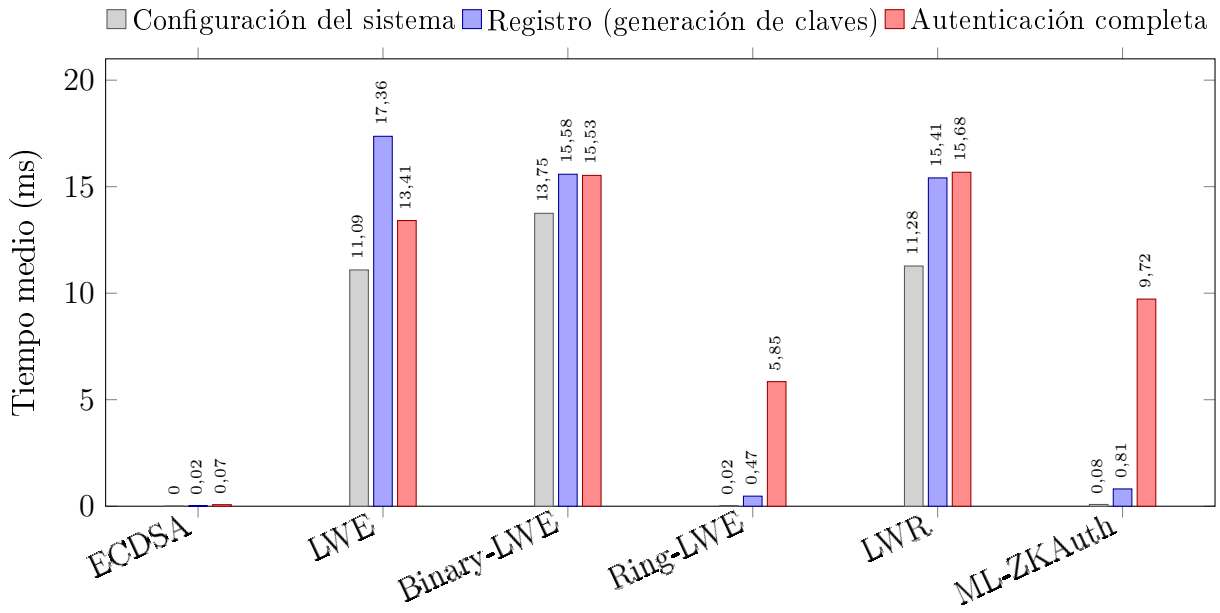


Figura 1: Tiempo medio de configuración del sistema, de registro y de autenticación por protocolo (200 mediciones de cada operación por candidato). La configuración se ejecuta una vez por despliegue; el registro, una vez por usuario; la autenticación, en cada inicio de sesión.

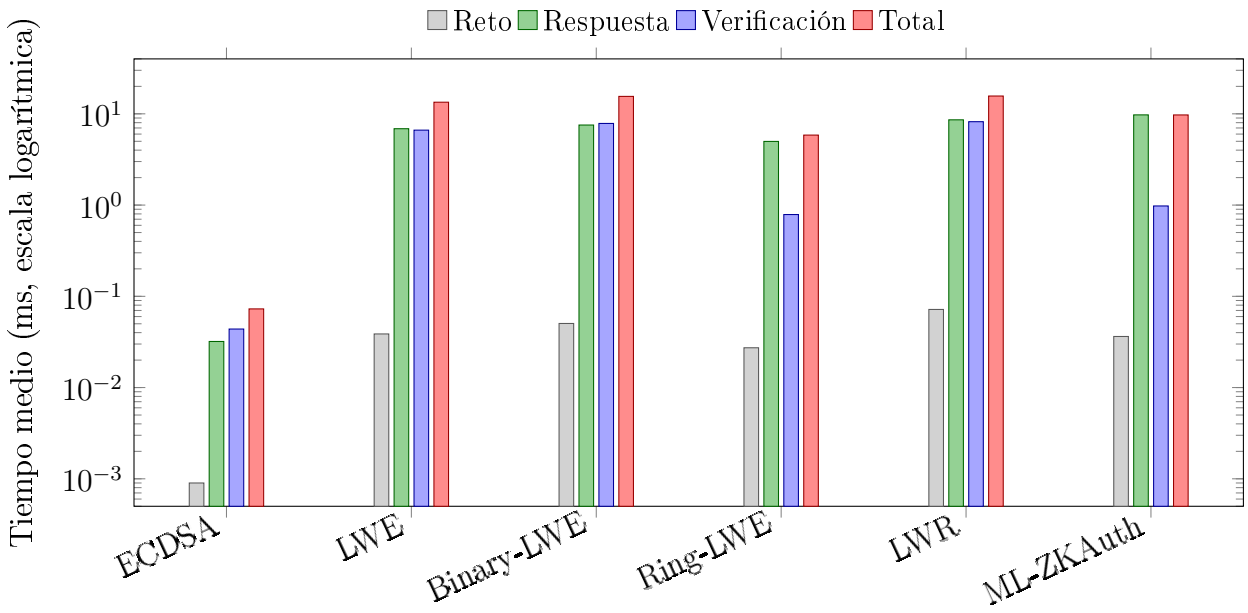


Figura 2: Tiempo medio de cada etapa de la autenticación por protocolo (200 autenticaciones por candidato). El eje vertical es logarítmico, para que se vean a la vez los tiempos del reto, de centésimas de milisegundo, y los de la respuesta, de varios milisegundos. El total se mide en una autenticación completa independiente (Tabla 21).

Los resultados muestran tres grupos. ECDSA completa una autenticación en 0.073 ms de media. Los candidatos de retículos tardan entre 5.85 ms (Ring-LWE) y 15.68 ms (LWR), entre 80 y 215 veces más que ECDSA. Dentro de ellos, los tres candidatos de matriz tardan entre 13.4 y 15.7 ms y reparten el costo casi por igual entre la respuesta y la verificación, mientras que los dos candidatos sobre anillos verifican en menos de 1 ms y concentran el costo en la respuesta: en ML-ZKAuth, la respuesta cuesta unas 10 veces más que la verificación.

La diferencia entre ML-ZKAuth (9.72 ms) y Ring-LWE (5.85 ms), los dos candidatos de retículos más rápidos, fue de 3.87 ms; ML-ZKAuth fue 1.38 veces más rápido que LWE estándar. Los dos candidatos sobre anillos presentan además la mayor dispersión, porque su probador repite un número aleatorio de intentos: ML-ZKAuth tiene una mediana de 7.53 ms y un percentil 95 de 24.1 ms, y Ring-LWE, una mediana de 4.03 ms y un percentil 95 de 14.9 ms. LWR registró una medición aislada de 234 ms, que eleva su media y su desviación; su mediana, 13.48 ms, es comparable a la de LWE.

Estas diferencias reflejan tanto los algoritmos como sus implementaciones: ECDSA usa la biblioteca `cryptography`, que ejecuta código compilado de OpenSSL, mientras que los candidatos de retículos son prototipos en Python con NumPy (Sección 4.21).

4.9. Resultados de consumo de memoria

Tabla 22: Consumo de memoria durante las operaciones criptográficas: pico medio de asignaciones de Python por operación y estado persistente por usuario.

Protocolo	**Configuración**	**Generación de claves**	**Autenticación**	**Estado persistente**
	KB	**KB**	**KB**	**bytes/usuario**
ECDSA	58.9	54.4	44.1	592
LWE	30 441.7	22 848.6	16 866.1	1 142 874
Binary-LWE	34 956.5	25 978.6	19 193.7	1 046 198
Ring-LWE	56.1	111.7	154.1	8 290
LWR	30 229.6	22 697.6	16 747.2	1 138 786
ML-ZKAuth	55.4	921.6	945.1	5 386

Las tres primeras columnas recogen el pico de memoria asignada por Python durante cada operación, medido con `tracemalloc`, como media de 20 repeticiones. El incremento de la memoria residente (RSS) del proceso, que el benchmark también registra, fue nulo durante la autenticación en los seis candidatos: el proceso reutiliza memoria que ya había reservado durante la configuración y el calentamiento. Solo se observaron incrementos de RSS al configurar y generar claves en los candidatos de matriz, de hasta unos 21 MB en la configuración de Binary-LWE. Las operaciones más simples, como la emisión del reto, registran unos 55 KB en todos los candidatos, que corresponden sobre todo a la propia instrumentación.

Durante la autenticación, los candidatos de matriz asignan entre 16.4 y 18.7 MB, porque operan con su matriz pública completa; ML-ZKAuth asigna unos 0.9 MB, Ring-LWE unos 0.15 MB y ECDSA unos 44 KB. El estado persistente es la suma de la clave pública, que guarda el servidor, y de la clave privada, que guarda el cliente.

4.10. Tamaño de claves y estructuras criptográficas

Tabla 23: Tamaño serializado de los principales objetos criptográficos.

Protocolo	Clave pública	Clave privada	Parámetros públicos	Total registro
	bytes	bytes	bytes	bytes
ECDSA	254	338	0	592
LWE	761 864	381 010	6 642 953	1 142 874
Binary-LWE	816 480	229 718	7 629 396	1 046 198
Ring-LWE	5 472	2 818	6 002	8 290
LWR	759 136	379 650	6 595 459	1 138 786
ML-ZKAuth	3 936	1 450	2 262	5 386

Los tamaños son los de la serialización JSON canónica que mide el benchmark, con los datos binarios codificados en Base64; por eso superan en cerca de un tercio a los tamaños binarios. Por ejemplo, la respuesta de ML-ZKAuth ocupa 4 721 bytes frente a los 3 524 bytes del token binario (Sección 3.8). La columna *Total registro* suma la clave pública y la privada de un usuario; los parámetros públicos se comparten entre todos los usuarios de un despliegue y se distribuyen una sola vez. Los candidatos de matriz tienen claves públicas de 759 a 816 KB y parámetros públicos de 6.6 a 7.6 MB, mientras que los dos candidatos sobre anillos tienen claves públicas de 3.9 y 5.5 KB.

4.11. Resultados de comunicación

Tabla 24: Datos transmitidos durante una autenticación.

Protocolo	Challenge	Response	Metadata	Total	Mensajes
	bytes	bytes	bytes	bytes	medidos
ECDSA	106	161	99	267	2
LWE	113	12 028	113	12 141	2
Binary-LWE	111	12 882	109	12 993	2
Ring-LWE	109	11 048	105	11 157	2
LWR	104	11 979	95	12 083	2
ML-ZKAuth	133	4 777	113	4 910	2

El benchmark mide los dos mensajes del contrato común (Sección 4.2), el reto y la respuesta, que forman un viaje de ida y vuelta, cada uno dentro de un sobre canónico con el tipo de mensaje, el protocolo y la carga criptográfica. La columna *Metadata* es el tamaño de esos sobres sin la carga criptográfica. Las cifras no incluyen cabeceras HTTP, TLS ni TCP/IP. ECDSA transmite 267 bytes; los candidatos de matriz y Ring-LWE, entre 11.2 y 13.0 KB; ML-ZKAuth, 4 910 bytes, 2.3 veces menos que Ring-LWE y 18.4 veces más que ECDSA.

Un inicio de sesión completo tiene además una solicitud inicial del cliente y un aviso del resultado, con lo que son cuatro mensajes en dos viajes de ida y vuelta. Esos dos mensajes existen en los seis protocolos y no dependen de su criptografía, por lo que se excluyen de todos por igual; en ML-ZKAuth son el Hello y el resultado (Tabla 8). El acuerdo de clave de sesión de ML-ZKAuth queda fuera del contrato y no se mide: si se usa, el token crece 1 152 bytes por la cuota $\mathbf{b}_{\mathcal{C}}$ y el resultado lleva 1 212 bytes con $(\mathbf{b}_{\mathcal{S}}, \mathbf{v})$ (Sección 4.5.1).

La Tabla 25 proyecta el tiempo del reto y la respuesta en los cinco escenarios de red de la Sección 4.7. Con una red rápida domina el cómputo y el orden es el de la Tabla 21; a medida que baja el ancho de banda pesa más el volumen transmitido, y desde el escenario de alta latencia ML-ZKAuth pasa a ser el candidato de retículos más rápido, por delante de Ring-LWE. Un inicio de sesión completo suma el segundo viaje de ida y vuelta, igual para los seis: de 0.2 a 250 ms según el escenario. Con acuerdo de clave, ML-ZKAuth transmite además 2 364 bytes, que en la red restringida añaden unos 19 ms.

Tabla 25: Tiempo proyectado del reto y la respuesta por escenario de red, en milisegundos (un viaje de ida y vuelta).

Protocolo	**Local**	**Baja latencia**	**Latencia media**	**Alta latencia**	**Restringida**
	0.2 ms; 1 000 Mbit/s	10 ms; 100 Mbit/s	50 ms; 25 Mbit/s	100 ms; 10 Mbit/s	250 ms; 1 Mbit/s
ECDSA	0.3	10.1	50.2	100.3	252.2
LWE	13.8	24.4	67.4	123.2	360.6
Binary-LWE	15.9	26.6	69.8	126.0	369.5
Ring-LWE	6.2	16.8	59.5	114.8	345.2
LWR	16.1	26.7	69.6	125.4	362.4
ML-ZKAuth	10.0	20.1	61.3	113.7	299.0

4.12. Escalabilidad de almacenamiento

A partir del almacenamiento requerido por usuario se estima el espacio necesario para diferentes tamaños del sistema.

Si S_u representa el almacenamiento persistente requerido por usuario, entonces:

$$S(N) = NS_u. \tag{83}$$

Tabla 26: Estimación de almacenamiento según el número de usuarios.

Protocolo	**1 usuario**	**1.000 usuarios**	**100.000 usuarios**	**1.000.000 usuarios**
ECDSA	254 B	254 kB	25.4 MB	254 MB
LWE	762 kB	762 MB	76.2 GB	762 GB
Binary-LWE	816 kB	816 MB	81.6 GB	816 GB
Ring-LWE	5.5 kB	5.5 MB	547 MB	5.5 GB
LWR	759 kB	759 MB	75.9 GB	759 GB
ML-ZKAuth	3.9 kB	3.9 MB	394 MB	3.9 GB

La tabla toma como S_u la clave pública serializada, que es lo que el servidor guarda por cada usuario; la clave privada queda en el dispositivo del cliente y los parámetros públicos se guardan una sola vez por despliegue. Con un millón de usuarios, los candidatos de matriz requieren entre 759 y 816 GB, mientras que Ring-LWE y ML-ZKAuth requieren 5.5 y 3.9 GB. ML-ZKAuth guarda además el registro de tickets consumidos, cuyo tamaño no crece con el número de usuarios registrados sino con la tasa de autenticaciones (Sección 3.9).

4.13. Evaluación de seguridad

La seguridad de las alternativas basadas en retículos no se determina únicamente por el nombre de la familia criptográfica. Depende de los parámetros concretos seleccionados para cada implementación.

Por este motivo, para cada candidato se documentan los parámetros y la estimación de seguridad correspondiente.

Tabla 27: Nivel de seguridad estimado para las configuraciones evaluadas.

Protocolo	**Problema base**	**Seguridad clásica**	**Seguridad cuántica**	**Método de estimación**
ECDSA	ECDLP (P-256)	2^{128}	Roto (algoritmo de Shor)	Valor de referencia de la curva
LWE	LWE	$2^{128,0}$	$2^{116,9}$	Core-SVP; dual híbrido, $\beta_{\mathrm{BKZ}} = 438$
Binary-LWE	LWE con secreto binario	$2^{128,5}$	$2^{117,9}$	Core-SVP; dual híbrido, $\beta_{\mathrm{BKZ}} = 440$
Ring-LWE	Ring-LWE	$2^{128,2}$	$2^{116,3}$	Core-SVP; primal, $\beta_{\mathrm{BKZ}} = 439$
LWR	LWR	$2^{128,6}$	$2^{117,4}$	Core-SVP; dual híbrido, $\beta_{\mathrm{BKZ}} = 440$
ML-ZKAuth	Module-LWE y SelfTargetMSIS sobre $\mathcal{R}_{q'}$	$2^{128,2}$	$2^{116,3}$	Core-SVP; primal, $\beta_{\mathrm{BKZ}} = 439$

Las cifras de los candidatos de retículos corresponden al ataque más barato contra la clave, estimado con el *lattice-estimator* (*commit* `53da598`) en el modelo de costo Core-SVP, tomando el mínimo entre los ataques primal y dual híbrido (Sección 2.3.6). La solidez de sus pruebas, estimada como un problema SIS o MSIS, queda por encima en todos los casos: $2^{168,5}$ (LWE), $2^{184,3}$ (Binary-LWE), $2^{150,7}$ (Ring-LWE), $2^{167,9}$ (LWR) y $2^{150,7}$ (protocolo propio). Sus espacios de desafíos tienen entre $2^{129,6}$ y $2^{131,6}$ elementos, que una búsqueda de Grover reduce a unas 2^{65} evaluaciones del hash, el mismo orden que la búsqueda de Grover sobre una clave de AES-128 (Sección 3.11). El valor de ECDSA es el de referencia para P-256 y no se estima con la herramienta. Los parámetros de entrada de cada instancia y el procedimiento completo están documentados en el repositorio del proyecto (`docs/estimacion-de-dureza.md`).

4.14. Análisis de ataques

Esta sección resume, para cada protocolo, las principales familias de ataques, la condición que las haría efectivas y la forma en que los parámetros y la implementación evaluados las

mitigan. Que exista una familia de ataque no implica que el protocolo sea vulnerable con los parámetros empleados; se documenta para identificar las condiciones que deben mantenerse para conservar el nivel de seguridad esperado. Las cifras de dureza son las estimaciones Core-SVP clásicas de la Sección 4.13.

ECDSA En ECDSA hay tres riesgos que considerar. El primero es el adversario cuántico: el algoritmo de Shor resuelve el logaritmo discreto en tiempo polinomial [31, 32, 34], de modo que un computador cuántico de escala suficiente recuperaría la clave privada a partir de la pública. No hay mitigación dentro de ECDSA: frente a ese adversario, la única medida es migrar a un esquema poscuántico. El segundo es el número secreto k que se usa en cada firma: si se repite en dos firmas o se filtra, la clave privada se recupera de inmediato, por lo que debe ser secreto, impredecible y distinto en cada firma [44]. En la implementación evaluada lo genera OpenSSL a través de la biblioteca `cryptography`. El tercero es la repetición: el cliente firma un reto aleatorio de 32 bytes emitido por el verificador, de modo que una firma capturada no sirve para otro reto; basta con que el verificador acepte cada reto una sola vez. Frente a un adversario clásico, P-256 ofrece 128 bits de seguridad [43].

Ataques comunes a los candidatos de retículos Los cinco candidatos de retículos usan Fiat–Shamir con abortos y comparten, por tanto, las mismas familias de ataques. La *reducción de retículos contra la clave*, con los ataques primal, dual e híbrido (Sección 2.3.6), busca recuperar el secreto a partir de la clave pública; su costo depende de la dimensión, del módulo y del ruido, y los parámetros se ajustaron para que el ataque más barato cueste al menos 2^{128} (Tabla 27). La *falsificación de pruebas* sin conocer la clave exigiría resolver una instancia de SelfTargetSIS o de su variante estructurada, cuya dureza se estima como la de un problema SIS o MSIS: entre $2^{150,7}$ y $2^{184,3}$, superior a la de la clave. La *búsqueda genérica del desafío*, que fija un desafío y prueba compromisos hasta que el hash lo produzca, necesita del orden de tantas evaluaciones del hash como desafíos posibles, entre $2^{129,6}$ y $2^{131,6}$ según el candidato; el algoritmo de Grover reduce ese costo a unas 2^{65} evaluaciones. La *filtración del secreto a través de la respuesta* se evita con el muestreo por rechazo, que hace que la respuesta publicada sea uniforme en su caja e independiente de la clave (Sección 2.2.2). La *repetición* se evita porque el reto del verificador (en ML-ZKAuth, el ticket y la marca de tiempo) y el resumen de la clave pública entran en el hash: una respuesta no verifica con otro reto ni con otra clave, algo que el benchmark comprueba antes de medir (Sección 4.15). Por último, los *canales laterales* quedan fuera del alcance: los prototipos, escritos en Python con NumPy, no son de tiempo constante.

LWE estándar El ataque más barato contra su clave es el dual híbrido, con un bloque BKZ de 438 y un costo de $2^{128,0}$. Para alcanzarlo, la dimensión se fijó en $n = m = 1116$, porque con $n = m = 1024$ la clave quedaría en $2^{114,9}$. Su solidez está en $2^{168,5}$ y su espacio de desafíos tiene $2^{129,6}$ elementos. Su riesgo principal es una selección de parámetros por debajo de la meta, que el ajuste documentado en el repositorio del proyecto evita (`docs/estimacion-de-dureza.md`).

Binary-LWE Cada coeficiente de su secreto toma solo dos valores, 0 o 1. Eso favorece a los ataques híbridos, que adivinan una parte del secreto y reducen el resto, y permite reescalar el retículo para aprovechar que el secreto es pequeño. Por eso necesita una dimensión mayor que LWE estándar para alcanzar la misma meta: con $n = m = 1196$, el ataque más barato es el dual híbrido, con $2^{128,5}$. Una comprobación con el catálogo completo de ataques del estimador, sobre instancias de dimensión cercana, no encontró ninguno más barato; las variantes híbridas con encuentro a mitad de camino (*meet-in-the-middle*) sobre el secreto binario costaron más de 2^{165}. Su solidez está en $2^{184,3}$.

Ring-LWE Su clave vive en un anillo de polinomios, con más estructura algebraica que una matriz uniforme. No se conocen ataques que aprovechen esa estructura en estas dimensiones, por lo que se estima como LWE de dimensión 1024; con ruido CBD(6), el ataque más barato es el primal, con $2^{128,2}$. Su riesgo específico es esa suposición: un avance que explotara la estructura del anillo afectaría a Ring-LWE y no a los candidatos de matriz. Su solidez está en $2^{150,7}$, y su espacio de desafíos, formado por polinomios con 16 coeficientes no nulos, tiene $2^{131,6}$ elementos.

LWR No muestrea un error aleatorio: lo sustituye por el redondeo de $q = 2^{23}$ a $p = 2^{21}$. Su seguridad se estima modelando ese redondeo como un error uniforme en $\{-1, 0, 1, 2\}$, la heurística habitual en los esquemas basados en LWR, porque las reducciones formales conocidas de LWR a LWE exigen una razón q/p que crece con el número de muestras y que estos parámetros, con $q/p = 4$, no cumplen. La cifra de $2^{128,6}$ del ataque dual híbrido depende, por tanto, de esa heurística, que es su riesgo específico. Su solidez está en $2^{167,9}$.

ML-ZKAuth Comparte los ataques comunes, con su clave en $2^{128,2}$ (ataque primal), su solidez en $2^{150,7}$ y un espacio de desafíos de $2^{131,1}$ elementos. Como Ring-LWE, se estima como LWE de dimensión 1024, sin aprovechar la estructura algebraica del módulo, y comparte ese riesgo. Su capa de transporte, con los parámetros de ML-KEM-768, está en $2^{174,3}$ y no limita la seguridad (Tabla 11). Los ataques sobre el canal se mitigan dentro del propio protocolo: la interceptación por intermediarios, con la ocultación del token; la repetición, con el ticket

de un solo uso, la marca de tiempo y el registro de tickets consumidos; y la sustitución de la cuota del acuerdo de clave, con el cifrado autenticado del token. Los relés en tiempo real quedan cubiertos solo en parte, y los canales laterales y la denegación de servicio quedan fuera del modelo; la Sección 3.12 y la Tabla 15 detallan cada caso.

4.15. Seguridad de implementación

Además de la seguridad matemática de cada construcción, se analizaron los riesgos relacionados con su implementación. La Tabla 28 resume la fuente de aleatoriedad, los retos y valores de un solo uso, la exposición a canales laterales y la validación de parámetros de cada candidato. Los cuatro candidatos de retículos de referencia comparten la misma plantilla de prueba y, con ella, las mismas propiedades de implementación, por lo que ocupan una sola fila.

Tabla 28: Aspectos de seguridad de las implementaciones evaluadas.

Protocolo	**Aleatoriedad**	**Reto y valores de un solo uso**	**Canales laterales**	**Parámetros**
ECDSA	Sistema operativo (reto); OpenSSL (clave y k)	Reto de 32 bytes del verificador; k nuevo en cada firma	Código compilado de OpenSSL; no se analizó	Fijos: P-256, SHA-256 y reto de 32 bytes
LWE, Binary-LWE, Ring-LWE y LWR	Sistema operativo (claves, máscaras y retos); nunca el generador de NumPy	Reto de 32 bytes ligado al resumen de la clave pública; máscara nueva en cada intento	Python con NumPy, sin tiempo constante; el número de intentos no depende de la clave	Rechaza valores desconocidos, espacios de desafíos menores que 2^{128}, máscaras que no ocultan el secreto y aceptación menor que 0.05
ML-ZKAuth	Sistema operativo (claves, máscaras, tickets y *nonces*); OpenSSL (ML-KEM)	Ticket de 256 bits, marca de tiempo y registro de consumo; *nonce* de AES-GCM de 96 bits	Igual que los candidatos de referencia (Sección 3.11)	Lo mismo y, además, exige que $2\gamma_2$ divida a $q' - 1$; transporte fijo

Además, antes de cualquier medición, el benchmark somete a cada candidato a una verificación de conformidad con el contrato común. Comprueba que sus parámetros, claves y mensajes se puedan serializar, que los retos sean nuevos en cada sesión, que una autenticación válida se acepte, que se rechacen una respuesta repetida con un reto nuevo, una respuesta alterada y una respuesta verificada con otra clave pública, y que ninguna operación modifique sus entradas. Un candidato que falle cualquiera de estas pruebas queda excluido de la comparación; los seis candidatos las superaron en la corrida oficial.

4.16. Requisitos operacionales y experiencia de usuario

La Tabla 29 resume lo que cada protocolo exige al registrar a un usuario y lo que cliente y servidor deben guardar después. Los tamaños son los serializados de la Tabla 23; los candidatos de matriz comparten las mismas características y ocupan una sola fila.

Tabla 29: Comparación de requisitos operacionales.

Protocolo	**Registro**	**Estado del servidor**	**Qué guarda el cliente**	**Verificación por terceros**
ECDSA	Sin parámetros públicos; el cliente envía una clave pública de 254 B	Claves públicas y retos pendientes de respuesta	Clave privada de 338 B	Sí: la firma del reto se verifica con la clave pública
LWE, Binary-LWE y LWR	El cliente obtiene parámetros de 6.6 a 7.6 MB y envía una clave pública de 759 a 816 KB	Igual que ECDSA	Clave privada de 230 a 381 KB y los parámetros	Sí: la prueba se verifica con la clave pública y el reto
Ring-LWE	El cliente obtiene parámetros de 6.0 KB y envía una clave pública de 5.5 KB	Igual que ECDSA	Clave privada de 2.8 KB y los parámetros	Sí, como en los anteriores
ML-ZKAuth	El cliente obtiene (ρ, vpk), de 2.3 KB, y envía una clave pública de 3.9 KB	Además, vsk y el registro de tickets consumidos	Clave privada de 1.4 KB y (ρ, vpk)	Solo el servidor puede leer la prueba; una vez descifrada, cualquiera puede verificarla

Ninguno de los seis funciona sin conexión: en todos, el servidor emite un reto nuevo en cada sesión y debe estar en línea para verificar la respuesta. El registro es un solo paso: el cliente obtiene los parámetros públicos, genera su par de claves y envía la clave pública, que el servidor asocia con su identidad. Un inicio de sesión son cuatro mensajes en dos viajes de ida y vuelta en todos los protocolos, de los que el benchmark mide el reto y la respuesta (Sección 4.11); en ML-ZKAuth, la solicitud inicial (Hello) no lleva el identificador del cliente.

La gestión de claves también es común. Si un dispositivo se pierde o se compromete, el servidor revoca su clave pública y el usuario se registra de nuevo con un par nuevo; los prototipos no implementan revocación ni renovación, que corresponden al despliegue. Cambiar los parámetros públicos invalida todas las claves registradas, porque cada clave pública se calcula con ellos. En ML-ZKAuth eso ocurre al cambiar ρ, pero no al renovar vpk, que solo exige distribuir la clave nueva a los clientes por un canal auténtico. En madurez, la diferencia es grande. ECDSA está estandarizado en FIPS 186-5 [44] y cuenta con implementaciones consolidadas, como la de OpenSSL que usa el benchmark. ML-ZKAuth combina primitivas estandarizadas, ML-KEM en FIPS 203 [27] y una prueba que sigue la plantilla de Dilithium, estandarizado como ML-DSA en FIPS 204 [17], pero el protocolo como tal no está estandarizado. Los cuatro candidatos de referencia implementan esquemas conocidos en la literatura, pero no estandarizados, en prototipos propios.

Desde la perspectiva del usuario final, la mayoría de estas diferencias quedan ocultas por la aplicación: en ninguno de los seis el usuario memoriza ni teclea un secreto, y la autenticación es un intercambio automático entre su dispositivo y el servidor. Lo que puede notar es el tiempo de espera. Sumando a la proyección de la Tabla 25 el segundo viaje de ida y vuelta de un inicio de sesión completo (Sección 4.11), en una red de latencia media (50 ms de ida y vuelta y 25 Mbit/s) la espera va de 100.2 ms (ECDSA) a 119.8 ms (Binary-LWE), con ML-ZKAuth en 111.3 ms; en una red restringida (250 ms y 1 Mbit/s), de 502.2 a 619.5 ms, con ML-ZKAuth en 549.0 ms. En ambos casos la latencia de la red pesa más que el cómputo, y la diferencia entre candidatos es menor que 120 ms. La diferencia que sí sería visible es el registro con un candidato de matriz: en la red restringida, obtener sus parámetros públicos tomaría entre 52.8 y 61.0 s, y enviar su clave pública, entre 6.1 y 6.5 s más, frente a menos de 0.1 s en total con Ring-LWE o ML-ZKAuth. Los parámetros podrían incluirse en la aplicación, a costa de aumentar su tamaño en 6.6 a 7.6 MB.

4.17. Dependencias de confianza y arquitectura

En lugar de tratar la centralización como una propiedad binaria, se analizan explícitamente las dependencias de confianza de cada construcción. ECDSA y los cuatro candidatos de retículos de referencia comparten el mismo modelo y se describen juntos.

ECDSA y los candidatos de retículos de referencia No dependen de una autoridad externa ni de una infraestructura de clave pública: la confianza descansa en el registro, en el que el servidor asocia una clave pública con la identidad del usuario, de modo que ese paso debe hacerse por un canal autenticado. El servidor no guarda secretos propios, solo claves públicas y retos pendientes: quien robe su base de datos no puede suplantar a ningún usuario, aunque quien pueda modificarla podría sustituir una clave pública por la suya. Los cuatro candidatos de retículos dependen además de que los parámetros públicos sean auténticos y se hayan generado al azar, porque todas las claves se calculan con ellos y la dureza de sus problemas supone que son uniformes; como se transmiten completos, el cliente debe confiar en que quien los configuró los generó así.

ML-ZKAuth Comparte esas dependencias, pero la de los parámetros se reduce a una semilla: $\mathbf{A}'$ se deriva de ρ con SHAKE128, de modo que basta con que ρ sea auténtica, y, si SHAKE128 se comporta como una función aleatoria, quien elige ρ no controla la estructura de la matriz. Añade, en cambio, una dependencia: el cliente debe tener la clave pública auténtica del servidor, vpk. El protocolo no define cómo se distribuye; puede incluirse en la aplicación o certificarse con una infraestructura de clave pública. Con una vpk falsa, un atacante descifraría los tokens, sabría qué cliente se autentica y, como la prueba no depende de vpk, podría volver a cifrarla para el servidor legítimo y situarse en medio del acuerdo de clave. El servidor guarda, además, un secreto, vsk, y el registro de tickets consumidos. Si vsk se compromete, el atacante puede leer los tokens y suplantar al servidor ante los clientes, aunque no a los clientes ante el servidor (Tabla 15); si puede modificar el registro, puede borrar entradas y repetir un token ya aceptado dentro de la ventana de 300 s.

4.18. Comparación global

La Tabla 30 resume las principales métricas cuantitativas. Esta tabla no pretende producir un ranking absoluto entre protocolos, sino permitir identificar visualmente los compromisos existentes entre las diferentes alternativas.

Tabla 30: Resumen cuantitativo de los resultados principales.

Protocolo	Auth. (ms)	Memoria pico KB	Comunicación bytes	Clave pública bytes	Clave privada bytes	Seguridad estimada bits
ECDSA	0.0728	44.1	267	254	338	128.0
LWE	13.41	16 866.1	12 141	761 864	381 010	128.0
Binary-LWE	15.53	19 193.7	12 993	816 480	229 718	128.5
Ring-LWE	5.849	154.1	11 157	5 472	2 818	128.2
LWR	15.68	16 747.2	12 083	759 136	379 650	128.6
ML-ZKAuth	9.723	945.1	4 910	3 936	1 450	128.2

El tiempo es la media de la autenticación completa (Tabla 21); la memoria pico, el pico de asignaciones de Python durante la autenticación (Tabla 22); la comunicación y las claves, los tamaños serializados de las Tablas 24 y 23. La seguridad es la estimación clásica de la Tabla 27: Core-SVP para los candidatos de retículos y el valor de referencia de P-256 para ECDSA, que no resiste a un adversario cuántico.

4.19. Análisis multidimensional

Una única métrica no permite determinar la conveniencia de una construcción criptográfica. Un protocolo puede reducir el tiempo de ejecución a costa de incrementar el tamaño de las claves, mientras que otro puede reducir la comunicación a costa de más tiempo de cómputo.

Por esta razón, los resultados se analizan como un conjunto de compromisos entre:

$$(\text{seguridad}, \text{tiempo}, \text{memoria}, \text{comunicación}, \text{requisitos operacionales}). \tag{84}$$

4.19.1. Tiempo frente a comunicación

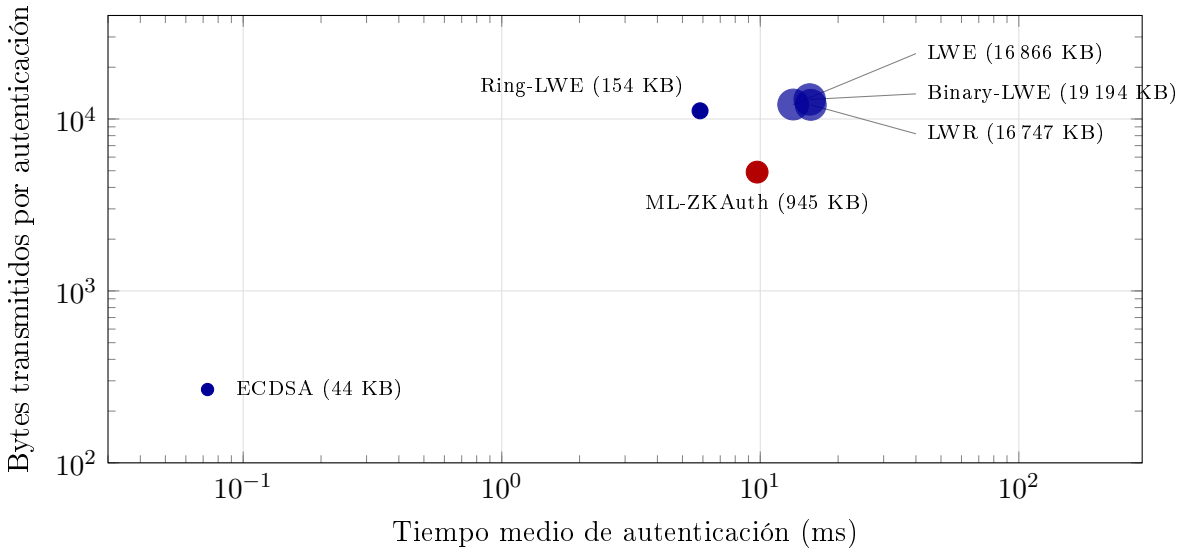


Figura 3: Relación entre tiempo de autenticación, volumen de comunicación y memoria. Ambos ejes son logarítmicos; el tamaño de cada punto crece con el pico de memoria asignada durante la autenticación, indicado entre paréntesis.

4.19.2. Seguridad frente a rendimiento

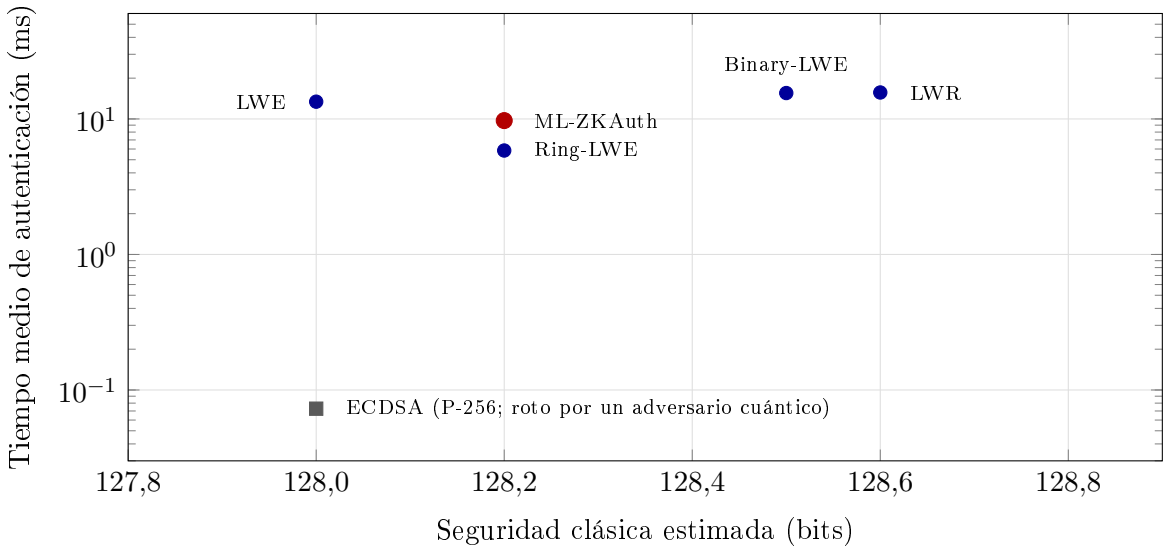


Figura 4: Relación entre seguridad estimada y costo computacional. La seguridad es la estimación clásica de la Tabla 27 (Core-SVP para los candidatos de retículos); el eje de tiempo es logarítmico.

Los cinco candidatos de retículos se ajustaron a la misma dureza estimada, entre $2^{128,0}$ y $2^{128,6}$ en Core-SVP, de modo que sus diferencias de costo, de hasta 2.7 veces en el tiempo medio, no se deben a niveles de seguridad distintos sino a su estructura: matrices frente a anillos y la forma del secreto y del error.

4.20. Discusión

Los experimentos muestran que no existe necesariamente una alternativa que optimice simultáneamente todas las dimensiones estudiadas.

Desde el punto de vista de rendimiento, ECDSA autentica entre 80 y 215 veces más rápido que los candidatos de retículos en este entorno. Entre estos, los construidos sobre anillos son los más rápidos (Ring-LWE, 5.85 ms, y ML-ZKAuth, 9.72 ms), y los de matriz tardan entre 13.4 y 15.7 ms.

Respecto al consumo de memoria, los candidatos de matriz asignan entre 16.4 y 18.7 MB por autenticación, porque operan con su matriz pública completa, frente a unos 0.15 MB de Ring-LWE, 0.9 MB de ML-ZKAuth y 44 KB de ECDSA.

En términos de comunicación, ML-ZKAuth transmite 4 910 bytes por autenticación, la menor cantidad entre los candidatos de retículos (entre 2.3 y 2.6 veces menos que los demás), aunque 18.4 veces más que los 267 bytes de ECDSA.

Respecto a la seguridad, los cinco candidatos de retículos tienen la misma dureza estimada, entre $2^{128,0}$ y $2^{128,6}$ en el modelo Core-SVP clásico y entre $2^{116,3}$ y $2^{117,9}$ en el cuántico, mientras que ECDSA ofrece 128 bits clásicos y ninguna resistencia frente a un adversario cuántico (Sección 4.13).

En términos de despliegue, los candidatos de matriz obligan a distribuir parámetros públicos de 6.6 a 7.6 MB y a guardar unos 0.8 MB por usuario en el servidor, mientras que los candidatos sobre anillos trabajan con unos pocos kilobytes. ML-ZKAuth añade además estado en el servidor: la clave privada del KEM y el registro de tickets consumidos (Sección 3.9).

Estos resultados evidencian un compromiso entre resistencia cuántica y costo, porque los candidatos de retículos pagan con tiempo, memoria y tamaño la protección frente a un adversario cuántico que ECDSA no tiene, y, dentro de los candidatos de retículos, entre estructura algebraica y tamaño: los candidatos sobre anillos reducen claves, parámetros y memoria en varios órdenes de magnitud frente a los de matriz, con la misma dureza estimada.

4.20.1. Comparación con ECDSA

ECDSA constituye el punto de referencia utilizado durante la evaluación.

Frente a esta alternativa, las construcciones basadas en retículos presentan resistencia estimada frente a adversarios cuánticos, con una dureza de $2^{116,3}$ a $2^{117,9}$ en el modelo Core-SVP cuántico, mientras que presentan costos adicionales asociados con el tiempo de autenticación, entre 80 y 215 veces mayor en este entorno, la comunicación, entre 18 y 49 veces mayor, y el tamaño de las claves.

En particular, una autenticación de ML-ZKAuth tarda 9.72 ms frente a 0.073 ms de ECDSA y transmite 4 910 bytes frente a 267. Parte de la diferencia de tiempo se debe a la implementación y no al algoritmo: ECDSA se ejecuta en código compilado de OpenSSL, mientras que los candidatos de retículos son prototipos en Python (Sección 4.21).

4.20.2. Comparación entre las variantes basadas en LWE

Las variantes evaluadas muestran diferencias significativas en tiempo, memoria, tamaño de las claves y parámetros públicos, más que en comunicación, que está entre 11.2 y 13.0 KB para las cuatro.

LWE estándar presentó 13.41 ms por autenticación, 16.5 MB de memoria asignada, una clave pública de 762 KB y parámetros públicos de 6.6 MB.

Binary-LWE presentó la clave privada más pequeña de los candidatos de matriz (230 KB frente a 381 KB), gracias a su secreto binario, pero también la dimensión mayor que ese secreto exige ($n = 1196$), que se traduce en la clave pública, los parámetros (7.6 MB), la memoria y la comunicación más grandes, y en un tiempo de 15.53 ms.

Ring-LWE presentó la autenticación más rápida de los candidatos de retículos (5.85 ms de media y 4.03 ms de mediana), la menor memoria (0.15 MB) y claves de pocos kilobytes, con una dispersión alta de los tiempos (percentil 95 de 14.9 ms).

LWR presentó tiempos y tamaños similares a los de LWE estándar (15.68 ms de media, 13.48 ms de mediana y una clave pública de 759 KB), sin muestrear el error de la clave, que sustituye por un redondeo determinista.

Estas diferencias permiten observar cómo modificaciones en la representación del secreto, la estructura algebraica o el mecanismo utilizado para introducir incertidumbre afectan los costos de una implementación real.

4.20.3. Evaluación del protocolo propuesto

El protocolo propuesto debe analizarse utilizando exactamente los mismos criterios que las demás alternativas.

Los resultados muestran que ML-ZKAuth autentica en 9.72 ms de media (7.53 ms de mediana), con la menor comunicación de los candidatos de retículos, 4 910 bytes, y con claves

de 3.9 KB (pública) y 1.4 KB (privada), a la misma dureza estimada que los demás ($2^{128,2}$ en Core-SVP).

Su mayor ventaja observada corresponde a el tamaño: la menor comunicación y la menor clave pública de los candidatos de retículos, con parámetros públicos de solo 2.3 KB. A ello se suman la ocultación del token y el acuerdo de clave de sesión, que los demás candidatos no ofrecen.

Su principal costo o limitación corresponde a el tiempo del probador: como necesita en promedio unos 13 intentos por prueba, la respuesta concentra el costo (9.74 ms frente a 0.98 ms de la verificación) y el tiempo varía mucho entre autenticaciones (percentil 95 de 24.1 ms). Además, el servidor debe guardar estado: la clave privada del KEM y el registro de tickets consumidos.

Frente a las demás variantes LWE, es entre 1.38 y 1.61 veces más rápido que los candidatos de matriz y 1.66 veces más lento que Ring-LWE; transmite entre 2.3 y 2.6 veces menos bytes que todos ellos, y asigna entre 18 y 20 veces menos memoria que los candidatos de matriz, aunque unas 6 veces más que Ring-LWE.

Frente a ECDSA, es unas 134 veces más lento en este entorno y transmite 18.4 veces más bytes, pero, según las estimaciones, resiste a un adversario cuántico, que rompe ECDSA.

Por tanto, los resultados sugieren que el protocolo es una alternativa poscuántica competitiva en tamaño frente a los demás candidatos de retículos evaluados, con un costo de tiempo intermedio entre ellos. Su comparación de tiempo con ECDSA debe leerse con cautela: ECDSA usa una implementación compilada y madura, y la de ML-ZKAuth es un prototipo en Python sin NTT ni código compilado.

4.21. Amenazas a la validez

Los resultados presentados deben interpretarse considerando las siguientes amenazas a la validez.

4.21.1. Validez interna

Los tiempos de ejecución pueden verse afectados por procesos del sistema operativo, planificación del procesador, cachés y otros factores externos. Para reducir este efecto se utilizaron múltiples repeticiones, iteraciones de calentamiento y un entorno experimental controlado.

Además, se identificaron las siguientes amenazas internas:

- **Implementaciones no equivalentes.** ECDSA usa la biblioteca `cryptography`, que ejecuta código compilado de OpenSSL; ML-ZKAuth usa la misma biblioteca para ML-

KEM y AES, y la aritmética de retículos de los cinco candidatos está escrita en Python con NumPy, sin NTT. Las diferencias de tiempo frente a ECDSA mezclan, por tanto, algoritmo e implementación.

- **Tiempos con mucha varianza.** El número de intentos de los probadores con muestreo por rechazo es aleatorio, de modo que la media de esos candidatos depende de la muestra; por eso se informan también la mediana y los percentiles.
- **Mediciones aisladas.** LWR registró una autenticación de 234 ms, muy por encima del resto de sus mediciones (su percentil 99 es de 29.8 ms); afecta a su media y su desviación, pero no a su mediana.
- **Memoria residente.** El incremento de RSS no capta las asignaciones que reutilizan memoria ya reservada por el proceso, y fue nulo durante la autenticación; por eso la comparación de memoria se basa en el pico de asignaciones de Python, que a su vez no ve toda la memoria nativa de las bibliotecas compiladas.

4.21.2. Validez externa

Los resultados corresponden al hardware, sistema operativo, versiones de software y parámetros descritos en la Sección 4.4. Por esta razón, los valores absolutos pueden variar al utilizar hardware diferente.

Sin embargo, el diseño experimental busca permitir la reproducción del procedimiento en otros entornos.

En particular, los experimentos se ejecutaron en un contenedor sobre la máquina virtual de WSL2 de Windows (Sección 4.4); otro sistema operativo, otro procesador o una implementación en hardware dedicado darían valores absolutos distintos. Además, los tamaños medidos son los de la serialización JSON con Base64; una codificación binaria los reduciría en cerca de un 25 %.

4.21.3. Validez de constructo

Las métricas utilizadas representan diferentes dimensiones de eficiencia y desplegabilidad, pero ninguna métrica individual captura completamente la calidad de un protocolo criptográfico.

En particular, los tiempos de red de las Secciones 4.11 y 4.16 son proyecciones y no mediciones: suman los viajes de ida y vuelta y el tiempo de transmisión de los bytes medidos, sin el establecimiento de la conexión, las cabeceras de TCP, TLS o HTTP, la congestión ni las retransmisiones, por lo que subestiman el tiempo que vería un usuario en una red real.

4.21.4. Limitaciones de la estimación de seguridad

Las estimaciones de seguridad dependen de los modelos de ataque, parámetros y herramientas disponibles al momento de realizar el estudio.

Las estimaciones de la Sección 4.13 se obtuvieron en el modelo de costo Core-SVP con el *lattice-estimator* (*commit* `53da598`), validado contra los tamaños de bloque publicados para Kyber512, Kyber768 y Dilithium2 (Sección 2.3.6). Sus supuestos y limitaciones son:

- Core-SVP cuenta una sola llamada a SVP en la dimensión del bloque: es una cota inferior conservadora para comparar parámetros, no el costo de un ataque completo, y no certifica una categoría de NIST. Las estimaciones refinadas en compuertas y memoria no se calcularon.
- Las instancias de Module-LWE y Ring-LWE se estiman como LWE de la misma dimensión, porque no se conocen ataques que aprovechen la estructura algebraica para estos parámetros.
- La solidez de las pruebas se estima como un problema SIS o MSIS; en el modelo de oráculo aleatorio cuántico, SelfTargetMSIS se toma como supuesto con esa dureza estimada.
- La búsqueda genérica del desafío queda fuera de la herramienta; se acota por el tamaño del espacio de desafíos.
- Las cifras dependen de los modelos de ataque disponibles al momento del estudio y pueden cambiar con nuevos avances en criptoanálisis.

4.22. Reproducibilidad

Con el propósito de facilitar la reproducción de los experimentos, el repositorio del proyecto conserva:

- las implementaciones evaluadas;
- los parámetros criptográficos utilizados;
- los scripts de benchmarking;
- la configuración del entorno de ejecución;
- las versiones de las dependencias;

- las migraciones o estructura de almacenamiento;
- los resultados crudos de cada ejecución;
- los procedimientos utilizados para generar las métricas agregadas.

El experimento correspondiente a este documento fue ejecutado utilizando el commit:

`05a59f7b7e7e05fdb5d6140417e338b23e6b1b3d`

y la configuración experimental `benchmarking/configs/default.json`, cuyo resumen SHA-256 es:

`004fdaa1bf1a817e8819e5cf9ffeb0b83bcd28baad0690f447ca1481eef45daf`

La corrida tiene el identificador `42c5501d-56c8-48d1-b630-695b6d4eb70f`, se ejecutó el 20 de octubre de 2026 entre las 13:50 y las 13:52 UTC sobre un árbol de trabajo sin cambios (`git_dirty = false`), y sus resultados crudos y agregados se exportaron a `benchmark-results/42c5501d-56c8-48d1-b630-695b6d4eb70f/`. Los candidatos se ejecutaron en el orden aleatorio ECDSA, LWR, Binary-LWE, LWE, ML-ZKAuth y Ring-LWE, fijado por la semilla de la configuración.

5. Conclusiones

A partir de los experimentos realizados se identificaron los siguientes hallazgos principales:

1. **Rendimiento:** en el entorno evaluado, los candidatos de retículos autentican entre 80 y 215 veces más lento que ECDSA, entre 5.85 ms (Ring-LWE) y 15.68 ms (LWR), en parte porque se comparan prototipos en Python con una implementación compilada. ML-ZKAuth, con 9.72 ms, queda entre los candidatos sobre anillos y los de matriz.
2. **Memoria:** la estructura algebraica es el factor decisivo. Los candidatos de matriz asignan entre 16.4 y 18.7 MB por autenticación y requieren parámetros públicos de 6.6 a 7.6 MB, mientras que Ring-LWE y ML-ZKAuth asignan menos de 1 MB y sus claves ocupan pocos kilobytes.
3. **Comunicación:** ML-ZKAuth transmite 4 910 bytes por autenticación, entre 2.3 y 2.6 veces menos que los demás candidatos de retículos, aunque 18.4 veces más que ECDSA.
4. **[TODO: hallazgo principal sobre seguridad]**.

5. **[TODO: hallazgo principal sobre despliegue]**.

6. **[TODO: hallazgo principal sobre el protocolo propuesto]**.

En conjunto, los resultados permiten caracterizar las diferentes alternativas no mediante una clasificación única, sino a partir de los compromisos que cada una establece entre seguridad, costo computacional, memoria, comunicación y complejidad operacional.

Esta caracterización proporciona la base experimental para determinar en qué contextos resulta apropiado utilizar cada familia de protocolos y permite evaluar de forma homogénea la propuesta desarrollada en este trabajo.

Referencias

[1] Identity Cybersecurity Share, "Statistics on identity-based attacks and authentication vulnerabilities," 2024, accedido: 2026. [Online]. Available: https://www.linkedin.com/posts/identity-cybersecurity-mfa-share-7423664654884745217-80-G/

[2] FIDO Alliance, "FIDO Alliance specifications and authentication standards," 2024, accedido: 2026. [Online]. Available: https://fidoalliance.org/

[3] V. Mulder, A. Mermoud, V. Lenders, and B. Tellenbach, Eds., *Trends in Data Protection and Encryption Technologies*. Springer, 2023.

[4] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.

[5] S. M. Badiezadegan, "Complexity analysis of representative cryptographic protocols: Zero-knowledge proofs, oblivious transfer, and lattice-based encryption," August 2026, independent Cryptography and Quantum Computing Researcher, Richmond, British Columbia, Canada.

[6] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology — CRYPTO '86*. Springer, 1986, pp. 186–194.

[7] M. Bellare and P. Rogaway, "Random oracles are practical: A paradigm for designing efficient protocols," in *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS '93)*. ACM, 1993, pp. 62–73.

[8] National Institute of Standards and Technology, "SHA-3 standard: Permutation-based hash and extendable-output functions," U.S. Department of Commerce, Federal Information Processing Standards Publication FIPS 202, Aug. 2015.

[9] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.

[10] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018.

[11] P.-L. Cayrel, R. Lindner, R. Silva, and M. Rückert, "Improved zero-knowledge identification with lattices," *Tatra Mountains Mathematical Publications*, vol. 53, no. 1, pp. 33–63, 2012.

[12] V. Lyubashevsky, N. K. Nguyen, and M. Plançon, "Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general," in *Advances in Cryptology — CRYPTO 2022*, ser. Lecture Notes in Computer Science, vol. 13508. Springer, 2022, pp. 71–101.

[13] V. Lyubashevsky, "Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures," in *Advances in Cryptology – ASIACRYPT 2009*, ser. Lecture Notes in Computer Science, vol. 5912. Springer, 2009, pp. 598–616.

[14] ——, "Lattice signatures without trapdoors," in *Eurocrypt 2012*. Springer, 2012, pp. 738–755.

[15] S. Bai, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: Algorithm specifications and supporting documentation (version 3.1)," Especificación de la ronda 3 del proceso de estandarización poscuántica de NIST, Feb. 2021. [Online]. Available: https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf

[16] S. Bai and S. D. Galbraith, "An improved compression technique for signatures based on Learning with Errors," in *Topics in Cryptology – CT-RSA 2014*, ser. Lecture Notes in Computer Science, vol. 8366. Springer, 2014, pp. 28–47.

[17] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," U.S. Department of Commerce, Federal Information Processing Standards Publication FIPS 204, Aug. 2024.

[18] S. Ma, R. Shi, H. Chen, X. Lin, and H. Wang, "LAASSO: Lattice-based anonymous single sign-on system with token hiding," *Journal of Information Security and Applications*, vol. 102, p. 104590, 2026.

[19] C. Baum and A. Nof, "Concretely-efficient zero-knowledge arguments for arithmetic circuits and their application to lattice-based cryptography," in *Public-Key Cryptography – PKC 2020*, ser. Lecture Notes in Computer Science, D. Aggarwal and S. Dziembowski, Eds., vol. 12111. Springer, Cham, 2020, pp. 495–526.

[20] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *Journal of Cryptology*, vol. 13, no. 3, pp. 361–396, 2000.

[21] Q. Liu and M. Zhandry, "Revisiting post-quantum Fiat-Shamir," in *Advances in Cryptology – CRYPTO 2019*, ser. Lecture Notes in Computer Science, A. Boldyreva and D. Micciancio, Eds., vol. 11693. Springer, Cham, 2019, pp. 326–355.

[22] A. Lombardi, E. Mook, W. Quach, and D. Wichs, "Post-quantum insecurity from LWE," Sep. 2022, preprint.

[23] E. Kiltz, V. Lyubashevsky, and C. Schaffner, "A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model," in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10822. Springer, 2018, pp. 552–586. [Online]. Available: https://eprint.iacr.org/2017/916

[24] J. Devevey, P. Fallahpour, A. Passelègue, D. Stehlé, and K. Xagawa, "A detailed analysis of Fiat-Shamir with aborts," in *Advances in Cryptology – CRYPTO 2023*. Springer, 2023. [Online]. Available: https://eprint.iacr.org/2023/245

[25] M. Barbosa, G. Barthe, C. Doczkal, J. Don, S. Fehr, B. Grégoire, Y.-H. Huang, A. Hülsing, Y. Lee, and X. Wu, "Fixing and mechanizing the security proof of Fiat-Shamir with aborts and Dilithium," in *Advances in Cryptology – CRYPTO 2023*. Springer, 2023. [Online]. Available: https://eprint.iacr.org/2023/246

[26] Z. An, J. Pan, Y. Wen, and F. Zhang, "Secret handshakes: Full dynamicity, deniability and lattice-based design," *Theoretical Computer Science*, vol. 940, pp. 14–35, 2023.

[27] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," U.S. Department of Commerce, Federal Information Processing Standards Publication FIPS 203, Aug. 2024.

[28] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: Algorithm specifications and supporting documentation (version 3.02)," Especificación de la ronda 3 del proceso de estandarización poscuántica de NIST, Aug. 2021. [Online]. Available: https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf

[29] M. J. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," National Institute of Standards and Technology, NIST Special Publication 800-38D, Nov. 2007.

[30] R. Cramer and V. Shoup, "Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack," *SIAM Journal on Computing*, vol. 33, no. 1, pp. 167–226, 2003.

[31] J.-J. Huang, Y.-F. Tseng, Q.-L. Yang, and C.-I. Fan, "A lattice-based group authentication scheme," *Applied Sciences*, vol. 8, no. 6, p. 987, 2018.

[32] P. Rewal, M. Singh, D. Mishra, K. Pursharthi, and A. Mishra, "Quantum-safe three-party lattice based authenticated key agreement protocol for mobile devices," *Journal of Information Security and Applications*, vol. 75, p. 103505, 2023.

[33] T. N. Tan and H. Lee, "High-secure fingerprint authentication system using Ring-LWE cryptography," *IEEE Access*, vol. 7, pp. 28 993–29 004, 2019.

[34] L. D. Mukil, M. Kolli, S. M. Rajagopal, and P. B. N., "Implementing lattice-based cryptosystems for post-quantum cryptography: Addressing quantum computing threats," *Procedia Computer Science*, vol. 283, pp. 1645–1655, 2026.

[35] Y. Yang, S. Song, and S. Guo, "Multi-server password authenticated key exchange protocol based on MLWE," in *Proceedings of the 2025 5th International Conference on Computer Network Security and Software Engineering (CNSSE 2025)*. Qingdao, China: Association for Computing Machinery, 2025, pp. 1–14.

[36] O. Regev, "The Learning with Errors problem (invited survey)," in *Proceedings of the 25th Annual IEEE Conference on Computational Complexity (CCC 2010)*, 2010, pp. 191–204. [Online]. Available: https://cims.nyu.edu/~regev/papers/lwesurvey.pdf

[37] C. Peikert, "Lattice cryptography for the internet," in *Post-Quantum Cryptography – PQCrypto 2014*, ser. Lecture Notes in Computer Science, vol. 8772. Springer, 2014, pp. 197–219. [Online]. Available: https://eprint.iacr.org/2014/070

[38] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, "Post-quantum key exchange—a new hope," in *Proceedings of the 25th USENIX Security Symposium*. USENIX Association, 2016, pp. 327–343. [Online]. Available: https://eprint.iacr.org/2015/1092

[39] M. R. Albrecht, R. Player, and S. Scott, "On the concrete hardness of Learning with Errors," *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.

[40] M. R. Albrecht *et al.*, "lattice-estimator: An estimator for the concrete security of lattice problems," Repositorio de software, versión del *commit* 53da598 (19 de agosto de 2026), 2026. [Online]. Available: https://github.com/malb/lattice-estimator

[41] National Institute of Standards and Technology, "Submission requirements and evaluation criteria for the post-quantum cryptography standardization process," Convocatoria del proceso de estandarización poscuántica, sección 4.A.5, Dec. 2016. [Online]. Available: https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf

[42] EMVCo, "EMV integrated circuit card specifications for payment systems, book 2: Security and key management, version 4.3," Especificación técnica, Nov. 2011.

[43] E. Barker, "Recommendation for key management: Part 1 – general," National Institute of Standards and Technology, NIST Special Publication 800-57 Part 1 Rev. 5, May 2020.

[44] National Institute of Standards and Technology, "Digital signature standard (DSS)," U.S. Department of Commerce, Federal Information Processing Standards Publication FIPS 186-5, Feb. 2023.